CS414 ITIMM Lab Exam Guide (Labs 1-12)

Nov 3, 2026 · @Divit

Exam overview and strategy

You draw one of 12 labs: four compulsory tasks worth 2 + 3 + 3 + 2 = 10 marks for performance, plus a 10-mark viva, all in 60 minutes. The paper says marks reward correct implementation, analysis, interpretation and verification, not screenshots alone, so always say what an output means.

Viva rubric (10 marks): conceptual understanding 3 (purpose, architecture, protocol, tool choice), implementation explanation 3 (commands, configuration, data flow, outputs), analysis and troubleshooting 3 (interpret results, faults, limitations, alternatives), technical communication 1 (clear, accurate, justified).

What the paper says about the environment: prepared installations, working services and datasets are supplied, with at least two variants of identifiers, datasets or fault conditions. Nothing large is installed during the exam; remediation tests are safe and controlled. You work independently and submit commands or configurations, screenshots or exported results, calculations and conclusions. The paper says no internet is needed; you told me to ignore the paper's general instructions, so this guide includes install commands as a fallback only, and everything is written to work offline on a prepared machine.

Lab	Topic	Technology	Modules
1	Linux CLI monitoring	Ubuntu CLI, sysstat	M1, M2
2	SNMP device monitoring	SNMP	M3
3	Centralized syslog	rsyslog	M1, M3
4	Network monitoring	Zabbix	M1, M3
5	Network monitoring	LibreNMS	M3
6	Security event monitoring	Wazuh	M3, M6
7	Log analysis	ELK or Grafana Loki	M2, M5
8	Infrastructure monitoring	Prometheus, Grafana	M1, M5
9	SRE golden signals	cURL, Prometheus	M2, M5
10	Container monitoring	Kubernetes, Minikube	M5

Lab	Topic	Technology	Modules
11	Industrial and physical monitoring	Modbus TCP, OPC UA or SNMP	M4
12	Event correlation and automation	Event correlation, Ansible	M6

Time plan (60 min): read the task sheet and verify the service is up (5 min); Tasks 1 and 2 (15 min); Task 3 analysis, with the calculation shown on paper or in a script (15 min); Task 4 configuration or report (10 min); leave the last 15 min for the viva, which is half of the marks. Say each result out loud as you produce it: what it is, why it came out that way, what you would do about it.

Answer formula that fits every viva question: definition, how it works in this lab, what the output proves, what could go wrong or what alternative exists.

Universal pre-flight checklist (first 3 minutes)

1. Read the whole task sheet, note the assigned host name, IP, device, dataset path and credentials.
2. Confirm you are on the right machine: `hostname; whoami; ip -br a`.
3. Confirm the service for your lab is running: `systemctl status <service> --no-pager` or `ss -ltnup | grep <port>`.
4. Make a working folder: `mkdir -p ~/lab && cd ~/lab`, and keep every command output with `| tee out_task1.txt` so you can submit it.
5. Copy supplied datasets before touching them: `cp dataset.* ~/lab/`.
6. Never change production-looking configuration without a backup: `sudo cp file file.bak`.

Environment and install reference (Ubuntu 22.04, fallback only)

Check first (`command -v tool`), install only what is missing. Versions on the exam machine may differ from those assumed here.

```
sudo apt update
# Labs 1, 9: CLI tools
sudo apt install -y sysstat procps curl jq bc gawk
sudo sed -i 's/ENABLED="false"/ENABLED="true"/' /etc/default/sysstat && sudo
systemctl enable --now sysstat
# Labs 2, 5, 11: SNMP client and agent
sudo apt install -y snmp snmpd snmp-mibs-downloader
# Lab 3: rsyslog
sudo apt install -y rsyslog
# Lab 8: Prometheus and Node Exporter from Ubuntu repos
sudo apt install -y prometheus prometheus-node-exporter
# Lab 10: kubectl and Minikube are normally pre-installed; check
kubectl version --client; minikube version
# Lab 11: Python clients for Modbus and OPC UA
pip3 install pymodbus asyncua
# Lab 12: Ansible
sudo apt install -y ansible
```

Zabbix, LibreNMS, Wazuh, ELK or Loki, and Grafana are large stacks: the exam machine will already have them running, so each lab below shows how to find the web address, port and login and how to use them, not how to build them.

General debugging method (all labs)

1. Read the **last line** of an error first; most tools name the exact problem.
2. Check in this order: right host and user, service running, port listening, firewall, credentials, config syntax.
3. Reproduce with the smallest command, change one thing at a time.
4. Use logs: `journalctl -u <service> -n 50 --no-pager`, `/var/log/syslog`, the tool's own log.
5. Explain your reasoning aloud: symptom, hypothesis, check, fix.

Problem	Check	Fix
`command not found`	`command -v tool`, `echo $PATH`	Install the package, or use the full path (`/usr/sbin/...`)
`Permission denied`	`id`, `ls -l`	Use `sudo`, or `chmod +x` a script
Service will not start	`systemctl status X`, `journalctl -u X -n 50`	Fix the config line shown, then `sudo systemctl restart X`

Problem	Check	Fix
Port already in use	`ss -ltnp | grep :PORT`	Stop the other service or change the port
Web UI unreachable	`curl -I http://localhost:PORT`	Service down, wrong port, or firewall: `sudo ufw status`
Works as root but not as user	`sudo -u user command`	Check file permissions and group membership
Time-based tools show nothing	`date`, `timedatectl`	Wrong time zone or clock drift; query a wider time window

Lab 1: Linux infrastructure monitoring using CLI tools

Scenario: check the health of an Ubuntu application server. Marks: Task 1 (2) one-shot snapshot, Task 2 (3) five samples, Task 3 (3) bottleneck analysis, Task 4 (2) health report with two thresholds.

Setup

```
mkdir -p ~/lab1 && cd ~/lab1
command -v top free df uptime vmstat iostat sar    # iostat and sar come from
the sysstat package
sudo apt install -y sysstat                         # only if iostat or sar
is missing
nproc                                               # number of CPU cores:
needed to judge load average
```

Task 1: top, free, df, uptime (CPU, memory, disk, load)

```
uptime | tee t1_uptime.txt                          # load average for 1, 5,
15 minutes
top -bn1 | head -15 | tee t1_top.txt                # batch mode, one
iteration, first 15 lines
free -h | tee t1_free.txt                           # human-readable memory
and swap
df -hT -x tmpfs -x devtmpfs | tee t1_df.txt         # disk space per
filesystem, hide pseudo filesystems
df -i | tee t1_inodes.txt                           # inode usage (a full
inode table also blocks writes)
```

How to read each (say this aloud):

Tool	Where to look	Healthy	Problem sign
`uptime`	`load average: 0.52, 0.48, 0.40`	1-minute load below the number of cores	Load above cores means processes are waiting; rising 1 < 5 < 15 order means worsening, falling means recovering
`top` header `%Cpu(s)`	`us` user, `sy` system, `id` idle, `wa` iowait, `st` stolen (VM)	idle above 20%	`us+sy` above 80% sustained = CPU saturated; `wa` above 10 to 20% = waiting on disk; `st` above 5% = noisy neighbour on the VM host
`top` process list	Press `P` for CPU sort, `M` for memory sort	No process pegged	One process near 100% per core is the culprit
`free -h`	`available` column, not `free`	`available` above 10 to 20% of total	Low available plus used swap means memory pressure (cache is reclaimable, so low `free` alone is normal)
`df -hT`	`Use%`	Below 80%	90% or more: alert; 100% stops writes
`df -i`	`IUse%`	Below 80%	Inode exhaustion with free space left (many tiny files)

Quick extraction of exact numbers for your report:

```
top -bn1 | grep '^%Cpu'                                  # the CPU line
free -m | awk '/^Mem:/ {printf "Memory used %.1f%%, available %d MB of %d
MB\n", ($2-$7)*100/$2, $7, $2}'
df -h --output=source,size,used,pcent,target | sort -k4 -r | head -5    #
fullest filesystems first
awk '{print "Load 1/5/15:", $1, $2, $3, "cores:", "'$(nproc)'"}'
/proc/loadavg
```

Task 2: vmstat, iostat, sar with five samples

The first line of `vmstat` and `iostat` is the average **since boot**, not a live sample. To get five real samples, ask for six and discard the first, or say so explicitly.

```
vmstat 1 6 | tee t2_vmstat.txt                          # 1 s interval, 6 lines:
line 1 is since boot
iostat -xz 1 6 | tee t2_iostat.txt                      # extended stats, skip
idle devices
sar -u 1 5 | tee t2_sar_cpu.txt                         # CPU: five real samples
+ Average line
sar -r 1 5 | tee t2_sar_mem.txt                         # memory
sar -q 1 5 | tee t2_sar_load.txt                        # run queue and load
average
sar -d -p 1 5 | tee t2_sar_disk.txt                     # per-disk activity with
device names
sar -n DEV 1 5 | tee t2_sar_net.txt                     # network interface
rates
```

Columns to explain:

- **vmstat:** `r` runnable processes (above cores = CPU queue), `b` blocked on I/O, `swpd` / `free` / `buff` / `cache` memory in KB, `si` / `so` swap in and out per second (any sustained non-zero = memory pressure), `bi` / `bo` blocks read and written, `in` interrupts, `cs` context switches, `us sy id wa st` CPU percentages.
- **iostat -x:** `r/s`, `w/s` requests per second, `rkB/s`, `wkB/s` throughput, `r_await` / `w_await` average wait in ms (device plus queue), `aqu-sz` average queue length, `%util` busy time (near 100% with rising await = saturated disk; on SSD or NVMe %util can mislead because they serve requests in parallel).
- **sar -u:** `%user %nice %system %iowait %steal %idle`, with an `Average:` line at the end.

Average the five samples yourself when the task asks for a summary:

```
vmstat 1 6 | awk 'NR>3 {us+=$13; sy+=$14; id+=$15; wa+=$16; n++} END {printf
"avg us=%.1f sy=%.1f id=%.1f wa=%.1f over %d samples\n", us/n, sy/n, id/n,
wa/n, n}'
```

Save binary data and replay it later (a favourite for "collect and analyze"):

```
sar -o ~/lab1/sar.dat 1 5 >/dev/null 2>&1           # record 5 samples to a
file
sar -u -f ~/lab1/sar.dat                            # read CPU from the file
sar -u -f /var/log/sysstat/sa$(date +%d)            # today's history
(sysstat must be enabled)
```

Task 3: analyze supplied CPU, memory and disk-I/O measurements

The examiner hands you numbers or output files. Use this decision table first, then confirm with the script below.

Evidence	Conclusion
`us+sy` at or above 80%, `r` well above cores, load above cores, `wa` low	**CPU-bound** (find the process: `top`, `ps aux --sort=-%cpu | head`)
`wa` at or above 20%, `b` greater than 0, high `await`, `%util` near 100%, CPU mostly idle	**Disk I/O-bound** (find the process: `iotop` or `pidstat -d 1`)
`available` memory very low, `si` / `so` greater than 0, high `wa` caused by swapping	**Memory-bound** (find the process: `ps aux --sort=-%mem | head`)
All below thresholds	No bottleneck; healthy
Load average high but CPU idle	Processes stuck in uninterruptible I/O wait (D state), so usually a disk or NFS problem, not CPU

Save as `bottleneck.py` and run `python3 bottleneck.py`. It asks for each measurement, validates the input, and prints the verdict with the numbers that triggered it.

```
#!/usr/bin/env python3
"""Lab 1 bottleneck classifier: type the measurements, get a verdict and
reasons."""
def ask(label, default=None, cast=float):
    while True:
        s = input(f"{label}" + (f" [{default}]" if default is not None else
"") + ": ").strip()
        if s == "" and default is not None:
            return cast(default)
        try:
            v = cast(s)
            if v < 0:
                raise ValueError
            return v
        except ValueError:
            print("  Enter a non-negative number")

cores = ask("CPU cores (nproc)", 4, int)
load1 = ask("Load average 1 min")
us = ask("CPU user %"); sy = ask("CPU system %"); wa = ask("CPU iowait %");
idle = ask("CPU idle %")
if us + sy + wa + idle > 105:
```

```
    print("  Warning: user+sys+iowait+idle is above 100%; check your
numbers")
runq = ask("vmstat r (run queue)", 0); blocked = ask("vmstat b (blocked)", 0)
mem_total = ask("Memory total MB"); mem_avail = ask("Memory available MB")
si = ask("Swap in per second (vmstat si)", 0); so = ask("Swap out per second
(vmstat so)", 0)
util = ask("Disk %util (iostat -x)", 0); await_ms = ask("Disk await ms", 0)

cpu = us + sy
mem_used_pct = 100 * (1 - mem_avail / mem_total) if mem_total else 0
reasons = []
if cpu >= 80 or runq > 2 * cores or load1 > 1.5 * cores:
    reasons.append(("CPU-bound", f"user+sys={cpu:.0f}% | r={runq:.0f} vs
{cores} cores | load {load1:.2f} vs {cores} cores"))
if wa >= 20 or util >= 80 or await_ms >= 50 or (blocked >= 2 and wa >= 10):
    reasons.append(("Disk I/O-bound", f"iowait={wa:.0f}% | %util={util:.0f} |
await={await_ms:.0f} ms | b={blocked:.0f}"))
if mem_used_pct >= 90 or si > 0 or so > 0:
    reasons.append(("Memory-bound", f"used={mem_used_pct:.0f}% | swap in/out=
{si:.0f}/{so:.0f} per s"))

print("\n==== Verdict ====")
if not reasons:
    print("No clear bottleneck: all measures within normal ranges.")
else:
    for name, why in reasons:
        print(f"* {name}: {why}")
    if len(reasons) > 1:
        print("Several symptoms: memory pressure causes swapping, which
causes disk wait; fix memory first." if any(r[0].startswith('Memory') for r
in reasons) and any(r[0].startswith('Disk') for r in reasons) else "Several
symptoms: treat the one with the largest deviation from its threshold as
primary.")
```

Test inputs (type in this order: cores, load1, us, sy, wa, idle, r, b, mem total MB, mem available MB, si, so, %util, await ms). Verified on this script:

Inputs	Expected verdict
`4, 9.5, 85, 10, 0, 5, 12, 0, 8000, 2000, 0, 0, 20, 5`	CPU-bound: user+sys=95%, r=12 vs 4 cores, load 9.50
`4, 3.1, 10, 5, 60, 25, 3, 6, 8000, 3000, 0, 0, 95, 120`	Disk I/O-bound: iowait=60%, %util=95, await=120 ms, b=6

Inputs	Expected verdict
`4, 2.0, 20, 10, 25, 45, 2, 3, 8000, 300, 150, 400, 85, 80`	Disk I/O-bound and Memory-bound; "fix memory first" because swapping causes the disk wait
`4, 0.4, 8, 2, 1, 89, 0, 0, 8000, 5000, 0, 0, 3, 2`	No clear bottleneck
`4, 7, 30, 10, 2, 58, 1, 0, 8000, 5000, 0, 0, 5, 3`	Flagged CPU-bound by the load rule only (user+sys=40%, r=1, load 7.00 vs 4 cores), which is a trap: with idle CPU and high load, say in the viva that high load with idle CPU usually means D-state I/O waiting, so verify with `ps -eo stat,cmd | grep '^D'`
letters or a negative number	Prompt "Enter a non-negative number" and asks again

To analyze **supplied files** instead of typing, pull the numbers out with the awk line from Task 2 (`vmstat` file) or read `Average:` lines in `sar` output (`grep Average t2_sar_cpu.txt`).

Task 4: brief infrastructure health report with two justified thresholds

Use this template and fill the numbers from your own commands. Keep it to half a page.

```
INFRASTRUCTURE HEALTH REPORT
Host: <hostname>   Date/time: <date>   Cores: <nproc>   Prepared by: <name>

1. Snapshot
   Load average (1/5/15): x / y / z  -> <below/above> core count (<n>)
   CPU: us <..>% sy <..>% wa <..>% idle <..>%
   Memory: <..> MB available of <..> MB (<..>% used); swap used <..>
   Disk: <fs> <..>% used; inodes <..>% used
2. Five-sample trend (vmstat/sar): CPU average <..>%, peak <..>%; iowait
average <..>%
3. Finding: <CPU / disk I/O / memory bottleneck, or healthy>, because <two
numbers>
4. Recommended thresholds
   - Warning at <X>, critical at <Y> for <metric>. Justification: <why>
   - Warning at <X>, critical at <Y> for <metric>. Justification: <why>
5. Action: <one concrete step>
```

Two defensible thresholds with reasons:

Metric	Warning / Critical	Justification
CPU utilization (`us+sy` , 5-minute average)	80% / 95%	Above 80% there is little headroom for bursts; using a 5-minute average avoids alerts for short spikes
Load average per core (15-minute)	1.0 / 2.0	Load equal to cores means full use; double means a queue that delays work
Memory available	below 15% / below 5%	Cache is reclaimable, so `available` is the honest figure; low plus swapping means the next allocation may invoke the OOM killer
Swap activity (`si` / `so`)	any sustained above 0 / above 100 pages/s	Swapping turns memory into disk speed, so latency collapses
Disk usage (`Use%`)	80% / 90%	Leaves time to react before writes fail at 100%; large log growth can consume the rest within hours
Disk `await`	above 20 ms / above 50 ms for HDD	Typical HDD service time is under 10 ms, so sustained high await means queueing

The examiner wants **justification**, so tie each threshold to the observed value (for example, "observed CPU peaked at 92%, so 80% would have warned about 20 seconds earlier").

Likely examiner modifications

- Different host or variant dataset: same commands, new numbers; re-run the script with them.
- Ask for **ten** samples or a **2-second** interval: `vmstat 2 11` , `sar -u 2 10` .
- Ask for one process: `top -b -n1 -p <pid>` , `pidstat -p <pid> 1 5` , `ps -p <pid> -o %cpu,%mem,rss,cmd` .
- Ask which process uses the most memory or CPU: `ps aux --sort=-%mem | head -5` , `ps aux --sort=-%cpu | head -5` .
- Ask for disk throughput only: `iostat -dx 1 5` ; per-process I/O: `pidstat -d 1 5` .

- Generate load to demonstrate (safe, short): `timeout 20 yes > /dev/null &` (CPU) then watch `top`; kill with `pkill yes`. For disk: `dd if=/dev/zero of=/tmp/test bs=1M count=500 oflag=dsync`, then `rm /tmp/test`.

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`iostat: command not found` / `sar: command not found`	sysstat not installed	`command -v iostat`	`sudo apt install sysstat`
`Cannot open /var/log/sysstat/saNN: No such file or directory`	History collection disabled or no data for that day	`systemctl status sysstat`, `ls /var/log/sysstat`	Enable collection (`ENABLED="true"` in `/etc/default/sysstat`, then `sudo systemctl restart sysstat`) or use live sampling `sar -u 1 5`
First `vmstat`/`iostat` line looks unrealistic	It is the average since boot	Read the man page note	Discard it, or request one extra sample
`top` output garbled when saved	Interactive mode escape codes	Open the file	Use batch mode `top -bn1`
`free` shows tiny "free" memory and you panic	Linux uses spare RAM as cache	Look at `available`	Use `available`, not `free`
`df` lists dozens of loop or tmpfs lines	Snap and pseudo filesystems	Read the Filesystem column	`df -hT -x tmpfs -x devtmpfs -x squashfs`
High load but CPU idle	Tasks in D state (disk or NFS wait)	`ps -eo stat,pid,cmd | awk '$1 ~ /^D/'`	Find the blocked device; check `iostat -x` and `dmesg`

Symptom or message	Likely cause	Check	Fix
`awk` average is wrong	Wrong column numbers or header rows counted	`vmstat 1 3` and count columns	Skip header rows (`NR>3`) and check that `us` is column 13
`bc: command not found` in a script	Not installed	`command -v bc`	`sudo apt install bc` or use `awk`

Viva theory

Monitoring vs observability. Monitoring collects known metrics against thresholds to tell you that something is wrong; observability (metrics, logs, traces) helps you work out why. This lab is metric-level monitoring of the host.

Load average. The average number of tasks either running or waiting to run (and on Linux, also in uninterruptible I/O sleep) over 1, 5 and 15 minutes. Compare with the core count: load 4 is fully busy on 4 cores but a 2-times overload on 2 cores. It is not CPU percentage.

CPU states. `us` user code, `sy` kernel, `ni` niced user, `id` idle, `wa` idle while waiting for I/O, `hi` / `si` interrupts, `st` time stolen by the hypervisor. High `wa` means the CPU is not the bottleneck; the disk is.

Memory. `free` is unused RAM, `buff/cache` is reclaimable cache, `available` estimates what a new program can get without swapping. Swap is disk used as slow memory; swapping (`si` / `so`) is the sign of real memory shortage. The OOM killer ends a process when memory runs out.

Disk metrics. `%util` is the share of time the device had requests outstanding; `await` is average time per request including queueing; `aqu-sz` is the queue length. Throughput (MB/s) and IOPS must be read with latency, because a disk can be slow either by volume or by saturation.

USE method (Brendan Gregg). For each resource check **U**tilization, **S**aturation and **E**rrors: CPU (utilization %, run queue, none), memory (used, swapping, OOM), disk (%util, queue and await, I/O errors in `dmesg`). It gives a systematic way to find the bottleneck.

How sysstat works. `sadc` collects counters periodically (cron or a systemd timer, normally every 10 minutes), `sar` reads and reports them, `iostat` and `mpstat` are views on the same kernel counters from `/proc` and `/sys` . Interval samples are differences between two counter readings.

Thresholds and baselines. Thresholds should come from a baseline of normal behaviour plus a safety margin, use averages over time to avoid false alarms, and have warning and critical levels so that action starts early. Too many false alerts cause alert fatigue.

Typical viva questions: Why is the first vmstat line different? (Since-boot average.) Why does free memory look low on a healthy server? (Page cache.) Load is 8 on a 4-core machine, is the CPU necessarily at 100%? (No: could be I/O wait.) What does iowait mean? (CPU idle while a request is outstanding; not a CPU shortage.) Difference between `top` and `vmstat`? (Interactive per-process view vs system-wide counters.) Why use `df -i`? (Inode exhaustion.) Why average over five samples? (Reduce noise from one-second spikes.) Name an alternative tool. (`htop`, `atop`, `glances`, `nmon`, `mpstat`, `pidstat`, `iotop`.)

Lab 2: SNMP-based network device monitoring

Scenario: monitor a simulated switch or an SNMP-enabled Linux server. Marks: Task 1 (2) `snmpget` of sysName, sysDescr, sysUpTime; Task 2 (3) `snmpwalk` of interface status, octet counters, interface info; Task 3 (3) throughput and percentage utilization from two counter samples; Task 4 (2) one abnormal interface condition plus an alert threshold.

Setup

```
mkdir -p ~/lab2 && cd ~/lab2
command -v snmpget snmpwalk snmptable           # client tools (package:
snmp)
sudo apt install -y snmp snmp-mibs-downloader    # only if missing; the MIB
package may need multiverse
# enable symbolic names such as sysName.0 and ifDescr (comment out the 'mibs
:' line):
sudo sed -i 's/^mibs :/# mibs :/' /etc/snmp/snmp.conf
export HOST=192.168.1.10 COMM=public             # use the host and community
on your task sheet
```

If the examiner gives a port other than 161 (for example an SNMP simulator on 1161), write the target as `HOST:1161` or `udp:HOST:1161`. If symbolic names fail (`Unknown Object Identifier`), use the numeric OIDs shown below; they always work.

Making your own Linux server the SNMP device (only if the device is not supplied):

```
sudo apt install -y snmpd
sudo cp /etc/snmp/snmpd.conf /etc/snmp/snmpd.conf.bak
# edit /etc/snmp/snmpd.conf: listen on all addresses and remove the
restrictive view
#   agentaddress udp:161
#   rocommunity public default           (replace the line that ends with -V
systemonly)
sudo systemctl restart snmpd && systemctl status snmpd --no-pager
sudo ss -lunp | grep 161                          # confirm UDP 161 is open
snmpget -v2c -c public localhost sysName.0        # self-test
```

Task 1: verify connectivity and retrieve sysName, sysDescr, sysUpTime

```
snmpget -v2c -c $COMM -t 2 -r 1 $HOST sysName.0 sysDescr.0 sysUpTime.0 | tee
t1.txt
# the same with numeric OIDs (works without MIB files):
snmpget -v2c -c $COMM $HOST 1.3.6.1.2.1.1.5.0 1.3.6.1.2.1.1.1.0
1.3.6.1.2.1.1.3.0
# values only, for the report:
snmpget -v2c -c $COMM -Oqv $HOST sysName.0 sysDescr.0
```

Object	OID	Meaning
sysDescr	1.3.6.1.2.1.1.1.0	Text description of the device and software
sysObjectID	1.3.6.1.2.1.1.2.0	Vendor identity OID
sysUpTime	1.3.6.1.2.1.1.3.0	Time since the SNMP agent restarted, in **timeticks** (hundredths of a second)
sysContact	1.3.6.1.2.1.1.4.0	Administrator contact
sysName	1.3.6.1.2.1.1.5.0	Hostname of the device
sysLocation	1.3.6.1.2.1.1.6.0	Physical location

Convert uptime: `Timeticks: (123456789) 14 days, 6:56:07.89` means 123,456,789 / 100 / 86,400 = 14.29 days. Connectivity is verified when a value comes back: a timeout means no SNMP response (see Common errors).

Other versions (say these if asked; the examiner may supply v1 or v3 details):

```
snmpget -v1  -c $COMM $HOST sysName.0
snmpget -v3 -l authPriv -u snmpuser -a SHA -A 'authpass123' -x AES -X
'privpass123' $HOST sysName.0
# create a v3 user on your own snmpd:
sudo systemctl stop snmpd
sudo net-snmp-create-v3-user -ro -A authpass123 -a SHA -X privpass123 -x AES
snmpuser
sudo systemctl start snmpd
```

Task 2: snmpwalk for interface status, octet counters and interface information

```
snmpwalk -v2c -c $COMM $HOST ifDescr          | tee t2_ifdescr.txt
snmpwalk -v2c -c $COMM $HOST ifOperStatus     | tee t2_oper.txt      # 1 up,
2 down, 3 testing
snmpwalk -v2c -c $COMM $HOST ifAdminStatus    | tee t2_admin.txt
snmpwalk -v2c -c $COMM $HOST ifInOctets       | tee t2_in.txt
snmpwalk -v2c -c $COMM $HOST ifOutOctets      | tee t2_out.txt
snmpwalk -v2c -c $COMM $HOST ifSpeed          | tee t2_speed.txt
snmpwalk -v2c -c $COMM $HOST ifInErrors       | tee t2_inerr.txt
snmpwalk -v2c -c $COMM $HOST ifOutErrors
snmpwalk -v2c -c $COMM $HOST ifInDiscards
# the whole interface table in one readable table (best for screenshots):
snmptable -v2c -c $COMM -Cb -CH $HOST ifTable | tee t2_table.txt
# faster on big devices (v2c and v3 only):
snmpbulkwalk -v2c -c $COMM $HOST ifDescr
# numeric equivalents if names are not available:
snmpwalk -v2c -c $COMM $HOST 1.3.6.1.2.1.2.2.1.2     # ifDescr
```

The interface table is "MIB-II / IF-MIB" under `1.3.6.1.2.1.2.2.1`. The last number of each OID is the **ifIndex**, so `ifOperStatus.2` belongs to the interface with ifIndex 2:

Column	OID	Meaning
ifIndex	.2.2.1.1	Interface number
ifDescr	.2.2.1.2	Name (eth0, GigabitEthernet0/1)
ifType	.2.2.1.3	Type (6 = Ethernet, 24 = loopback)
ifMtu	.2.2.1.4	Maximum frame size
ifSpeed	.2.2.1.5	Speed in bits/s (capped at 4,294,967,295, so use ifHighSpeed above 4 Gbps)
ifPhysAddress	.2.2.1.6	MAC address

Column	OID	Meaning
ifAdminStatus	.2.2.1.7	Configured state: 1 up, 2 down, 3 testing
ifOperStatus	.2.2.1.8	Actual state: 1 up, 2 down, 3 testing, 7 lowerLayerDown
ifInOctets	.2.2.1.10	Bytes received (32-bit counter)
ifInDiscards / ifInErrors	.2.2.1.13 / .14	Received packets dropped, received packets with errors
ifOutOctets	.2.2.1.16	Bytes sent (32-bit counter)
ifOutDiscards / ifOutErrors	.2.2.1.19 / .20	Sent packets dropped, send errors
ifHCInOctets / ifHCOutOctets	1.3.6.1.2.1.31.1.1.1.6 / .10	64-bit byte counters for fast links
ifHighSpeed	1.3.6.1.2.1.31.1.1.1.15	Speed in Mbit/s
ifAlias	1.3.6.1.2.1.31.1.1.1.18	Description set by the admin

A quick one-line summary of every interface, built with `paste` (an interface that is administratively up but operationally down stands out immediately):

```
paste <(snmpwalk -v2c -c $COMM -Oqv $HOST ifDescr) <(snmpwalk -v2c -c $COMM -Oqv $HOST ifAdminStatus) <(snmpwalk -v2c -c $COMM -Oqv $HOST ifOperStatus)
```

Task 3: throughput and percentage utilization from two counter samples

SNMP counters only ever count up, so rate is the **difference between two samples divided by the time between them**:

$$\text{bits/s} = \frac{(C_2 - C_1) \times 8}{\Delta t}, \qquad \text{Utilization}\,\% = \frac{\text{bits/s}}{\text{ifSpeed}} \times 100$$

- **Counter wrap:** a 32-bit counter returns to 0 after 4,294,967,295. If `C2 < C1`, use `C2 + 2^32 - C1`. At 100 Mbps a 32-bit octet counter wraps in about 5.7 minutes, so poll often or use the 64-bit `ifHC` counters.
- **Time:** use the `sysUpTime` difference (ticks / 100) for the exact interval rather than the `sleep` time.
- **Duplex:** for full-duplex links report inbound and outbound separately (utilization of the busiest direction); for half-duplex use (in + out) / speed.

- **Worked example:** in1 = 1,000,000,000, in2 = 1,150,000,000, interval 30 s, ifSpeed 100,000,000. Delta = 150,000,000 bytes; 150,000,000 x 8 / 30 = 40,000,000 bit/s = 40 Mbps; 40 / 100 = **40% utilization**.

Manual two-sample collection (record the sysUpTime too):

```
IDX=2                                                    # ifIndex of the
interface from Task 2
snmpget -v2c -c $COMM $HOST sysUpTime.0 ifInOctets.$IDX ifOutOctets.$IDX
ifSpeed.$IDX | tee s1.txt
sleep 30
snmpget -v2c -c $COMM $HOST sysUpTime.0 ifInOctets.$IDX ifOutOctets.$IDX
ifSpeed.$IDX | tee s2.txt
```

Interactive calculator, `ifrate.py`: type the counters from `s1.txt` and `s2.txt`; it handles wrap and prints in, out and utilization.

```
#!/usr/bin/env python3
"""Lab 2: interface throughput and utilization from two SNMP counter
samples."""
def num(label, default=None, allow_zero=True):
    while True:
        s = input(label + (f" [{default}]" if default is not None else "") +
": ").strip()
        if s == "" and default is not None:
            return float(default)
        try:
            v = float(s.replace(",", ""))
            if v < 0 or (v == 0 and not allow_zero):
                raise ValueError
            return v
        except ValueError:
            print("  Enter a valid non-negative number" + ("" if allow_zero
else " greater than 0"))

def delta(c1, c2, bits):
    return (c2 - c1) % (2 ** bits)          # handles counter wrap

def fmt(bps):
    for unit, d in (("Gbps", 1e9), ("Mbps", 1e6), ("kbps", 1e3)):
        if bps >= d:
            return f"{bps / d:.2f} {unit}"
    return f"{bps:.0f} bps"

bits = 64 if input("Counter size 32 or 64 bit (ifInOctets=32,
ifHCInOctets=64) [32]: ").strip() == "64" else 32
```

```
in1 = num("ifInOctets  sample 1"); in2 = num("ifInOctets  sample 2")
out1 = num("ifOutOctets sample 1"); out2 = num("ifOutOctets sample 2")
secs = num("Seconds between samples (or sysUpTime ticks diff / 100)",
allow_zero=False)
speed = num("Link speed in bits per second (ifSpeed, e.g. 100000000)",
allow_zero=False)
din, dout = delta(in1, in2, bits), delta(out1, out2, bits)
if in2 < in1: print(f"  Note: inbound counter wrapped ({bits}-bit); corrected
by modulo 2^{bits}")
if out2 < out1: print(f"  Note: outbound counter wrapped ({bits}-bit);
corrected by modulo 2^{bits}")
bin_, bout = din * 8 / secs, dout * 8 / secs
print(f"\nInbound : {din:.0f} bytes in {secs:g}s = {fmt(bin_)}  ({100 * bin_
/ speed:.2f}% of link)")
print(f"Outbound: {dout:.0f} bytes in {secs:g}s = {fmt(bout)}  ({100 * bout /
speed:.2f}% of link)")
u = 100 * max(bin_, bout) / speed
print(f"Utilization, full-duplex (busiest direction): {u:.2f}%")
print(f"Utilization, half-duplex ((in+out)/speed)  : {100 * (bin_ + bout) /
speed:.2f}%")
print("Status:", "CRITICAL (>= 90%)" if u >= 90 else "WARNING (>= 80%)" if u
>= 80 else "OK (< 80%)")
```

Test inputs (type in this order: size, in1, in2, out1, out2, seconds, speed). Verified on this script:

Inputs	Expected result
`32, 1000000000, 1150000000, 500000000, 530000000, 30, 100000000`	In 40.00 Mbps (40.00%), out 8.00 Mbps (8.00%), full-duplex 40.00%, half-duplex 48.00%, OK
`32, 4294967000, 500, 0, 0, 10, 100000000`	"inbound counter wrapped"; delta 796 bytes = 637 bps, so wrap handling works
`32, 0, 1125000000, 0, 100, 10, 1000000000`	In 900.00 Mbps = 90.00% of 1 Gbps: CRITICAL (>= 90%)
`32, 0, 1000000000, 0, 0, 10, 1000000000`	In 800.00 Mbps = 80.00%: WARNING
`64, 0, 11250000000, 0, 100, 10, 10000000000`	64-bit counters on 10 Gbps: 9 Gbps, 90%: CRITICAL
letters, a negative number, or `0` seconds or speed	Asks again with "Enter a valid non-negative number"

If the examiner supplies **two `snmpwalk` outputs** for all interfaces, extract the counters with `awk` and loop, for example:

```
awk '{print $1, $NF}' w1_in.txt > a.txt; awk '{print $NF}' w2_in.txt > b.txt
# then feed values to ifrate.py
```

Fully automatic collector, `ifrate.sh` (two samples, real interval from sysUpTime, wrap-safe). Run `./ifrate.sh HOST COMMUNITY IFINDEX [SECONDS]`.

```
#!/bin/bash
H=${1:?host}; C=${2:?community}; I=${3:?ifIndex}; W=${4:-10}
IN=1.3.6.1.2.1.2.2.1.10.$I; OUT=1.3.6.1.2.1.2.2.1.16.$I;
SP=1.3.6.1.2.1.2.2.1.5.$I; UP=1.3.6.1.2.1.1.3.0
g()   { snmpget -v2c -c "$C" -t 2 -r 1 "$H" "$1" 2>/dev/null; }
val() { g "$1" | awk '{print $NF}'; }
tk()  { g $UP | sed -E 's/.*\(([0-9]+)\).*/\1/'; }
[ -n "$(val $UP)" ] || { echo "No SNMP response from $H (check host,
community, UDP 161)"; exit 1; }
t1=$(tk); i1=$(val $IN); o1=$(val $OUT)
sleep "$W"
t2=$(tk); i2=$(val $IN); o2=$(val $OUT); sp=$(val $SP)
awk -v t1=$t1 -v t2=$t2 -v i1=$i1 -v i2=$i2 -v o1=$o1 -v o2=$o2 -v sp=$sp
'BEGIN {
  m = 4294967296; s = (t2 - t1) / 100
  if (s <= 0) { print "Time did not advance: check sysUpTime values"; exit 1
}
  di = (i2 - i1 + m) % m; do_ = (o2 - o1 + m) % m
  bi = di * 8 / s; bo = do_ * 8 / s
  printf "interval %.2f s | in %.2f Mbps (%.2f%%) | out %.2f Mbps
(%.2f%%)\n", s, bi/1e6, 100*bi/sp, bo/1e6, 100*bo/sp
  u = (bi > bo ? bi : bo) * 100 / sp
  printf "utilization (busiest direction) %.2f%% -> %s\n", u, (u >= 90 ?
"CRITICAL" : u >= 80 ? "WARNING" : "OK")
}'
```

I checked the calculation block and the parsing lines with sample numbers; I could not run the SNMP calls themselves here, so run it once against your device and compare with `ifrate.py` before the exam.

Task 4: identify an abnormal interface condition and recommend an alert threshold

Work through this checklist on the data you collected; pick the condition that the data shows and quote the numbers.

Condition	Evidence in the data	Likely cause	Recommended alert
Link down	`ifAdminStatus = 1` but `ifOperStatus = 2`	Cable, remote port, or SFP failure	Alert when oper status is not up for 2 consecutive polls (avoids a single missed poll)
High utilization	Utilization at or above 80%	Backup job, loop, DoS, undersized link	Warning at 80%, critical at 90%, averaged over 5 minutes
Errors rising	`ifInErrors` delta above 0 each interval; error rate = delta errors / delta packets	Bad cable, duplex mismatch, failing NIC	Alert when error rate exceeds 0.1% (or more than 10 errors per minute)
Discards rising	`ifOutDiscards` delta above 0	Congestion: queues full	Alert when discards are non-zero for 3 polls in a row
Flapping	`ifLastChange` timestamp keeps changing, `sysUpTime` small	Unstable link, loop, spanning-tree	Alert on more than 3 state changes in 10 minutes
Zero traffic on an "up" link	Counters do not move between samples	Remote side silent, VLAN or routing fault	Alert when the rate is 0 for 10 minutes on a normally busy port
Recent reboot	`sysUpTime` smaller than before; counters reset	Device restarted	Alert when sysUpTime decreases

In the report give: the condition, the measured evidence, the threshold you recommend, and why (for example "warning at 80% averaged over 5 minutes because brief bursts above 80% are normal for a switch but sustained use leaves no headroom"). Error-rate calculation: `awk 'BEGIN{printf "%.3f%%\n", 25*100/50000}'` gives 0.050%.

Likely examiner modifications

- Different community, port, SNMP version or an interface index: adjust the variables; run `snmpwalk ... ifDescr` to map names to indexes.
- "Use numeric OIDs only" or "use names only": both are shown above.
- "Use 64-bit counters": replace `ifInOctets.$IDX` with `ifHCInOctets.$IDX` (`1.3.6.1.2.1.31.1.1.1.6.$IDX`) and answer 64 at the counter prompt.

- "Show only interfaces that are up": `snmpwalk -v2c -c $COMM $HOST ifOperStatus | grep 'up(1)'`.
- "Get one interface": `snmpget -v2c -c $COMM $HOST ifDescr.2 ifOperStatus.2 ifInOctets.2`.
- "Find the interface with the most traffic": run the collector for each ifIndex in a loop and sort by utilization.
- Walk a subtree from a number: `snmpwalk -v2c -c $COMM $HOST 1.3.6.1.2.1.2`.
- Show the raw OID/type: `snmpwalk -On ...` (numeric OIDs), `snmpget -Ov ...` (value only).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`Timeout: No Response from HOST`	Wrong IP, wrong community, snmpd not running, firewall blocking UDP 161, or snmpd listens on localhost only (Ubuntu default)	`ping HOST`; `sudo ss -lunp | grep 161` on the device; `sudo ufw status`	Correct the community and host; set `agentaddress udp:161`; allow UDP 161; use `-t 5 -r 3` on slow links
`snmpwalk: Unknown Object Identifier` / `Cannot find module`	MIB files not loaded	`ls /usr/share/snmp/mibs`	Install `snmp-mibs-downloader` and comment out `mibs :` in `/etc/snmp/snmp.conf`, or use numeric OIDs
`No Such Object available on this agent at this OID`	Missing `.0` on a scalar, wrong index, or the device does not support the object	Retry with `snmpwalk` on the parent OID	Use `sysName.0`, not `sysName`; check ifIndex with `ifDescr`

Symptom or message	Likely cause	Check	Fix
`No more variables left in this MIB View (It is past the end of the MIB tree)`	The community's view hides that subtree (Ubuntu's `-V systemonly`)	Read the `rocommunity` line in `/etc/snmp/snmpd.conf`	Remove `-V systemonly` (or add `view all included .1`), restart snmpd
`Authentication failure` / `Timeout` with v3	Wrong user, passwords, or protocols	Re-run with `-d` for packet debug	Match `-a`, `-A`, `-x`, `-X` and the security level exactly
Counter goes backwards between samples	32-bit wrap or device reboot	Compare `sysUpTime`	Use modulo 2^32, or ignore the interval after a reboot
Utilization above 100%	Wrong ifSpeed (value in Mbps instead of bps), duplicated interval, wrap not handled	Print the raw numbers	Use bits per second, check interval, use `ifHCOctets`
`ifSpeed` is 4294967295	Link faster than 4 Gbps	`snmpget ... ifHighSpeed.IDX`	Use ifHighSpeed (Mbps) times 1,000,000
Different output each run for the same interface	ifIndex changed after a reboot	Re-walk ifDescr	Select interfaces by name, not by remembered number

Viva theory

SNMP (Simple Network Management Protocol). An application-layer protocol over **UDP port 161** (queries) and **162** (traps) that lets a manager (NMS) read and set variables on an **agent** running on a managed device. It is the standard way to monitor switches, routers, servers and printers.

Architecture. NMS (manager, such as Zabbix or LibreNMS) polls the agent. The agent exposes a **MIB** (Management Information Base): a tree of objects identified by **OIDs**. Operations: `GET`, `GETNEXT`, `GETBULK` (v2c/v3), `SET`, and unsolicited **traps/INFORMs** from the agent to the manager.

OID tree. `iso(1).org(3).dod(6).internet(1)`, so everything standard starts with `1.3.6.1`. `mgmt.mib-2` is `1.3.6.1.2.1` (system = `.1`, interfaces = `.2`); vendor-specific data sits under `private.enterprises` = `1.3.6.1.4.1`.

Versions. v1: 32-bit counters, community string, no encryption. v2c: adds GETBULK and 64-bit counters, still plain-text community. v3: adds authentication (MD5/SHA), encryption (DES/AES) and access control (USM and VACM); security levels `noAuthNoPriv`, `authNoPriv`, `authPriv`. Community strings travel in clear text, so v1 and v2c are only safe on trusted networks.

Counters versus gauges. A **Counter** only increases and wraps (rates need two samples); a **Gauge** is an instantaneous value (CPU load); **TimeTicks** are hundredths of a second.

Polling vs traps. Polling gives regular data and trends but adds load and can miss short events; traps report events immediately but are UDP (can be lost) and carry no history. Real systems use both.

Why not rely on ifOperStatus alone? It shows state, not quality. Errors, discards, utilization and flapping reveal degraded links that are still "up".

Typical viva questions: What port does SNMP use? (UDP 161 and 162.) Difference between `snmpget`, `snmpwalk`, `snmpgetnext`, `snmpbulkwalk`? (One OID; walk a subtree with repeated GETNEXT; next OID; walk with GETBULK, faster.) Why compute rates from two samples? (Counters are cumulative.) What if the counter wraps? (Add 2^32.) Why prefer v3? (Authentication and encryption.) What is a MIB? (The definition of the object tree, mapping names to OIDs.) What is ifIndex? (Interface number used as the table index.) Why 64-bit counters? (A 32-bit octet counter wraps in under 6 minutes at 100 Mbps and about 34 seconds at 1 Gbps.) What is a community string? (A shared password, `public` for read-only by default, a security risk if left default.)

Lab 3: Centralized syslog monitoring and analysis

Scenario: investigate authentication and service events from several Linux hosts. Marks: Task 1 (2) verify an existing rsyslog forwarding configuration; Task 2 (3) generate test events with `logger` and confirm arrival on the log server; Task 3 (3) analyze a supplied log dataset for repeated authentication failures, severity and affected hosts; Task 4 (2) create a log-filtering rule or alert condition and demonstrate it.

I tested the rsyslog configurations in this section on a real rsyslog 8.2312 server and forwarding client (Ubuntu 22.04 ships 8.2112; the syntax used here is the same), and tested the analysis and alert scripts on sample data.

Setup

```
mkdir -p ~/lab3 && cd ~/lab3
rsyslogd -v | head -1                    # version
systemctl status rsyslog --no-pager      # running?
sudo apt install -y rsyslog              # only if missing
export SERVER=192.168.1.10               # log server IP from your task sheet
```

Reference: how the pieces are normally configured (you verify these, you usually do not create them).

On the **log server** (`/etc/rsyslog.d/10-server.conf`): receive over UDP and TCP port 514 and store each host's logs in its own folder.

```
module(load="imudp")
input(type="imudp" port="514")
module(load="imtcp")
input(type="imtcp" port="514")
template(name="PerHostLog" type="string"
string="/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log")
if $fromhost-ip != '127.0.0.1' then ?PerHostLog
& stop
```

```
sudo mkdir -p /var/log/remote && sudo chown syslog:adm /var/log/remote
sudo rsyslogd -N1 && sudo systemctl restart rsyslog
sudo ss -tulnp | grep ':514'             # both udp and tcp 514 should be
listening
sudo ufw allow 514/tcp; sudo ufw allow 514/udp     # if ufw is active
```

On each **client** (`/etc/rsyslog.d/50-forward.conf`): forward everything to the server. One `@` means UDP, two `@@` mean TCP.

```
*.* @@192.168.1.10:514
```

The more robust form (tested): TCP with a disk-backed queue so messages survive a server outage.

```
*.* action(type="omfwd" target="192.168.1.10" port="514" protocol="tcp"
action.resumeRetryCount="-1" queue.type="LinkedList" queue.size="10000"
queue.filename="fwd_q" queue.saveOnShutdown="on")
```

Task 1: verify the forwarding configuration between client and server

On the **client**:

```
grep -rnE '^[^#]*@' /etc/rsyslog.conf /etc/rsyslog.d/ | tee t1_client_cfg.txt
# @ or @@ lines = forwarding rules
grep -rn 'omfwd' /etc/rsyslog.conf /etc/rsyslog.d/
# the action() form
sudo rsyslogd -N1                       # syntax check: must end with "End of
config validation run" and no errors
systemctl is-active rsyslog             # active
nc -vz $SERVER 514                      # TCP reachability (succeeded = open)
nc -vzu $SERVER 514                     # UDP check (weak: UDP has no
handshake)
sudo ss -tnp | grep ':514'              # an ESTABLISHED TCP connection to
the server proves forwarding is live
```

On the **server**:

```
sudo ss -tulnp | grep -E ':514'         # listening sockets
ls -l /var/log/remote/                  # one folder per client host once
messages arrive
sudo tail -n 20 /var/log/syslog         # server's own log, shows rsyslog
start-up or errors
```

What to state: the protocol (`@` UDP vs `@@` TCP), the target IP and port, the selector (`*.*` = every facility and severity), that the configuration validates, and that the service is active.

Task 2: generate test events with logger and confirm arrival

A `logger` message passes through the **local** rsyslog, which forwards it. Use a unique marker so you can find it.

```
MARK="LAB3-$(date +%s)"
logger -t labtest -p local0.warning "$MARK warning from $(hostname)"
logger -t labtest -p auth.err      "$MARK auth error from $(hostname)"
logger -t sshd    -p authpriv.info "Failed password for invalid user testuser
from 203.0.113.50 port 4444 ssh2"
echo $MARK
# check locally first:
grep "$MARK" /var/log/syslog | tail -3
```

On the **server**:

```
sudo grep -r "LAB3-" /var/log/remote/ /var/log/syslog | tail -5
sudo tail -f /var/log/remote/<client-hostname>/labtest.log          # live
view
```

Send directly to a server without relying on forwarding (useful to separate "client forwarding problem" from "server receiving problem"):

```
logger -n $SERVER -P 514 -T -t labtest "$MARK direct over TCP"      # -T TCP
logger -n $SERVER -P 514 -d -t labtest "$MARK direct over UDP"      # -d UDP
```

In my test both the TCP and UDP messages arrived and created `remote/<host>/labtest.log` with lines like `2026-11-03T16:54:55+00:00 vm labtest TEST-TCP-001 hello from client`.

Facility.severity	Meaning
Facilities	`kern` 0 `user` 1 `mail` 2 `daemon` 3 `auth` 4 `syslog` 5 `lpr` 6 `news` 7 `uucp` 8 `cron` 9 `authpriv` 10 `local0` to `local7` 16 to 23
Severities	`emerg` 0 `alert` 1 `crit` 2 `err` 3 `warning` 4 `notice` 5 `info` 6 `debug` 7 (lower number = more severe)
PRI value	facility x 8 + severity, e.g. `<86>` = authpriv(10) x 8 + info(6); `<38>` = auth(4) x 8 + info(6); `<27>` = daemon(3) x 8 + err(3)

Task 3: analyze a supplied log dataset

The dataset is usually `auth.log` or a collection of per-host files. First try the quick shell analysis, then the script.

```
LOG=auth.log                                                    # the supplied file
wc -l $LOG                                                      # size
grep -c 'Failed password' $LOG                                  # number of failed
SSH passwords
grep 'Failed password' $LOG | grep -oE 'from ([0-9]{1,3}\.){3}[0-9]{1,3}' |
awk '{print $2}' | sort | uniq -c | sort -rn   # top source IPs
grep 'Failed password' $LOG | grep -oP 'for (invalid user )?\K\S+' | sort |
uniq -c | sort -rn                                # targeted users
grep 'Failed password' $LOG | awk '{print $4}' | sort | uniq -c | sort -rn
# hosts (traditional format: field 4)
grep 'Failed password' $LOG | awk '{print $1,$2,substr($3,1,5)}' | uniq -c
# failures per minute
grep -E 'Accepted (password|publickey)' $LOG | tail
# successes: check for one right after failures
```

If the log has a priority prefix (`<86>`), strip it before using field numbers: `sed -E 's/^<[0-9]+>//' $LOG` . If the files are `@@` forwarded into folders, analyze `cat /var/log/remote/*/sshd.log` .

Save `logscan.py` . It reads the file (or asks for the path), counts failures per host, source IP and user, decodes severity from the `<PRI>` prefix, finds brute-force sources (N failures within M minutes) and flags a **successful login that followed failures from the same IP**.

```
#!/usr/bin/env python3
"""Lab 3: analyse an auth/syslog file for failed logins, hosts, IPs, severity
and brute force.
Usage: python3 logscan.py [FILE] [--threshold N] [--window MIN]   (prompts if
FILE is omitted)"""
import argparse, re, sys
from collections import Counter, defaultdict, deque
from datetime import datetime

SEV = ["emerg", "alert", "crit", "err", "warning", "notice", "info", "debug"]
LINE = re.compile(r"^(?:<(?P<pri>\d{1,3})>)?(?P<ts>[A-Z][a-z]{2}\s+\d+\s[\d:]
{8})\s(?P<host>\S+)\s(?P<prog>[^\s:\[]+)(?:\[\d+\])?:\s(?P<msg>.*)$")
FAIL = [re.compile(p) for p in (
    r"Failed password for (?:invalid user )?(?P<user>\S+) from (?P<ip>
[\d.]+)",
    r"Invalid user (?P<user>\S+) from (?P<ip>[\d.]+)",
    r"authentication failure.*?(?:rhost=(?P<ip>[\d.]*)).*?user=(?
P<user>\S+)",
    r"password authentication failed for user \"(?P<user>[^\"]+)\"")]
OK = re.compile(r"Accepted (?:password|publickey) for (?P<user>\S+) from (?
P<ip>[\d.]+)")

ap = argparse.ArgumentParser()
ap.add_argument("file", nargs="?"); ap.add_argument("--threshold", type=int);
ap.add_argument("--window", type=int)
a = ap.parse_args()
path = a.file or input("Log file path: ").strip()
thr = a.threshold or int(input("Failures that count as brute force [5]: ") or
5)
win = a.window or int(input("Window in minutes [10]: ") or 10)
try:
    lines = open(path, errors="replace").read().splitlines()
except OSError as e:
    sys.exit(f"Cannot read {path}: {e}")

fails, byhost, byip, byuser, sevs = [], Counter(), Counter(), Counter(),
Counter()
okevents, bad = [], 0
```

```
for ln in lines:
    m = LINE.match(ln)
    if not m:
        bad += 1; continue
    d = m.groupdict()
    ts = datetime.strptime(f"{datetime.now().year} {'
'.join(d['ts'].split())}", "%Y %b %d %H:%M:%S")
    sev = SEV[int(d["pri"]) % 8] if d["pri"] else None
    for rx in FAIL:
        f = rx.search(d["msg"])
        if f:
            ip = f.groupdict().get("ip") or f"local@{d['host']}"      # no
source address: local login on that host
            user = f.groupdict().get("user") or "?"
            fails.append((ts, d["host"], ip, user, sev)); byhost[d["host"]]
+= 1; byip[ip] += 1; byuser[user] += 1
            if sev: sevs[sev] += 1
            break
    o = OK.search(d["msg"])
    if o:
        okevents.append((ts, d["host"], o["ip"], o["user"]))

print(f"Lines read: {len(lines)} (unparsed: {bad}) | authentication failures:
{len(fails)}")
print("\nFailures per host:");  [print(f"  {c:4d}  {h}") for h, c in
byhost.most_common()]
print("Failures per source IP:"); [print(f"  {c:4d}  {i}") for i, c in
byip.most_common()]
print("Failures per user:"); [print(f"  {c:4d}  {u}") for u, c in
byuser.most_common()]
if sevs:
    print("Severity of the failure lines (decoded from the <PRI> prefix):",
dict(sevs))
else:
    print("Severity: no <PRI> prefix in this file, so it cannot be decoded.
Note sshd logs failed logins at 'info' by default.")

print(f"\nBrute-force suspects (>= {thr} failures within {win} min):")
by_src = defaultdict(list)
for t, h, ip, u, s in fails: by_src[ip].append((t, h))
found = False
for ip, ev in by_src.items():
    q = deque()
    for t, h in ev:
        q.append(t)
        while (t - q[0]).total_seconds() > win * 60: q.popleft()
```

```
        if len(q) >= thr:
            hosts = sorted({x[1] for x in ev}); found = True
            print(f"  {ip}: {len(ev)} failures on {', '.join(hosts)}"); break
if not found: print("  none")

print("\nSuccessful logins that followed failures from the same IP (possible
compromise):")
hit = False
for t, h, ip, u in okevents:
    if any(f[2] == ip and f[0] <= t for f in fails):
        hit = True; print(f"  {t:%H:%M:%S} {h}: {u} from {ip}")
if not hit: print("  none")
```

Test dataset (paste it to create `auth_sample.log`, then run the script and compare with the expected output below):

```
cat > auth_sample.log <<'EOF'
<38>Nov  3 10:15:01 web01 sshd[1201]: Failed password for invalid user admin
from 203.0.113.5 port 52122 ssh2
<38>Nov  3 10:15:04 web01 sshd[1201]: Failed password for invalid user admin
from 203.0.113.5 port 52123 ssh2
<38>Nov  3 10:15:07 web01 sshd[1203]: Failed password for root from
203.0.113.5 port 52130 ssh2
<38>Nov  3 10:15:10 web01 sshd[1203]: Failed password for root from
203.0.113.5 port 52131 ssh2
<38>Nov  3 10:15:13 web01 sshd[1203]: Failed password for root from
203.0.113.5 port 52132 ssh2
<38>Nov  3 10:15:20 web01 sshd[1210]: Accepted password for root from
203.0.113.5 port 52140 ssh2
<38>Nov  3 10:16:02 db01 sshd[880]: Failed password for postgres from
198.51.100.7 port 40010 ssh2
<38>Nov  3 10:16:30 db01 sshd[881]: Failed password for postgres from
198.51.100.7 port 40011 ssh2
<86>Nov  3 10:17:11 app01 sudo: pam_unix(sudo:auth): authentication failure;
logname=dev uid=1001 euid=0 tty=/dev/pts/0 ruser=dev rhost=  user=dev
<86>Nov  3 10:18:45 app01 sshd[501]: Accepted publickey for deploy from
192.0.2.20 port 50000 ssh2
<27>Nov  3 10:19:00 db01 postgres[900]: FATAL: password authentication failed
for user "report"
<38>Nov  3 10:40:00 web01 sshd[1301]: Failed password for invalid user test
from 198.51.100.99 port 41000 ssh2
EOF
python3 logscan.py auth_sample.log --threshold 5 --window 10
```

Result to expect	Value
Total authentication failures	10 (8 are sshd "Failed password", 1 sudo PAM, 1 PostgreSQL)
Per host	web01 6, db01 3, app01 1
Per source IP	203.0.113.5: 5, 198.51.100.7: 2, 198.51.100.99: 1, local@app01: 1, local@db01: 1
Per user	root 3, admin 2, postgres 2, dev 1, report 1, test 1
Severity	info 9, err 1 (the PostgreSQL FATAL line, `<27>` = daemon.err)
Brute-force suspects (5 in 10 min)	203.0.113.5 on web01
Success after failures	10:15:20 web01: root from 203.0.113.5, a probable compromise
With `--threshold 2`	also 198.51.100.7 on db01
`grep -c 'Failed password'`	8

How to word the analysis: "203.0.113.5 made 5 failed logins in 12 seconds against web01, using the users admin and root, which is a brute-force attack; the root login at 10:15:20 from the same address succeeded, so the account may be compromised and should be locked and its password reset. db01 shows two failures for postgres from another address. Severity is info because sshd logs failed logins at info, so a severity filter alone would not catch this."

Task 4: create a log-filtering rule or alert condition and demonstrate it

Option A: an rsyslog filter rule (on the server). Put failed SSH logins into their own file. Create `/etc/rsyslog.d/20-authfail.conf` (must load before the catch-all rule, so keep the lower number):

```
if $programname == 'sshd' and $msg contains 'Failed password' then
/var/log/remote/ssh_failures.log
```

```
sudo rsyslogd -N1 && sudo systemctl restart rsyslog
logger -n $SERVER -P 514 -T -t sshd "Failed password for invalid user ops
from 203.0.113.77 port 5555 ssh2"   # trigger
logger -n $SERVER -P 514 -T -t sshd "Accepted password for alice from
192.0.2.9 port 6666 ssh2"               # must NOT match
sudo cat /var/log/remote/ssh_failures.log
```

In my test only the "Failed password" line went into `ssh_failures.log`, while both lines were still stored in the per-host `sshd.log`. Other tested filter forms that validate under `rsyslogd -N1`:

```
:msg, contains, "Failed password" /var/log/remote/legacy_failures.log
if re_match($msg, 'Failed password for (invalid user )?[a-z]+ from ([0-9]
{1,3}[.]){3}[0-9]{1,3}') then /var/log/remote/regex_failures.log
if ($syslogfacility-text == 'authpriv' or $syslogfacility-text == 'auth') and
$syslogseverity <= 6 then /var/log/remote/auth_all.log
auth,authpriv.* /var/log/remote/auth_all.log
```

Two syntax traps I hit: in `re_match()` a single backslash (`\.`) is a parse error, so use `[.]` (or `\\.`); and there is no `$msg regex` operator, use `re_match()`.

Option B: a threshold alert. Save `authalert.sh` and run it against the filtered file or `auth.log`. It counts failures in the last N minutes, raises an alert at the threshold and also logs it as `auth.alert`. Works with both the traditional (`Nov 3 10:15:01`) and ISO timestamp formats; exit code 1 means alert, so it can run from cron.

```
#!/bin/bash
# usage: ./authalert.sh LOGFILE [THRESHOLD] [MINUTES]   -> raises an alert if
enough failures are recent
LOG=${1:?log file}; THRESH=${2:-5}; MIN=${3:-10}
[ -r "$LOG" ] || { echo "Cannot read $LOG"; exit 2; }
since=$(date -d "-$MIN min" +%s); count=0
while IFS= read -r line; do
  case "$line" in [0-9]*) ts=${line%% *} ;; *) ts=$(echo "$line" | awk
'{print $1, $2, $3}') ;; esac   # ISO or traditional timestamp
  e=$(date -d "$ts" +%s 2>/dev/null) || continue
  [ "$e" -ge "$since" ] && count=$((count + 1))
done < <(grep 'Failed password' "$LOG")
if [ "$count" -ge "$THRESH" ]; then
  msg="ALERT: $count failed SSH logins in the last $MIN min (threshold
$THRESH) in $LOG"
  echo "$msg"; logger -p auth.alert -t authalert "$msg"; exit 1
fi
echo "OK: $count failed SSH logins in the last $MIN min (threshold $THRESH)"
```

Demonstration (generate six recent failures, then run it):

```
for i in 1 2 3 4 5 6; do logger -t sshd -p authpriv.info "Failed password for
root from 203.0.113.5 port 22 ssh2"; sleep 1; done
./authalert.sh /var/log/auth.log 5 10        # expect: ALERT: 6 failed SSH
logins in the last 10 min ...
./authalert.sh /var/log/auth.log 10 10       # expect: OK (below the
threshold of 10)
```

I tested it: six recent failures plus one old one gave `ALERT: 6 ... (threshold 5)` with exit 1; threshold 10 gave `OK`; a 1-minute window counted only the 2 most recent; a missing file gave exit 2. (`/var/log/auth.log` may need `sudo`.) Justify the threshold: five failures in ten minutes is above normal typing mistakes and below the rate of an automated attack, so it catches brute force with few false alarms.

Likely examiner modifications

- Switch the transport: UDP `@host:514` to TCP `@@host:514` and say why (TCP is reliable and ordered; UDP can silently drop messages).
- Change the port: `*.* @@host:10514` on the client and `input(type="imtcp" port="10514")` on the server.
- Forward only some messages: `auth,authpriv.* @@host:514` (only authentication) or `*.warning @@host:514` (warning and above).
- Different test severity or facility: `logger -p local3.crit "msg"`; filter by severity: `if $syslogseverity <= 3 then /var/log/remote/errors.log` (err and worse).
- Show only one host's messages: `sudo cat /var/log/remote/web01/*.log` or `grep ' web01 ' file`.
- Count by severity, or find the most recent failure: `grep 'Failed password' $LOG | tail -1`.
- A different dataset (variant): re-run `logscan.py`; if the format differs (no host field or a JSON line), adapt the grep patterns and field numbers.
- Time-restricted queries: `awk '$3 >= "10:15:00" && $3 <= "10:16:00"' file`, or `journalctl --since "10 min ago" -t sshd`.

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Message never reaches the server	Firewall, wrong IP, or the server not listening	`nc -vz $SERVER 514`; on the server `sudo ss -tulnp | grep 514`; `sudo ufw status`	Open port 514 (tcp and udp); load `imtcp` or `imudp` and restart rsyslog on the server
Client sends over TCP but the server only listens on UDP (or the reverse)	`@` vs `@@` mismatch	Compare the client's `@` / `@@` with the server's `input(...)` lines	Match the protocol on both sides
`rsyslogd -N1` reports `syntax error on token`	Typo, curly quotes from copy-paste, or a legacy/new syntax mix	The line number in the message	Fix that line; use plain straight quotes; re-run `-N1`
Rule added but no effect	rsyslog not restarted, or an earlier rule consumed the message (`& stop`)	`sudo systemctl restart rsyslog`; file order in `/etc/rsyslog.d/` (alphabetical)	Restart; give your rule a lower number such as `20-`
Files not created on the server	Directory missing or wrong owner (rsyslog drops privileges to `syslog`)	`ls -ld /var/log/remote`; `journalctl -u rsyslog -n 30`	`sudo mkdir -p /var/log/remote && sudo chown syslog:adm /var/log/remote`
All remote logs land in the server's own `/var/log/syslog`	No remote-specific rule, or it runs after the default rule	Order of rules	Place the remote rule first and end it with `& stop`

Symptom or message	Likely cause	Check	Fix
Folder name is wrong (`localhost`, IP)	`%HOSTNAME%` comes from the message header	Look at a raw line	Use `%FROMHOST-IP%` or `%fromhost%` in the template
Hours off in the timestamps	Time zone or clock mismatch between hosts	`timedatectl` on each host	Fix the time zone; enable NTP (`timedatectl set-ntp true`)
`logger` seems to do nothing	The marker was written but filtered, or rsyslog is stopped	`grep MARK /var/log/syslog`; `systemctl status rsyslog`	Start rsyslog; check the facility and severity selectors
Burst of messages missing	rsyslog rate limiting (`imuxsock lost N messages due to rate-limiting`)	`journalctl -u rsyslog | grep rate`	Send test messages one second apart; raise `SystemLogRateLimitInterval`
`logscan.py` shows zero failures	Different log format (no `Mmm dd hh:mm:ss`, ISO timestamps)	`head -3 file`	Convert the timestamps or adjust the `LINE` pattern
`grep: binary file matches`	A control character in the log	Try `grep -a`	Use `grep -a`
Queue or disk full on the server	No log rotation	`df -h /var/log`	Configure `logrotate` for `/var/log/remote/*/*.log`

Viva theory

Syslog. A standard for generating, labelling and transporting log messages (RFC 3164 legacy, RFC 5424 modern). Each message has a **facility** (the source category), a **severity** (0 emergency to 7 debug), a timestamp, a hostname, a tag/program, and text. **PRI = facility x 8 + severity.**

rsyslog architecture. Inputs (`imuxsock` for local `/dev/log` , `imklog` for the kernel, `imudp` , `imtcp` , `imfile`) feed messages to rules (filters on facility, severity, program name or text), which run actions (`omfile` write to a file, `omfwd` forward over the network, `omprog` run a program). Templates define the output format and the file name pattern.

Transport options. UDP 514 is fast but unreliable (no acknowledgement). TCP 514 is reliable and ordered. RELP (port 2514) adds application-level acknowledgements. TLS (6514) adds encryption and authentication. Queues (disk-assisted) keep messages when the server is down.

Why centralize logs. One place to search and correlate events across hosts; attackers who gain root on a host can edit local logs but not copies already shipped away; retention and compliance; alerting from one point; faster incident investigation.

Brute-force indicators. Many `Failed password` or `Invalid user` events from one source in a short time, many different usernames from one IP (user enumeration), failures followed by one `Accepted` (success after guessing), failures across many hosts from the same IP (spraying). Responses: block the IP (`ufw deny from`), `fail2ban` , key-only SSH, `PermitRootLogin no` , MFA.

Severity versus importance. The severity label is chosen by the program, not by security impact: failed SSH logins are `info` , so alerting only on `err` or worse misses brute force. Content-based filters are needed.

Log management basics. Rotate with `logrotate` , keep clocks in sync (NTP), restrict permissions on log files, protect the log server, and define retention.

journald versus rsyslog. `systemd-journald` stores structured binary logs (`journalctl`); on Ubuntu rsyslog receives messages from journald and writes the classic text files and handles forwarding.

Typical viva questions: What does `*.*` mean? (Every facility, every severity.) What is the difference between `@` and `@@` ? (UDP vs TCP.) How do you test forwarding? (`logger` with a unique marker, then grep on the server.) What is the severity of `Failed password` ? (info.) Why can UDP syslog lose events? (No acknowledgement or retransmission.) How would you secure syslog transport? (TLS on 6514 with certificates.) Which file holds SSH login events on Ubuntu? (`/var/log/auth.log` .) What does `$fromhost-ip` mean? (The IP

the message was received from.) How would you stop duplicates in `/var/log/syslog`? (End the remote rule with `& stop`.) How do you decode `<86>`? (86 = 10 x 8 + 6, which is authpriv.info.)

Lab 4: Network monitoring using Zabbix

Scenario: monitor a Linux application server using an existing Zabbix installation. Marks: Task 1 (2) add or enable the assigned host and verify agent availability; Task 2 (3) configure or select items for CPU, memory and network utilization; Task 3 (3) analyze a supplied performance anomaly and name the probable bottleneck; Task 4 (2) configure a trigger and demonstrate the problem event with a safe test condition.

Honesty note: the Zabbix web interface cannot be run in my sandbox, so the menu paths below are from Zabbix 6.0 LTS (the usual version on Ubuntu 22.04) and the item keys and trigger syntax follow its documentation. The diagnosis script in Task 3 was run and tested. If the exam server is Zabbix 5.x or 7.x, menu names differ slightly, so read the screen.

Setup and how the pieces fit

Component	Role	Port
Zabbix server	Collects data, evaluates triggers, raises problems	10051/tcp
Zabbix frontend	Web UI (Apache or Nginx, PHP), usually `http://SERVER/zabbix`	80/443
Database	MySQL or PostgreSQL stores configuration and history	3306/5432
Zabbix agent (or agent2)	Runs on the monitored host and answers for items	10050/tcp

Passive check: the server asks the agent (port 10050). Active check: the agent asks the server for its item list and sends results (port 10051). Default login of a fresh install: `Admin` / `zabbix`, so use whatever the exam supplies.

On the **monitored host** (only if the agent is not already installed):

```
sudo apt update && sudo apt install -y zabbix-agent          # or zabbix-
agent2; Ubuntu's own repo has 5.0 (6.0 on newer releases), the zabbix.com
repo has 6.0/7.0
sudo nano /etc/zabbix/zabbix_agentd.conf                     # agent2:
/etc/zabbix/zabbix_agent2.conf
#   Server=192.168.1.10          <- IP of the Zabbix server (passive checks)
#   ServerActive=192.168.1.10    <- same IP (active checks)
#   Hostname=appserver01         <- MUST match the host name entered in the
frontend (active checks)
sudo systemctl restart zabbix-agent && sudo systemctl enable zabbix-agent
sudo systemctl status zabbix-agent --no-pager
sudo ss -tlnp | grep 10050                                    # agent
listening
sudo tail -n 20 /var/log/zabbix/zabbix_agentd.log             # errors show
here
sudo ufw allow from 192.168.1.10 to any port 10050 proto tcp   # if ufw is
active
```

On the **Zabbix server** (to test without the web UI):

```
sudo apt install -y zabbix-get                                # provides
zabbix_get
zabbix_get -s 192.168.1.20 -p 10050 -k agent.ping             # expect: 1
zabbix_get -s 192.168.1.20 -k system.cpu.util                  # expect: a
number such as 3.2
zabbix_get -s 192.168.1.20 -k agent.version
sudo systemctl status zabbix-server --no-pager
sudo tail -n 30 /var/log/zabbix/zabbix_server.log
```

Task 1: add or enable the assigned host and verify agent availability

Frontend steps (Zabbix 6.0):

1. Log in, then **Data collection > Hosts** (in 5.x: Configuration > Hosts).
2. If the host exists but is disabled, click the red **Disabled** status to enable it. Otherwise click **Create host** (top right).
3. **Host name**: the exact name given (and the same as `Hostname=` in the agent config for active checks). **Templates**: add `Linux by Zabbix agent` (passive) or `Linux by Zabbix agent active`. **Host groups**: `Linux servers`.
4. **Interfaces > Add > Agent**: IP address of the server, port `10050`, connect by IP.
5. Click **Add** (or **Update**).
6. Wait one to two minutes, then reload the host list. The **Availability** icon **ZBX** must turn green. Red means the server cannot reach the agent; grey means no check has run yet.

Proof for the report: screenshot of the host list with a green ZBX, plus `zabbix_get ... agent.ping` returning `1`, plus **Monitoring > Latest data** showing fresh values with recent timestamps.

If ZBX is red, hover over it to read the error and use the common-errors table below.

Task 2: configure or select monitoring items for CPU, memory and network utilization

The template `Linux by Zabbix agent` already contains most items, so first check **Monitoring > Latest data**, filter by host, and tick the ones you need. If the task says to **configure** items, create them manually: open the host, **Items > Create item**.

Metric	Item key	Type	Units	Meaning
CPU utilization	`system.cpu.util`	Zabbix agent	%	Total busy time (user + system + others)
CPU iowait	`system.cpu.util[,iowait]`	Zabbix agent	%	Time waiting on disk
CPU load	`system.cpu.load[all,avg1]`	Zabbix agent	load	1-minute run queue average (compare with core count)
CPU cores	`system.cpu.num`	Zabbix agent	cores	Capacity
Memory available	`vm.memory.size[available]`	Zabbix agent	B	Free plus reclaimable memory
Memory available %	`vm.memory.size[pavailable]`	Zabbix agent	%	Best single memory health number
Memory total	`vm.memory.size[total]`	Zabbix agent	B	Capacity
Swap free %	`system.swap.size[,pfree]`	Zabbix agent	%	Low value means heavy swapping

Metric	Item key	Type	Units	Meaning
Network in	`net.if.in[eth0]`	Zabbix agent	bps (Change per second)	Bytes received; the interface name must be right
Network out	`net.if.out[eth0]`	Zabbix agent	bps	Bytes sent
Interface discovery	`net.if.discovery`	Zabbix agent		Lists interfaces for LLD
Agent reachability	`agent.ping`	Zabbix agent	1 or nothing	Value 1 means the agent answers

Manual item form fields: **Name** (for example `CPU utilization`), **Type** `Zabbix agent`, **Key** (from the table), **Type of information** `Numeric (float)` for percentages or `Numeric (unsigned)` for counters, **Units**, **Update interval** `30s` (use a short interval so data appears quickly in the exam), **History** `7d`, **Trends** `365d`. For `net.if.in[eth0]` choose **Preprocessing > Change per second** if the key returns a counter, then multiply by 8 for bits (`Custom multiplier 8`, units `bps`). Find the real interface name with `ip -br link`.

Verify before moving on:

```
zabbix_get -s 192.168.1.20 -k 'net.if.in[eth0]'            # raw byte
counter; run twice to see it grow
zabbix_get -s 192.168.1.20 -k 'vm.memory.size[pavailable]'
```

In the UI: **Monitoring > Latest data** > select the host > the **Graph** link on each item gives a time-series graph; **Monitoring > Hosts > Graphs** shows the template's pre-made graphs (CPU utilization, memory usage, network traffic on an interface).

Compare Zabbix with the command line from Lab 1: `system.cpu.util` should match `top`'s 100 minus idle, and `vm.memory.size[pavailable]` should match the "available" column of `free -m`.

Task 3: analyze a performance anomaly and identify the bottleneck

You will usually be given a graph, a table of item values or screenshots. Read them in this order:

1. **CPU util high, load above the core count, iowait low**: CPU-bound (a runaway process, compute load).

2. **iowait high (above 20%), CPU user low, load high**: disk I/O bottleneck (slow disk, heavy writes, database fsync).
3. **Memory available very low (under 10%), swap used rising, disk activity up**: memory exhaustion, then swapping, which causes the disk and CPU symptoms.
4. **Network in or out near link speed**: network saturation; CPU and memory look normal.
5. **Everything normal but `agent.ping` stopped**: the host or agent or the network path is down, which is an availability problem and not a resource one.

When several symptoms appear, find the cause that came first on the time axis: memory pressure causes swap, which causes iowait, which makes the load average rise. Name that first cause.

Save `zbxdiag.py`. It asks for the latest values (or takes them as arguments) and names the probable bottleneck, with the reason.

```
#!/usr/bin/env python3
"""Lab 4: classify a Zabbix performance anomaly from latest item values.
Usage: python3 zbxdiag.py [cpu iowait load1 cores mem_avail_pct swap_used_pct
net_in_mbps net_out_mbps link_mbps]
Prompts for anything not supplied."""
import sys
names = ["CPU util % (system.cpu.util)", "CPU iowait %
(system.cpu.util[,iowait])", "Load avg 1 min (system.cpu.load[all,avg1])",
         "CPU cores (system.cpu.num)", "Available memory %
(vm.memory.size[pavailable])", "Swap used % (100 -
system.swap.size[,pfree])",
         "Network in Mbps", "Network out Mbps", "Link speed Mbps"]
a = sys.argv[1:]
vals = []
for i, n in enumerate(names):
    try:
        vals.append(float(a[i]) if i < len(a) else float(input(f"{n}: ")))
    except ValueError:
        sys.exit("Numbers only.")
cpu, iow, load, cores, memav, swap, rin, rout, link = vals
if cores <= 0 or link <= 0:
    sys.exit("cores and link speed must be > 0")
findings = []
if iow >= 20 or (cpu < 80 and iow >= 10 and load > cores):
    findings.append(("DISK I/O", f"iowait {iow:.0f}% with load {load:g} on
{cores:g} cores: processes wait on storage"))
if cpu >= 80 or load > 1.5 * cores:
    findings.append(("CPU", f"CPU {cpu:.0f}% / load {load:g} vs {cores:g}
cores: run queue exceeds capacity"))
if memav <= 10 or swap >= 20:
```

```
    findings.append(("MEMORY", f"only {memav:.0f}% memory available, swap
used {swap:.0f}%: memory pressure"))
util = max(rin, rout) / link * 100
if util >= 80:
    findings.append(("NETWORK", f"busiest direction uses {util:.0f}% of
{link:g} Mbps link"))
print("\nProbable bottleneck(s):", ", ".join(f[0] for f in findings) if
findings else "none (all within normal range)")
for k, why in findings:
    print(f"  {k}: {why}")
if len(findings) > 1:
    print("Several symptoms: fix the earliest cause first (memory pressure ->
swap -> disk I/O wait -> apparent CPU load).")
```

Test inputs I ran (order: cpu iowait load cores mem% swap% in out link):

Command arguments	Expected result
`95 2 8 4 60 0 10 5 1000`	CPU (95% busy, load 8 on 4 cores)
`10 45 6 4 50 2 10 5 1000`	DISK I/O (iowait 45%, CPU mostly idle)
`30 3 1 4 5 60 10 5 1000`	MEMORY (5% available, 60% swap)
`20 1 0.5 4 70 0 900 20 1000`	NETWORK (90% of the link)
`20 1 0.5 4 70 0 100 20 1000`	none (all within normal range)

Example wording: "CPU 10%, iowait 45%, load 6 on 4 cores: the CPUs are idle but processes are blocked waiting for storage, so the bottleneck is disk I/O, not CPU." Lab 1 commands confirm it on the host: `iostat -xz 1 5` (high `%util`, high `await`) and `vmstat 1 5` (high `wa`, `b` column above 0).

Create a real anomaly for practice (safe, ends by itself):

```
sudo apt install -y stress-ng
stress-ng --cpu 4 --timeout 120s                  # CPU anomaly
stress-ng --vm 2 --vm-bytes 80% --timeout 60s     # memory pressure (careful
on small VMs)
stress-ng --hdd 2 --timeout 60s                   # disk I/O
# without stress-ng:  timeout 120 sh -c 'yes > /dev/null' &  (one per core)
```

Task 4: configure a trigger and demonstrate the problem event with a safe test condition

Trigger on the host: **Data collection > Hosts > (host) > Triggers > Create trigger**.

Field	Value
Name	`High CPU utilization on {HOST.NAME}`
Severity	Warning (or Average/High)
Expression (6.0 and later)	`avg(/appserver01/system.cpu.util,1m)>80`
Recovery expression (optional)	`avg(/appserver01/system.cpu.util,1m)<60`
OK event generation	Expression (or Recovery expression, which gives hysteresis)

Syntax notes: Zabbix 6.0 and later use `function(/host/key,parameter)`; Zabbix 5.x used `{host:key.function(param)}`, for example `{appserver01:system.cpu.util.avg(1m)}>80`. Use the **Add** button next to Expression to build it from a dialog and avoid typos. For other metrics: `last(/appserver01/vm.memory.size[pavailable])<10`, `last(/appserver01/agent.ping)=0`, `nodata(/appserver01/agent.ping,2m)=1` (no data for 2 minutes), `last(/appserver01/system.swap.size[,pfree])<50`.

Safe test condition (choose one):

- **Lower the threshold temporarily** so normal activity triggers it, for example `last(/appserver01/system.cpu.util)>1` or `last(/appserver01/vm.memory.size[pavailable])<95`. Nothing is stressed; revert afterwards. This is the safest.
- **Generate short, bounded load:** `stress-ng --cpu 2 --timeout 90s` (it stops by itself). Keep the trigger window short (`avg(...,1m)`) and the item interval at `30s`.
- **Use a trapper item** for a fully artificial value: create an item of type **Zabbix trapper**, key `lab.test`, numeric; trigger `last(/appserver01/lab.test)>90`. Send values from the server or any host with:

```
zabbix_sender -z 192.168.1.10 -s appserver01 -k lab.test -o 95     # raises
the problem
zabbix_sender -z 192.168.1.10 -s appserver01 -k lab.test -o 10     # sends a
recovery value
```

Show the result in **Monitoring > Problems** (the new problem with severity, start time and duration), and then show recovery (status Resolved after the value falls). Also screenshot **Monitoring > Hosts**, which shows the problem count against the host. If asked about notification, mention **Alerts > Actions > Trigger actions** plus a media type (email), which needs an SMTP server and a user with media. Without it, the problem appears only in the UI, which is enough for "demonstrate the problem event".

Do not leave the lowered threshold in place, and delete or disable test triggers when finished unless told to keep them.

Likely examiner modifications

- Different metric: disk space trigger `last(/appserver01/vfs.fs.size[/,pfree])<20`; process count `proc.num[,,run]`; web check `net.tcp.service[http,,80]`.
- Different threshold or duration: change `80` and `1m`, and explain why short windows give noisy alerts and long windows delay detection.
- Different severity: Warning vs High and why. Add a **second trigger with a higher threshold** (for example 80 warning, 95 high).
- Monitor a second interface: `net.if.in[ens33]`.
- Active instead of passive: switch the template to `Linux by Zabbix agent active` and make sure `Hostname=` equals the frontend host name and `ServerActive` is set.
- Show an item's history: **Latest data > History**, or use the **Graphs** tab with a time range.
- Explain what `agent.ping` returns when the agent is down (no data, so use `nodata()` or `fuzzytime()`).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
ZBX icon red: `Get value from agent failed: cannot connect to [[IP]:10050]: Connection refused`	Agent not running or listening	`systemctl status zabbix-agent`; `ss -tlnp | grep 10050`	Start and enable the agent; check the interface IP in the host
`Connection timed out`	Firewall or routing between server and agent	From the server `nc -vz AGENT 10050`; `ufw status`	Allow 10050/tcp from the server IP

Symptom or message	Likely cause	Check	Fix
`Received empty response from Zabbix Agent ... Assuming that agent dropped connection because of access permissions`	The server's IP is not in `Server=`	`grep ^Server /etc/zabbix/zabbix_agentd.conf`	Put the server IP in `Server=` and restart the agent
Items show `Not supported`	Wrong key, wrong interface name, missing permission	Hover the red info icon; `zabbix_get -k KEY`	Correct the key (`ip -br link` for the interface name)
Host added but no data and no availability	Wrong host interface IP, or the host is disabled	Host list status column	Fix the IP; enable the host
Active checks give `no active checks on server`	`Hostname` differs from the frontend host name	`grep -E '^(Hostname|ServerActive)' agent.conf` and the agent log	
Graph is empty	Item interval not passed yet, wrong time range, or type of information wrong	Latest data timestamp	Wait one interval; widen the range; set the right type
Network item shows an ever-growing number	Counter without preprocessing	Item value vs time	Add the **Change per second** preprocessing step
Trigger never fires	Expression host/key spelled wrongly or threshold unreachable	Trigger list shows red error icon; Latest data value	Fix the expression; use the Add dialog; lower the threshold for the test
`Invalid trigger expression` after upgrade notes	5.x syntax used on 6.0	Compare the syntax above	Rewrite as `avg(/host/key,5m)>80`

Symptom or message	Likely cause	Check	Fix
`zabbix_sender` says `failed: 1`	Trapper item missing, wrong host name, or the allowed hosts field blocks the source	Item type and **Allowed hosts**	Create the trapper item; clear Allowed hosts or add your IP
Server log: `cannot connect to the database`	Database stopped or wrong password	`sudo tail /var/log/zabbix/zabbix_server.log`; `systemctl status mysql`	Start the DB; correct `DBPassword` in `zabbix_server.conf`
Frontend shows `Zabbix server is not running`	Server process down	`systemctl status zabbix-server`	Start it and read its log
Time differs between server and host	Clock drift	`timedatectl`	Enable NTP

Viva theory

What Zabbix is. An open-source enterprise monitoring platform for servers, networks, applications and services. It collects metrics with agents, SNMP, IPMI, JMX, HTTP and scripts, stores them, draws graphs and raises alerts.

Architecture. Server (core process), database (history and configuration), web frontend, optional proxies (collect for remote sites), and agents. Data flows: agent to server (active) or server to agent (passive) and then into the database and triggers.

Hosts, items, triggers, actions. A **host** is a monitored entity. An **item** is one metric collected by key at an interval. A **trigger** is a logical expression over item data that sets a problem state. An **action** reacts to events (send mail, run a script). A **template** is a reusable bundle of items, triggers and graphs linked to many hosts.

Passive vs active. Passive: server polls the agent on port 10050. Active: the agent requests its checks from the server on 10051 and pushes data; it is better for hosts behind NAT or firewalls and lowers server load. Both can be used together.

History vs trends. History is every raw value (large, kept for days). Trends are hourly min, avg and max (small, kept for a year), used for long graphs.

Severity levels. Not classified, Information, Warning, Average, High, Disaster.

Hysteresis. A recovery expression lower than the problem threshold prevents flapping, so a value hovering around 80 does not open and close problems repeatedly.

LLD (low-level discovery). Automatically creates items for things that vary per host, for example each network interface (`net.if.discovery`) or file system (`vfs.fs.discovery`).

Why `avg(...,5m)` rather than `last()`. `last()` reacts to one sample, so a brief spike raises a false alarm; `avg` over a window ignores short bursts, at the cost of slower detection.

Zabbix vs Prometheus vs Nagios. Zabbix: agent and SNMP polling, built-in UI, triggers, templates, SQL store. Prometheus: pull model, time-series database, PromQL, relies on Alertmanager and Grafana. Nagios: check-based state monitoring with plugins, weaker graphing.

Typical viva questions: Default agent port? (10050; server 10051.) Which file configures the agent? (`/etc/zabbix/zabbix_agentd.conf` .) What does the green ZBX mean? (The server reached the agent.) What does `Hostname=` matter for? (Active checks match by it.) What key gives CPU use? (`system.cpu.util` .) What does load average mean? (Average number of runnable plus uninterruptible processes; compare with core count.) Trigger expression for memory under 10%? (`last(/host/vm.memory.size[pavailable]) <10` .) What is a trapper item? (An item that accepts values pushed with `zabbix_sender` .) How would you test an alert safely? (Lower the threshold or use a trapper item, then restore.) What is the difference between a problem and an event? (An event is any state change; a problem is the open event of a trigger in the PROBLEM state.)

Lab 5: Network monitoring using LibreNMS

Scenario: monitor a network device and evaluate interface performance. Marks: Task 1 (2) discover or verify the assigned SNMP-enabled device; Task 2 (3) display interface utilization, device uptime and interface operational status; Task 3 (3) analyze supplied interface traffic and error-counter data to find abnormal behavior; Task 4 (2) configure an alert rule for interface failure or excessive utilization and explain its significance.

Honesty note: LibreNMS cannot be run in my sandbox, so the UI paths and `lnms` commands come from its documentation and may differ a little between versions. The analysis script in Task 3 was run and tested.

Setup and how it works

LibreNMS is a PHP and MySQL/MariaDB application, normally installed in `/opt/librenms` and run as the user `librenms` . It finds devices with SNMP (v1/v2c/v3), polls them every 5 minutes through cron, stores counters in RRD files and the database, and evaluates alert rules after each poll.

```
cd /opt/librenms
sudo -u librenms ./validate.php                    # health check of the
install: read every FAIL line
sudo systemctl status snmpd mariadb nginx php8.1-fpm --no-pager   # services
(names vary: apache2 / php-fpm version)
cat /etc/cron.d/librenms                           # poller and discovery
schedule
sudo -u librenms ./lnms device:list                # devices already known
(older versions: ./lnms list:device)
```

First prove that SNMP works from the LibreNMS host, because most failures here are SNMP failures (use Lab 2 commands):

```
snmpget -v2c -c public 192.168.1.1 1.3.6.1.2.1.1.5.0          # sysName
snmpwalk -v2c -c public 192.168.1.1 1.3.6.1.2.1.2.2.1.2       # interface
names
```

Task 1: discover or verify the assigned SNMP device

UI (current versions): **Devices > Add Device**. Enter **Hostname or IP**, **SNMP version** `v2c`, **Community** (for example `public`), leave the port at 161 and the transport at UDP, then click **Add Device**. LibreNMS tests SNMP, identifies the OS, and queues discovery. Open **Devices > All Devices** and click the device. The status dot should be green.

Command line:

```
cd /opt/librenms
sudo -u librenms ./lnms device:add -c public -v v2c 192.168.1.1     # older
versions: ./addhost.php 192.168.1.1 public v2c
sudo -u librenms ./lnms device:discover 192.168.1.1                 # older:
./discovery.php -h 192.168.1.1
sudo -u librenms ./lnms device:poll 192.168.1.1                     # older:
./poller.php -h 192.168.1.1 -m ports
```

If the device already exists, verify it: open the device page and check **Overview** (hostname, OS, uptime, last polled and last discovered times, "Up" status), **Graphs**, and **Ports**. Verify the target is polled recently: the "Last polled" time must be within a few minutes. Check **Settings (gear) > Validate Config** or run `./validate.php` for global problems.

To prove SNMP independently of LibreNMS: `snmpget -v2c -c COMMUNITY HOST sysUpTime.0` and compare the uptime with the device page.

Task 2: display interface utilization, uptime and operational status

1. **Uptime:** the device page **Overview** shows "Uptime", derived from `sysUpTime` (OID `1.3.6.1.2.1.1.3.0`, in hundredths of a second). A reboot appears as a sudden drop. Compare with `snmpget ... sysUpTime.0`.
2. **Interface list and status:** device page > **Ports**. Columns: interface name, speed, traffic in and out, **operational status** (`ifOperStatus`: up, down) and **admin status** (`ifAdminStatus`). A port that is admin up but oper down has a link fault. A port that is admin down was disabled on purpose.
3. **Utilization:** device page > **Graphs** > **Ports** (or click a port > **Traffic**). Select the time range (day, week). For percent usage, click the port and read the **Percent Utilization** graph, or open **Ports > Port Traffic** to show a table of in and out rates and `Utilization %`.
4. **Error counters:** click the port > **Errors** graph (`ifInErrors`, `ifOutErrors`), and **Packets** (unicast, non-unicast), plus discards (`ifInDiscards`, `ifOutDiscards`).
5. **Quick view for many ports: Ports > All Ports**, filter by device, sort by traffic, or use **Ports > Errored Ports / Down Ports** reports.

The OIDs behind the displays (same as Lab 2):

What	OID	Notes
Uptime	`sysUpTime.0` 1.3.6.1.2.1.1.3.0	Timeticks, 1/100 s
Oper status	`ifOperStatus` 1.3.6.1.2.1.2.2.1.8	1 up, 2 down, 7 lowerLayerDown
Admin status	`ifAdminStatus` 1.3.6.1.2.1.2.2.1.7	1 up, 2 down
Speed	`ifSpeed` .2.2.1.5 / `ifHighSpeed` 1.3.6.1.2.1.31.1.1.1.15	bps / Mbps
Octets	`ifHCInOctets` 1.3.6.1.2.1.31.1.1.1.6, `ifHCOutOctets` ...1.10	64-bit counters
Errors	`ifInErrors` .2.2.1.14, `ifOutErrors` .2.2.1.20	
Discards	`ifInDiscards` .2.2.1.13, `ifOutDiscards` .2.2.1.19	

Utilization % = (delta bytes x 8) / (seconds x speed in bps) x 100, exactly as in Lab 2. Cross-check one reading with `ifrate.py` from Lab 2 against LibreNMS's graph.

Task 3: analyze interface traffic and error data

You will normally be handed a table or screenshots of counters per interface. Decide using this logic:

- **Utilization above 80 to 90% with discards**: congestion; the link or queue is saturated (needs a faster link, QoS, or load balancing).
- **Errors (CRC, input errors) above about 0.1% of packets with low utilization**: a physical-layer fault, such as a bad cable, dirty or failing SFP, duplex mismatch or a speed mismatch.
- **Oper status down while admin is up**: link failure (cable unplugged, peer down, wrong configuration).
- **Discards at low utilization**: policy, ACL or buffer problems, not capacity.
- **Very high out traffic at odd hours, or a sudden spike**: possible data exfiltration, backup, or a loop or broadcast storm. Check packet types (broadcast and multicast counters).
- **Counters suddenly reset to a small number**: the device rebooted or the 32-bit counter wrapped (check uptime).

Save `ifaudit.py`. Input is a CSV of *deltas* over a polling interval (differences between two polls). It computes in and out utilization, the error rate and the discard rate per interface, and gives an assessment.

```
#!/usr/bin/env python3
"""Lab 5: audit interface traffic/error samples. Usage: python3 ifaudit.py
[FILE.csv] (prompts for the path if omitted)
CSV header:
name,status,speed_mbps,interval_s,in_bytes,out_bytes,in_pkts,out_pkts,in_err,
out_err,in_disc,out_disc
(all counts are DELTAS over interval_s seconds)"""
import csv, sys
path = sys.argv[1] if len(sys.argv) > 1 else input("CSV file: ").strip()
try:
    rows = list(csv.DictReader(open(path)))
except OSError as e:
    sys.exit(f"Cannot read {path}: {e}")
print(f"{'Interface':<10}{'Status':<7}{'In%':>6}{'Out%':>6}{'Err%':>7}
{'Disc%':>7}  Assessment")
for r in rows:
    try:
        n = r["name"]; up = r["status"].strip().lower() == "up"
        sp = float(r["speed_mbps"]) * 1e6; t = float(r["interval_s"])
        ui = float(r["in_bytes"]) * 8 / t / sp * 100; uo =
float(r["out_bytes"]) * 8 / t / sp * 100
```

```
        pk = float(r["in_pkts"]) + float(r["out_pkts"])
        er = float(r["in_err"]) + float(r["out_err"]); di =
float(r["in_disc"]) + float(r["out_disc"])
    except (KeyError, ValueError, ZeroDivisionError) as e:
        print(f"{r.get('name','?'):<10}bad row ({e})"); continue
    ep = er / pk * 100 if pk else 0.0; dp = di / pk * 100 if pk else 0.0
    notes = []
    if not up:
        notes.append("DOWN: interface failure (check cable, peer, admin
state)")
    else:
        u = max(ui, uo)
        if u >= 90: notes.append("CRITICAL utilization: saturated link")
        elif u >= 80: notes.append("WARNING utilization: near capacity")
        if ep >= 1: notes.append("CRITICAL errors: physical fault (cable,
SFP, duplex mismatch)")
        elif ep >= 0.1: notes.append("WARNING errors: check cable and
duplex")
        if di > 0: notes.append("discards with " + ("high load:
congestion/queue drops" if u >= 80 else "low load: policy/buffer or QoS
issue"))
        if pk == 0 and u == 0: notes.append("idle")
    print(f"{n:<10}{'up' if up else 'down':<7}{ui:6.1f}{uo:6.1f}{ep:7.2f}
{dp:7.2f}  {'; '.join(notes) or 'normal'}")
```

Test dataset (create it, run the script, compare with the expected output):

```
cat > if_sample.csv <<'EOF'
name,status,speed_mbps,interval_s,in_bytes,out_bytes,in_pkts,out_pkts,in_err,
out_err,in_disc,out_disc
Gi0/1,up,1000,300,3750000000,1500000000,3000000,1200000,0,0,0,0
Gi0/2,up,1000,300,36000000000,9000000000,30000000,8000000,0,0,12000,3000
Gi0/3,up,100,300,150000000,60000000,120000,50000,4200,900,0,0
Gi0/4,down,1000,300,0,0,0,0,0,0,0,0
Gi0/5,up,1000,300,12000000,8000000,10000,7000,5,0,0,0
EOF
python3 ifaudit.py if_sample.csv
```

Interface	Expected result	Why
Gi0/1	In 10.0%, out 4.0%, normal	3.75 GB x 8 / 300 s = 100 Mbps on 1 Gbps
Gi0/2	In 96.0%, CRITICAL utilization, discards 0.04% under high load: congestion	36 GB x 8 / 300 s = 960 Mbps

Interface	Expected result	Why
Gi0/3	In 4.0%, out 1.6%, errors 3.00%, CRITICAL errors: physical fault	5100 errors in 170000 packets at low load
Gi0/4	DOWN: interface failure	status down
Gi0/5	Errors 0.03%, normal	5 errors in 17000 packets is below the 0.1% threshold

Wording for the answer: "Gi0/2 runs at 96% inbound with discards, so it is congested and needs more bandwidth or QoS. Gi0/3 shows 3% errors with only 4% load, which points to a physical problem, most likely a cable, SFP or duplex mismatch, not capacity. Gi0/4 is down and needs a link check." Check duplex and speed on the device (`ethtool eth0` on Linux, `show interface` on switches) and watch whether the error counter keeps increasing, because old errors that stop growing are not an active fault.

Task 4: configure an alert rule and explain its significance

UI: **Alerts > Alert Rules > Create new alert rule** (button at the top; in some versions **Alerts > Alert Rules > Create rule from collection**, which offers ready-made rules). Fill in:

Field	Interface down	Excessive utilization
Name	`Port down`	`Port utilization over 80%`
Rules (builder)	`ports.ifOperStatus` equal `down` AND `ports.ifAdminStatus` equal `up`	`ports.ifInOctets_perc` greater than `80` (or `ifOutOctets_perc`)
Match devices	All devices, or only the assigned device/group	same
Severity	Critical	Warning
Count / delay	Count -1 (no limit), Delay 5 m	Delay 10 m (avoid short spikes)
Recovery alerts	on	on

The equivalent alert rule written in the old SQL style: `macros.port_down` (a macro for "admin up and oper down") and `macros.port_usage_perc >= 80`. Click **Save Rule**. Then add a **transport** so the alert goes somewhere: **Alerts > Alert Transports > Create** (mail,

Slack, or a file/API); attach it to the rule under the rule's **Transports/Contacts** or through an **Alert Template**. If no mail server exists, the alert is still visible in **Alerts > Alerts** (active) and **Alerts > Notifications/Alert History**, which is enough to demonstrate it.

Demonstrate safely:

- **Low threshold test:** set utilization `> 1` so normal traffic fires it, or create a rule on a metric that is certainly true, then restore the real value.
- **Admin down test (only on a lab device or a spare port the examiner approves):** shut a spare interface, wait for the next poll (up to 5 minutes), confirm the alert, then bring it back to confirm recovery. Do not shut the management link.
- Force an immediate poll to avoid waiting: `sudo -u librenms ./lnms device:poll DEVICE`, then wait for the alert queue (the dispatcher or `./alerts.php` cron).

Significance (the explanation expected): interface-down alerts give immediate notice of an outage or a failed link before users complain, which cuts the mean time to detect (MTTD) and repair (MTTR). Utilization alerts give early warning of saturation so capacity can be added before packet loss and slowness occur. Delay and count values prevent alert fatigue from brief spikes, and recovery notices confirm a fix. Alerts should be tied to a severity and to someone who must act.

Likely examiner modifications

- Alert on errors: `ports.ifInErrors_delta` greater than `10` (errors in the last poll) or `ifOutErrors_delta`.
- Alert on device down (ping and SNMP): rule `devices.status` equal `0`; uptime-restart rule `devices.uptime` less than `300` (device restarted in the last 5 minutes).
- Different community string or SNMP version, for example adding with `v3` (needs auth and priv settings), and explain why v2c is insecure.
- Add a second device, or a device group, and apply the rule to the group.
- Explain 32-bit vs 64-bit counters and why `ifHCInOctets` matters on 1 Gbps links (a 32-bit counter wraps in about 34 seconds at 1 Gbps).
- Show the poll interval and change it (default 300 s) and explain the trade-off (finer data, higher load).
- Read traffic in different units (bits vs bytes) and the 95th percentile (used for billing).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`Could not reach SNMP` / device add fails	Wrong community, SNMP off on the device, UDP 161 blocked, ACL on the device	`snmpget -v2c -c COMM HOST 1.3.6.1.2.1.1.1.0`	Correct the community; enable and allow the LibreNMS IP in snmpd (`agentAddress`, `rocommunity`)
`Timeout: No Response from HOST:161`	Firewall or host unreachable	`ping HOST`; `sudo ufw status` on the agent; `nc -vzu HOST 161` is unreliable	Open UDP 161
Device added but graphs are empty	The poller has not run yet, or the cron is missing	`cat /etc/cron.d/librenms`; device page > "Last polled"	Wait 5 to 10 minutes; run `lnms device:poll`; install the cron entry
Device shows as down but pings fine	SNMP failing, while ICMP works	Device page > Settings > check SNMP details	Fix the SNMP community or version
`validate.php` FAIL: database schema, permissions, or `Poller is not running`	Install problems	The message text	Run the command it suggests, often `./daily.sh` or `chown -R librenms:librenms /opt/librenms`
No interfaces listed under Ports	Discovery has not run or the ports module is disabled	Run `lnms device:discover HOST`	Discover again; enable "ports" in device discovery modules
Port shows `down` but is in use	A different interface was selected, or the port is ignored/disabled in LibreNMS	Port page > Settings	Select the right port; clear the Ignore flag
Utilization appears too low or too high	Wrong speed, 32-bit counter, or the "ifSpeed override"	`snmpget IF-MIB::ifHighSpeed.N`	Override the interface speed in the port settings; use 64-bit

Symptom or message	Likely cause	Check	Fix
Alert rule never fires	Wrong entity/macro, delay not elapsed, rule not mapped to the device	Alerts > Alert Rules (device mapping) and `Alerts > Alerts`	Use a rule from the collection; reduce the delay; match devices
Alert fires but nothing is received	No transport configured, or mail is not working	Alerts > Alert Transports > Test	Add and test a transport
Web page 500 or blank	PHP-FPM, nginx or permissions broken	`tail /opt/librenms/storage/logs/librenms.log`; `journalctl -u nginx`	Fix the permissions on `storage/` and `bootstrap/cache/`; restart services
Times look wrong	Time zone mismatch between PHP, MySQL and the system	`timedatectl`; `validate.php`	Set the same time zone everywhere

Viva theory

LibreNMS. An open-source, auto-discovering network monitoring system based on SNMP. It builds graphs (RRDtool), keeps device and port inventory, and has a flexible alert system. It is a fork of Observium.

SNMP in one minute. A manager (LibreNMS) polls agents on devices (UDP 161) for values identified by OIDs from MIBs (for example IF-MIB). Operations: GET, GETNEXT, GETBULK, SET; traps and informs are sent from the device to the manager (UDP 162). v1 and v2c use a community string in clear text; v3 adds authentication (MD5/SHA) and privacy (DES/AES).

Discovery vs polling. Discovery finds what exists on the device (interfaces, sensors, modules) and runs every 6 hours; polling collects the values every 5 minutes.

Interface metrics. Utilization from octet counters, errors and discards, operational status, speed, and packet counts. Counters are monotonic and wrap; a rate is the difference divided by time. Always use 64-bit counters on fast links.

Errors vs discards. Errors are damaged frames (CRC, runts, giants, alignment) and indicate a physical or duplex problem. Discards are good frames dropped on purpose, typically from congestion, full queues or policies.

Alert logic. A rule is a condition on collected data, with a severity, delay, and count. States: active, acknowledged, recovered. Alerts go through transports to people or systems.

Baseline. Normal behaviour over time. A utilization alert is meaningful only against a baseline and capacity.

LibreNMS vs Zabbix. LibreNMS is network-device-centric, auto-discovery and SNMP-first, with very little configuration; Zabbix is a general monitoring platform with agents, templates and flexible triggers, needing more setup.

Typical viva questions: Which protocol and port does LibreNMS use? (SNMP, UDP 161.) What is `ifOperStatus`? (Actual link state: up or down.) What is the difference between oper and admin status? (Operational is the real state; admin is the configured state.) How is utilization computed? (Octet delta x 8 / time / speed.) Why use 64-bit counters? (A 32-bit counter wraps in 34 seconds at 1 Gbps.) How often does it poll? (Every 5 minutes by default.) What does a rising error counter at low load suggest? (A physical-layer problem.) How do you avoid alert flapping? (Delay, count, thresholds with recovery.) How do you add a device from the CLI? (`lnms device:add`.) Where are the logs? (`/opt/librenms/logs` and `storage/logs/librenms.log`.)

Lab 6: Security event monitoring using Wazuh

Scenario: detect suspicious authentication activity on a monitored Ubuntu server. Marks: Task 1 (2) verify Wazuh manager and agent connectivity; Task 2 (3) generate approved test authentication events or ingest a supplied event dataset; Task 3 (3) analyze alerts to identify affected hosts, event frequency, severity and possible causes; Task 4 (2) configure or demonstrate a local detection rule and recommend an incident-response action.

Honesty note: a Wazuh server cannot run in my sandbox. Paths, ports and commands below are from the Wazuh 4.x documentation. The alert-analysis script in Task 3 was run and tested on a sample `alerts.json`. Rule IDs shown (5710, 5760, 5712, 5715) are the standard SSH rules; confirm them in your version's rule files if the exam output differs.

Setup and how it works

Component	Role	Port
Wazuh agent	Runs on the monitored host; reads logs, checks file integrity, inventories software	outbound to manager

Component	Role	Port
Wazuh manager (server)	Receives events, decodes them, matches rules, generates alerts	1514/tcp (agent events), 1515/tcp (enrollment), 55000/tcp (API)
Wazuh indexer	Stores alerts (OpenSearch based)	9200/tcp
Wazuh dashboard	Web interface	443/tcp

Key paths on the manager: configuration `/var/ossec/etc/ossec.conf`, custom rules `/var/ossec/etc/rules/local_rules.xml`, alerts `/var/ossec/logs/alerts/alerts.json` and `alerts.log`, service log `/var/ossec/logs/ossec.log`. On the agent: `/var/ossec/etc/ossec.conf` and `/var/ossec/logs/ossec.log`.

Task 1: verify Wazuh manager and agent connectivity

On the **manager**:

```
sudo systemctl status wazuh-manager --no-pager
sudo /var/ossec/bin/wazuh-control status                    # wazuh-analysisd,
remoted, etc. must say "is running"
sudo /var/ossec/bin/agent_control -l                        # list agents;
Active = connected
sudo /var/ossec/bin/agent_control -i 001                    # details of agent
001: name, IP, status, last keep alive
sudo ss -tlnp | grep -E '1514|1515|55000'                   # manager ports
sudo tail -n 30 /var/ossec/logs/ossec.log                   # look for "Agent
... connected" and for ERROR lines
```

On the **agent**:

```
sudo systemctl status wazuh-agent --no-pager
sudo grep -A3 '<server>' /var/ossec/etc/ossec.conf          # <address> must be
the manager's IP
sudo tail -n 30 /var/ossec/logs/ossec.log                   # look for
"Connected to the server"
nc -vz MANAGER_IP 1514; nc -vz MANAGER_IP 1515
```

If the agent is missing, enroll it: `sudo /var/ossec/bin/agent-auth -m MANAGER_IP` then `sudo systemctl restart wazuh-agent` (modern installs set `WAZUH_MANAGER` during installation and enroll automatically). Dashboard check: **Agents** page shows the agent as **Active** (green), with its last keep-alive time and OS.

What to report: the manager services are running, the agent status is Active, the agent's IP, ID and name, the version, and the last keep-alive.

Task 2: generate test authentication events or ingest a dataset

Option A: real failed SSH logins (approved test on the monitored host):

```
sudo apt install -y sshpass     # install first if the command is missing
# make 6 failed attempts with a user that does not exist
for i in 1 2 3 4 5 6; do sshpass -p wrongpass ssh -o StrictHostKeyChecking=no
-o PreferredAuthentications=password nosuchuser@localhost true 2>/dev/null;
done
sudo grep -E 'Failed password|Invalid user' /var/log/auth.log | tail
```

If `sshpass` is not available, run `ssh nosuchuser@localhost` and type a wrong password repeatedly.

Option B: write synthetic log lines into the log the agent monitors (quick and safe; the agent reads `/var/log/auth.log` by default):

```
for i in 1 2 3 4 5 6 7 8 9 10; do logger -p auth.info -t sshd "Failed
password for invalid user admin from 203.0.113.50 port 4$i ssh2"; sleep 1;
done
```

The messages are in `/var/log/auth.log` as `sshd[...]`, so the manager decodes them as SSH events. The tenth within 2 minutes typically triggers rule **5712** ("sshd: brute force trying to get access to the system", level 10).

Option C: test a log line against rules without sending an event (on the manager):

```
sudo /var/ossec/bin/wazuh-logtest
# paste the line, press Enter:
# Nov  3 10:15:07 web01 sshd[1203]: Failed password for root from 203.0.113.5
port 52130 ssh2
# output shows Phase 1 predecoder, Phase 2 decoder (sshd), Phase 3 rule id,
level and description
```

See the result: dashboard > **Security events** (or the Threat Hunting module) filtered by the agent, or on the manager `sudo tail -f /var/ossec/logs/alerts/alerts.json`.

If a **supplied dataset** is provided, it is usually an `alerts.json` (one JSON alert per line) or a log file; analyze it with the script below.

Task 3: analyze alerts for hosts, frequency, severity and causes

Useful fields of each alert in `alerts.json`: `timestamp`, `rule.id`, `rule.level`, `rule.description`, `agent.name`, `data.srcip`, `data.dstuser`, `full_log`. Quick shell checks (needs `jq`; `sudo apt install -y jq`):

```
F=/var/ossec/logs/alerts/alerts.json        # or the supplied file
jq -r '.agent.name' $F | sort | uniq -c | sort -rn                      #
alerts per host
jq -r '.rule.level' $F | sort -n | uniq -c                               #
per severity
jq -r '[.rule.id, .rule.description] | @tsv' $F | sort | uniq -c | sort -rn
# per rule
jq -r '.data.srcip // empty' $F | sort | uniq -c | sort -rn             # per
source IP
jq -r 'select(.rule.level >= 10) | [.timestamp, .agent.name,
.rule.description] | @tsv' $F   # high severity
jq -r '.timestamp[0:16]' $F | uniq -c                                   #
alerts per minute
```

Save `wazuhscan.py`. It reads `alerts.json` (or asks for the path), summarizes per host, level, rule, source IP and user, gives the alert frequency, lists the high-severity alerts, and warns when a successful login (rule 5715) follows failures from the same IP. `--minlevel N` filters low levels.

```
#!/usr/bin/env python3
"""Lab 6: summarise Wazuh alerts.json (one JSON object per line).
Usage: python3 wazuhscan.py [alerts.json] [--minlevel N]   (prompts for the
path if omitted)"""
import argparse, json, sys
from collections import Counter
from datetime import datetime
ap = argparse.ArgumentParser(); ap.add_argument("file", nargs="?");
ap.add_argument("--minlevel", type=int, default=0)
a = ap.parse_args()
path = a.file or input("alerts.json path: ").strip()
try:
    lines = open(path, errors="replace").read().splitlines()
except OSError as e:
    sys.exit(f"Cannot read {path}: {e}")
ev, bad = [], 0
for ln in lines:
    if not ln.strip():
        continue
    try:
        j = json.loads(ln)
```

```
        ts = datetime.strptime(j["timestamp"][:19], "%Y-%m-%dT%H:%M:%S")
        ev.append(dict(ts=ts, host=j["agent"]["name"], rid=j["rule"]["id"],
lvl=int(j["rule"]["level"]),
                       desc=j["rule"]["description"], ip=j.get("data",
{}).get("srcip", "-"), user=j.get("data", {}).get("dstuser", "-")))
    except (ValueError, KeyError):
        bad += 1
ev = [e for e in ev if e["lvl"] >= a.minlevel]
if not ev:
    sys.exit("No alerts to analyse (check the path/format, or lower --
minlevel).")
print(f"Alerts: {len(ev)} (unparseable lines: {bad}) | period {min(e['ts']
for e in ev)} to {max(e['ts'] for e in ev)}")
print("\nPer host:", dict(Counter(e["host"] for e in ev).most_common()))
print("Per severity level:", dict(sorted(Counter(e["lvl"] for e in
ev).items(), reverse=True)))
print("\nPer rule (id, level, description):")
for (rid, lvl, d), c in Counter((e["rid"], e["lvl"], e["desc"]) for e in
ev).most_common():
    print(f"  {c:3d}  {rid:<6} L{lvl:<2} {d}")
print("\nSource IPs:", dict(Counter(e["ip"] for e in ev if e["ip"] != "-
").most_common()))
print("Targeted users:", dict(Counter(e["user"] for e in ev if e["user"] !=
"-").most_common()))
span = max(e["ts"] for e in ev) - min(e["ts"] for e in ev)
print(f"Frequency: {len(ev) / max(span.total_seconds() / 60, 1):.1f}
alerts/min over {span}")
high = [e for e in ev if e["lvl"] >= 10]
print(f"\nHigh severity (>=10): {len(high)}")
for o in [e for e in ev if e["rid"] == "5715"]:
    if any(e["ip"] == o["ip"] and e["ts"] <= o["ts"] and e["rid"] != "5715"
for e in ev):
        print(f"  POSSIBLE COMPROMISE: success for {o['user']} from {o['ip']}
on {o['host']} at {o['ts']} after earlier failures from that IP")
```

Test dataset (builds a 10-alert `alerts_sample.json`; then run the script):

```
python3 - <<'EOF'
import json
rows=[("2026-11-03T10:15:01","web01","5710",5,"sshd: Attempt to login using a
non-existent user","203.0.113.5","admin"),
("2026-11-03T10:15:04","web01","5710",5,"sshd: Attempt to login using a non-
existent user","203.0.113.5","admin"),
("2026-11-03T10:15:07","web01","5760",5,"sshd: authentication
failed.","203.0.113.5","root"),
("2026-11-03T10:15:10","web01","5760",5,"sshd: authentication
failed.","203.0.113.5","root"),
("2026-11-03T10:15:13","web01","5760",5,"sshd: authentication
failed.","203.0.113.5","root"),
("2026-11-03T10:15:14","web01","5712",10,"sshd: brute force trying to get
access to the system.","203.0.113.5","-"),
("2026-11-03T10:15:20","web01","5715",3,"sshd: authentication
success.","203.0.113.5","root"),
("2026-11-03T10:16:02","db01","5760",5,"sshd: authentication
failed.","198.51.100.7","postgres"),
("2026-11-03T10:16:30","db01","5760",5,"sshd: authentication
failed.","198.51.100.7","postgres"),
("2026-11-03T10:40:00","app01","5710",5,"sshd: Attempt to login using a non-
existent user","198.51.100.99","test")]
with open("alerts_sample.json","w") as f:
    for t,h,r,l,d,ip,u in rows:
        o={"timestamp":t+".000+0000","rule":
{"id":r,"level":l,"description":d},"agent":{"name":h},"data":{"srcip":ip}}
        if u!="-": o["data"]["dstuser"]=u
        f.write(json.dumps(o)+"\n")
EOF
python3 wazuhscan.py alerts_sample.json
```

Result to expect	Value
Alerts	10, period 10:15:01 to 10:40:00
Per host	web01 7, db01 2, app01 1
Per level	10: 1, 5: 8, 3: 1
Per rule	5760 x5 (L5), 5710 x3 (L5), 5712 x1 (L10), 5715 x1 (L3)
Source IPs	203.0.113.5: 7, 198.51.100.7: 2, 198.51.100.99: 1
Users	root 4, admin 2, postgres 2, test 1
Frequency	0.4 alerts/min over 0:24:59

Result to expect	Value
Compromise warning	root from 203.0.113.5 on web01 at 10:15:20
`--minlevel 10`	only the one rule 5712 alert remains

Possible causes to name: an automated SSH brute-force or credential-stuffing attack from one external address (203.0.113.5 tried admin, then root, in 15 seconds and then succeeded), dictionary attacks against service accounts such as `postgres`, and a lone probe with a default user (`test`). Benign causes to rule out: an administrator typing the wrong password, a misconfigured script or monitoring tool with an old password, a user locked out after a password change. Severity logic: a single failure is level 5 (low), repeated failures escalate to level 10 (high) through frequency-based rules, and a success after failures is the most important event.

Task 4: local detection rule and incident-response action

Custom rules go in `/var/ossec/etc/rules/local_rules.xml` on the manager, with IDs from 100000 upwards (lower IDs are reserved for built-in rules). Example: an alert when five SSH authentication failures from one IP occur within 60 seconds (a tighter version of the built-in rule), plus a rule that catches a login by root.

```
<group name="local,syslog,sshd,">
  <rule id="100100" level="10" frequency="5" timeframe="60">
    <if_matched_sid>5760</if_matched_sid>
    <same_source_ip />
    <description>Local: 5 SSH authentication failures from the same IP within
60 seconds</description>
    <mitre><id>T1110</id></mitre>
  </rule>
  <rule id="100101" level="12">
    <if_sid>5715</if_sid>
    <user>root</user>
    <description>Local: successful SSH login as root</description>
  </rule>
</group>
```

```
sudo nano /var/ossec/etc/rules/local_rules.xml            # add the rules
inside the existing file
sudo /var/ossec/bin/wazuh-analysisd -t                    # syntax test of
rules; no output = OK (prints errors otherwise)
sudo systemctl restart wazuh-manager
# demonstrate: send 6 failures with logger (Task 2, option B) then look for
rule 100100
sudo grep 100100 /var/ossec/logs/alerts/alerts.json | tail -2
```

You can also demonstrate without a restart-loop using `wazuh-logtest`: paste several failure lines in one session; the final line should show `rule id 100100` once five are within the timeframe (logtest keeps the session state). Explain each attribute: `level` severity 0 to 15, `frequency` and `timeframe` count events within seconds, `if_matched_sid` rule(s) that must repeat, `same_source_ip` groups per source, `if_sid` fires on a single event of that parent rule, `mitre` maps to ATT&CK technique T1110 (brute force).

A simpler example that matches text directly (use when the log format is non-standard):

```
<rule id="100110" level="8">
  <decoded_as>sshd</decoded_as>
  <match>Failed password for invalid user</match>
  <description>Local: SSH login attempt for a non-existent user</description>
</rule>
```

Recommended incident-response actions (name them in this order): (1) confirm that it is an attack (source IP, frequency, targeted users, any success); (2) contain: block the source address (`sudo ufw deny from 203.0.113.5`) or let Wazuh's **active response** do it automatically; (3) if there was a success, treat the account as compromised: lock it (`sudo passwd -l root`), end its sessions (`pkill -KILL -u USER` or `loginctl terminate-user`), reset credentials, review `last` , `lastb` , `auth.log` and new users or keys; (4) harden: disable password login (`PasswordAuthentication no`), `PermitRootLogin no` , key-based login, `fail2ban` , MFA, restrict SSH by firewall; (5) document the incident and lessons learned.

Active response in `ossec.conf` on the manager (uses a built-in script):

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100100</rules_id>
  <timeout>600</timeout>
</active-response>
```

It blocks the source address on the agent for 600 seconds when the rule fires. Name it as a recommendation; enabling it in an exam is risky because a wrong rule could lock out your own address.

Likely examiner modifications

- Change the threshold or timeframe (for example 3 failures in 30 s) and the level.
- Rule for a different event: sudo failures (`sudo: ... authentication failure`), a new user creation (`useradd`), a modified `/etc/passwd` (File Integrity Monitoring), or a root login.
- Use `wazuh-logtest` to show which rule matches a given log line, and explain the three phases.
- Filter the dashboard by agent, by rule level above 7, or by a time range, and screenshot it.
- Compare alerts between two hosts and say which is more attacked.
- Explain the difference between **alerts** (what was matched) and **archives** (all events when logall is enabled).
- Modify an existing rule's level without editing the built-in file: override with `<rule id="5710" level="3" overwrite="yes">` in `local_rules.xml` .

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Agent shows `Disconnected` or `Never connected`	Wrong manager address, firewall, key mismatch	`ossec.log` on the agent; `nc -vz MANAGER 1514`	Correct `<address>` ; open 1514/1515; re-enroll with `agent-auth`
`Invalid authentication key` / `Unable to add agent`	Duplicate or stale agent entry	`agent_control -l` and `manage_agents`	Remove the stale agent on the manager and re-register
Rule changes have no effect	Manager not restarted, or syntax error	`wazuh-analysisd -t` ; `ossec.log` ERROR lines	Fix XML and restart the manager
`ERROR: Invalid element in the configuration`	Typo in XML (unclosed tag, wrong attribute)	The line number in the message	Correct the XML; keep backups of `local_rules.xml`

Symptom or message	Likely cause	Check	Fix
Custom rule ID rejected or overlaps	ID below 100000 or duplicated	Search the ID across `/var/ossec/ruleset`	Use unique IDs from 100000 upward
No alerts after `logger` messages	Agent not reading that log, message not in `auth.log`, or the level is below the alert threshold	`grep logger-test /var/log/auth.log`; `<localfile>` list in agent `ossec.conf`; `<log_alert_level>` on the manager	Use the correct tag and facility; lower `log_alert_level` for the test
Frequency rule never fires	Events too slow for the `timeframe`, or `same_source_ip` mismatch	`wazuh-logtest` with several lines	Send them faster or widen the timeframe
Alerts in `alerts.json` but not in the dashboard	Filebeat or indexer problem	`systemctl status filebeat wazuh-indexer`; `filebeat test output`	Fix certificates or the indexer; restart
Dashboard login fails or page not found	Wrong URL or credentials; the service is down	`systemctl status wazuh-dashboard`; use `https://IP`	Restart; use the supplied admin password
`sshpass: command not found`	Not installed	`which sshpass`	`sudo apt install -y sshpass`
SSH test locks me out	You triggered an active-response block on your own IP	`sudo iptables -L -n`	Wait for the timeout, or remove the rule with `sudo iptables -D INPUT -s IP -j DROP`
Time in alerts looks wrong	Time zone mismatch	`timedatectl` on both hosts	Align time zones and enable NTP

Viva theory

Wazuh. An open-source security platform combining a SIEM (log analysis, correlation, alerting) and an XDR/HIDS capability (host intrusion detection). It evolved from OSSEC. Capabilities: log analysis, file integrity monitoring (FIM), vulnerability detection,

configuration assessment (SCA), rootkit detection, active response, and compliance mapping (PCI DSS, GDPR, MITRE ATT&CK).

Architecture. Agents collect and forward events (TLS-secured, AES-encrypted) to the manager. The manager runs **analysisd**, which does pre-decoding, decoding (extracts fields using decoders) and rule matching (rules produce alerts with an ID and level). Alerts go to the indexer (via Filebeat) and are shown in the dashboard.

Event pipeline in three phases (visible in `wazuh-logtest`): (1) pre-decoding extracts timestamp, host and program from the syslog header; (2) decoding identifies the program (sshd) and extracts fields (srcip, dstuser); (3) rule matching assigns the rule ID, level and description.

Rule levels. 0 ignored, 1 to 4 low (system notifications), 5 to 7 user errors and low-importance events, 8 to 11 important or suspicious events, 12 to 15 critical (attacks). The default alert threshold is level 3.

Brute force vs password spraying vs credential stuffing. Brute force tries many passwords against one account; spraying tries one common password against many accounts; credential stuffing uses leaked username and password pairs. Indicators: failures from one IP, many usernames, failure then success.

HIDS vs NIDS. A host-based IDS (Wazuh) watches logs, files and processes on the host; a network IDS (Suricata, Snort) watches packets. They complement each other.

Active response. A defined command (for example `firewall-drop`) that the manager tells an agent to run on a rule match, for a timeout period. It enables automatic containment but can cause denial of service if abused.

FIM. Syscheck monitors files and registry keys for changes (hash, size, permissions, owner), important for `/etc`, binaries and web roots.

Incident response phases (NIST). Preparation, detection and analysis, containment, eradication and recovery, and post-incident activity.

Typical viva questions: Which port do agents use? (1514/tcp; enrollment on 1515.) Where are alerts stored? (`/var/ossec/logs/alerts/alerts.json`.) Why start custom rule IDs at 100000? (Lower IDs belong to the built-in ruleset.) What does `wazuh-logtest` do? (Tests a log line against decoders and rules without generating an alert.) Which rule detects SSH brute force? (5712, via repeated 5760 or 5710.) What is the difference between a decoder and a rule? (A decoder extracts fields; a rule decides if an alert is raised.) What does level 10 mean? (A high-severity, important event.) What does `frequency` with `timeframe` mean? (That many matching events within that many seconds.) What would you do on a successful login after many failures? (Treat as compromise: contain, lock, investigate, reset.)

Lab 7: Log analysis using ELK or Grafana Loki

Scenario: analyze web application logs to identify service failures. Marks: Task 1 (2) verify the existing log-ingestion pipeline and identify the assigned log source; Task 2 (3) filter logs by timestamp, host, severity or HTTP status code; Task 3 (3) calculate the HTTP error rate from the supplied dataset and identify the period of maximum failures; Task 4 (2) create a visualization showing errors over time and explain the observed pattern.

Honesty note: Elasticsearch, Kibana, Loki and Grafana are not run in my sandbox, so queries and menu paths come from their documentation (ELK 8.x, Loki 2.9/3.x). The error-rate script in Task 3 was run and tested on a generated log, and it also produces the chart in Task 4. You will use either ELK or Loki, whichever the exam provides, so read the section for your stack and keep the script as the fallback that always works.

Setup and how the pieces fit

	ELK (Elastic)	Loki (Grafana)
Shipper	Filebeat (or Logstash)	Promtail (or Grafana Alloy)
Store and index	Elasticsearch (indexes all fields; port 9200)	Loki (indexes only labels; stores log text compressed; port 3100)
UI	Kibana (port 5601)	Grafana (port 3000), Explore
Query language	KQL, Lucene, ES|QL	LogQL

Task 1: verify the log-ingestion pipeline and identify the log source

ELK.

```
sudo systemctl status elasticsearch kibana filebeat --no-pager
curl -s -k -u elastic:PASSWORD https://localhost:9200/_cluster/health?pretty
# status green or yellow; 8.x uses HTTPS and a password (-k skips cert check,
test only)
# 7.x or security disabled:  curl -s localhost:9200/_cluster/health?pretty
curl -s -k -u elastic:PASSWORD 'https://localhost:9200/_cat/indices?v'
# filebeat-* or logs-* index; docs.count must be growing
sudo filebeat test config -c /etc/filebeat/filebeat.yml
# Config OK
sudo filebeat test output
# connection to Elasticsearch/Logstash: OK
sudo grep -A6 'filebeat.inputs' /etc/filebeat/filebeat.yml
# the assigned log path, e.g. /var/log/nginx/access.log
sudo filebeat modules list | head
# nginx module enabled?
sudo journalctl -u filebeat -n 30 --no-pager
```

In Kibana: **Stack Management > Data Views** (older: Index Patterns) should list a data view such as `filebeat-*`; **Discover** with that data view must show recent events. Identify the log source from fields such as `log.file.path`, `host.name`, `service.name`, `event.dataset` (for example `nginx.access`).

Loki.

```
sudo systemctl status loki promtail grafana-server --no-pager
curl -s localhost:3100/ready                                  # ready
curl -s localhost:3100/loki/api/v1/labels                     # labels such
as job, filename, host
curl -s localhost:3100/loki/api/v1/label/job/values           # values of the
job label = log sources
curl -s localhost:9080/targets | head -30                     # promtail:
files it tails (active targets)
sudo grep -B2 -A12 'scrape_configs' /etc/promtail/config.yml  # the path,
labels and job name of the assigned source
sudo journalctl -u promtail -n 30 --no-pager
```

In Grafana: **Connections > Data sources** should list Loki (click **Save & test**: "Data source successfully connected"), then **Explore**, choose Loki, and pick the label and value (for example `job = nginx`) to see the stream.

Task 2: filter by timestamp, host, severity or status code

ELK (Kibana Discover, KQL in the search bar, time range in the top-right picker):

```
http.response.status_code >= 500
http.response.status_code : 404 and host.name : "web01"
nginx.access.remote_ip : "203.0.113.5"
log.level : "error"
url.original : "/api/exam/submit" and not http.response.status_code : 200
message : "timeout"
```

Set the time range with the picker (for example "Absolute" 10:00 to 10:10), add columns from the left field list (`@timestamp` , `host.name` , `http.response.status_code` , `url.original`), and use **Filter** pills or click a field value to include or exclude. API form of the same query:

```
curl -s -k -u elastic:PASSWORD -H 'Content-Type: application/json'
'https://localhost:9200/filebeat-*/_count' -d '{"query":{"range":
{"http.response.status_code":{"gte":500}}}}'
```

Loki (Grafana Explore, LogQL):

```
{job="nginx"}                                              all lines of the
source
{job="nginx", host="web01"}                                one host (labels
must exist)
{job="nginx"} |= "500"                                     lines containing
500 (crude)
{job="nginx"} |~ "\" 5[0-9]{2} "                           regex for HTTP 5xx
in the access-log format
{job="nginx"} != "healthcheck"                             exclude noise
{job="nginx"} | pattern `<ip> - - [<ts>] "<method> <path> <_>" <status>
<size> <_>` | status >= 500     parse then filter numerically
{job="app"} | json | level="error"                         JSON logs filtered
by severity
```

The operators: `|=` contains, `!=` not contains, `|~` regex match, `!~` regex not match. Time range is selected with the picker at the top right; the query-range API accepts `start` and `end` . On the command line with `logcli` :

```
logcli query '{job="nginx"} |~ " 5[0-9]{2} "' --since=1h --limit=50     #
needs logcli installed
curl -G -s localhost:3100/loki/api/v1/query_range --data-urlencode 'query=
{job="nginx"}' --data-urlencode 'limit=5'
```

Plain shell on the supplied file (always works):

```
LOG=access.log
awk '$9 >= 500' $LOG | wc -l                                                    #
count of 5xx  (field 9 = status in combined format)
awk '$9 == 404' $LOG | head
awk '$4 >= "[03/Nov/2026:10:03" && $4 < "[03/Nov/2026:10:05"' $LOG | head
# time window (string comparison)
awk '{print $9}' $LOG | sort | uniq -c | sort -rn                               #
status distribution
grep ' 502 ' $LOG | awk '{print $7}' | sort | uniq -c | sort -rn                #
which URL returns 502
```

Task 3: HTTP error rate and the period of maximum failures

Definitions: **error rate = 5xx responses / total responses x 100**. Report 4xx separately (client errors are usually not a server fault, but a spike in 404/401/403 can show scanning or broken links). Availability (success rate) is `100 - error rate` or counts 2xx and 3xx as success. The peak period is the minute (or bucket) with the most 5xx, and it is also useful to report the highest percentage.

In Kibana: Visualize/Lens: a Date Histogram (`@timestamp` , interval 1 minute) of Count with the filter `http.response.status_code >= 500` , compared with the total count. **In Loki:** the error rate per minute:

```
sum(count_over_time({job="nginx"} | pattern `<_> - - [<_>] "<_> <_> <_>"
<status> <_> <_>` | status >= 500 [1m]))
  /
sum(count_over_time({job="nginx"} [1m])) * 100
```

Save `weblog.py` . It parses a combined-format access log (nginx or Apache), buckets by minute (or `--bucket N` minutes), prints the totals, status distribution, a table with a text bar chart, the peak period and the URLs that fail the most, and with `--png file.png` draws the chart.

```
#!/usr/bin/env python3
"""Lab 7: HTTP error-rate analysis of an nginx/Apache combined access log.
Usage: python3 weblog.py [LOG] [--bucket MIN] [--png out.png]   (prompts for
LOG if omitted)"""
import argparse, re, sys
from collections import Counter
from datetime import datetime
ap = argparse.ArgumentParser(); ap.add_argument("log", nargs="?");
ap.add_argument("--bucket", type=int, default=1); ap.add_argument("--png")
a = ap.parse_args()
path = a.log or input("Access log path: ").strip()
```

```
RX = re.compile(r'^(?P<ip>\S+) \S+ \S+ \[(?P<ts>[^\]]+)\] "(?P<req>[^"]*)" (?
P<st>\d{3}) (?P<sz>\S+)')
try:
    lines = open(path, errors="replace").read().splitlines()
except OSError as e:
    sys.exit(f"Cannot read {path}: {e}")
tot, err5, err4 = Counter(), Counter(), Counter()
status, urls, bad = Counter(), Counter(), 0
for ln in lines:
    m = RX.match(ln)
    if not m:
        bad += 1; continue
    t = datetime.strptime(m["ts"].split()[0], "%d/%b/%Y:%H:%M:%S")
    t = t.replace(minute=t.minute - t.minute % a.bucket, second=0)
    st = int(m["st"]); parts = m["req"].split()
    tot[t] += 1; status[st] += 1
    if st >= 500: err5[t] += 1; urls[parts[1] if len(parts) > 1 else "?"] +=
1
    elif st >= 400: err4[t] += 1
n = sum(tot.values())
if not n:
    sys.exit("No parsable lines (expect combined log format).")
e5, e4 = sum(err5.values()), sum(err4.values())
print(f"Requests {n} (unparsed {bad}) | 5xx {e5} = {e5/n*100:.2f}% | 4xx {e4}
= {e4/n*100:.2f}% | success {(n-e5-e4)/n*100:.2f}%")
print("Status codes:", dict(sorted(status.items())))
print(f"\n{'Period':<17}{'Req':>6}{'5xx':>5}{'Rate%':>8}")
for t in sorted(tot):
    r = err5[t] / tot[t] * 100
    print(f"{t:%d %b %H:%M}     {tot[t]:6d}{err5[t]:5d}{r:8.1f}  {'#' * int(r
/ 2)}")
w = max(tot, key=lambda t: (err5[t], err5[t] / tot[t]))
print(f"\nPeak failures: {w:%H:%M} with {err5[w]} 5xx of {tot[w]} requests
({err5[w]/tot[w]*100:.1f}%)")
print("Top failing URLs:", dict(urls.most_common(3)))
if a.png:
    try:
        import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt
        ts = sorted(tot); plt.figure(figsize=(9, 3.5))
        plt.plot(ts, [err5[t] / tot[t] * 100 for t in ts], marker="o");
plt.ylabel("5xx error rate %"); plt.title("HTTP 5xx rate over time");
plt.grid(alpha=.3)
        plt.tight_layout(); plt.savefig(a.png, dpi=120); print("Saved",
```

```
a.png)
    except ImportError:
        print("matplotlib missing: pip install matplotlib (the table above is
the fallback)")
```

Test dataset (deterministic generator: 7 minutes x 20 requests, with a failure burst around 10:04):

```
cat > gen_access.py <<'EOF'
plan = {0: (20, 0, 1), 1: (20, 0, 0), 2: (20, 1, 0), 3: (20, 4, 1), 4: (20,
10, 0), 5: (20, 2, 0), 6: (20, 0, 0)}   # minute: (requests, 5xx, 404)
rows = []
for m, (tot, e5, e4) in plan.items():
    for i in range(tot):
        s = i * 60 // tot
        if i < e5: st, u = [500, 502, 503][i % 3], "/api/exam/submit"
        elif i < e5 + e4: st, u = 404, "/old.html"
        else: st, u = 200, ["/", "/login", "/api/exam/list"][i % 3]
        rows.append(f'10.0.0.{2 + i} - - [03/Nov/2026:10:{m:02d}:{s:02d}
+0000] "GET {u} HTTP/1.1" {st} 512 "-" "curl"')
open("access_sample.log", "w").write("\n".join(rows) + "\n")
EOF
python3 gen_access.py && python3 weblog.py access_sample.log
```

Result to expect	Value
Totals	140 requests, 17 five-hundred = 12.14%, 2 four-hundred = 1.43%, success 86.43%
Status codes	200: 121, 404: 2, 500: 8, 502: 5, 503: 4
Per minute 5xx	10:02 1 (5%), 10:03 4 (20%), 10:04 10 (50%), 10:05 2 (10%), others 0
Peak	10:04 with 10 of 20 (50.0%)
Failing URL	`/api/exam/submit` for all 17
`--bucket 5`	10:00 bucket 15 of 100 (15%), 10:05 bucket 2 of 40 (5%)
Wrong or empty file	`No parsable lines` / `Cannot read ...` message

Cross-check with the shell: `awk '$9>=500' access_sample.log | wc -l` must print 17, and `awk '$9>=500 {print substr($4,2,17)}' access_sample.log | sort | uniq -c` shows the minute counts. A larger bucket hides a short burst (the 5-minute bucket shows only 15%, not the 50% peak), which is a good viva point about aggregation windows.

Task 4: visualization of errors over time and its explanation

Kibana (Lens): Analytics > Visualize Library > Create > Lens (or Dashboard > Create visualization). Choose the data view, drag `@timestamp` to the horizontal axis (Date histogram, interval 1 minute), set the vertical axis to **Count** of records, and add the filter `http.response.status_code >= 500` in the query bar. Optionally break down by `http.response.status_code` (stacked bars), or add a formula for the rate, `count(kql='http.response.status_code >= 500') / count() * 100`. Save it, then add it to a dashboard.

Grafana with Loki: Dashboards > New > Add visualization, data source Loki, query `sum by (status) (count_over_time({job="nginx"} | pattern ... | status >= 500 [1m]))`, visualization **Time series** (or Bars), title "HTTP 5xx per minute", legend by status. Apply, then Save dashboard.

Script: `python3 weblog.py access_sample.log --png errors.png` writes a line chart of the 5xx rate (needs `pip install matplotlib` if missing; the text table is a fallback).

What to say about the pattern in the test data: "Errors rose from 5% at 10:02 to a 50% peak at 10:04 and fell back to 0% by 10:06. A burst that rises and falls, confined to one endpoint (`/api/exam/submit`), with a mix of 500, 502 and 503, indicates a transient backend failure or overload (for example a database connection pool or an upstream restart) and not a permanent outage. 502/503 point to the proxy not reaching healthy backends; 500 points to application errors. Recovery means the service self-healed or was restarted. Next steps: correlate with deploy time, CPU and memory, and application logs for the same minutes."

Pattern vocabulary: **spike** (short sharp rise), **step** (sudden permanent change, such as a bad deploy), **ramp** (gradual rise, such as a leak or growing load), **periodic** (cron jobs or backups at the same time daily), **plateau** (steady failure, such as a dead dependency).

Likely examiner modifications

- Compute a different rate: 4xx rate, 404 rate, or the rate for one URL or one host: `awk '$7=="/login" && $9>=500' log | wc -l`.
- A different bucket (5 minutes or hourly) or a given time window.
- Show the top 5 client IPs, top URLs, or the slowest requests (if the log has response time).
- A different log format (JSON, or Apache common log): change the regex, or for JSON use `jq -r 'select(.status>=500)'`.
- Filter by severity in application logs: `grep -E 'ERROR|CRITICAL'`, KQL `log.level : ("error" or "critical")`, LogQL `| json | level=~"error|critical"`.

- Create an alert on error rate (Grafana alert rule or Kibana rule) with a threshold such as above 5% for 5 minutes.
- Explain the label-cardinality problem in Loki (never use IP or user ID as a label).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Kibana Discover shows no data	Wrong time range, wrong data view, or nothing ingested	Set the range to "Last 24 hours"; `_cat/indices` doc count	Widen the time range; select the right data view; fix the shipper
`curl: (52) Empty reply` / `SSL certificate problem` / `missing authentication credentials`	Elasticsearch 8 has HTTPS and authentication on	The URL scheme and `-u elastic:PW`	Use `https`, `-k` (test only) or `--cacert`, supply the password
`curl: (7) Failed to connect to localhost port 9200`	Elasticsearch down or still starting	`systemctl status elasticsearch`; `journalctl -u elasticsearch -n 50`	Start it and wait (it takes about a minute); check Java heap (`jvm.options`) and disk
Cluster status `red` / `yellow`	Unassigned shards; single node with replicas	`_cluster/health?pretty`; `_cat/shards`	On a single node yellow is normal; fix disk watermarks if red
Filebeat `connection refused` / `x509` errors	Wrong host, port or certificates	`filebeat test output`	Correct `output.elasticsearch.hosts`, credentials, and `ssl.certificate_authorities`
Filebeat runs but no new documents	Log path wrong, file permission, or the registry thinks the file was read	`grep paths filebeat.yml`; `ls -l /var/log/nginx`	Fix the path; add filebeat to the `adm` group; use `filebeat -e -d "*"` to debug
Fields such as `http.response.status_code` missing	Raw message not parsed (module disabled)	A document in Discover shows only `message`	`sudo filebeat modules enable nginx`, `filebeat setup`, restart
Kibana `Kibana server is not ready yet`	Elasticsearch not reachable or the enrollment/token problem	`journalctl -u kibana -n 50`	Fix Elasticsearch first; check `elasticsearch.hosts` in `kibana.yml`
Loki `Ingester not ready` right after start	Normal during the first 15 seconds	`curl localhost:3100/ready`	Wait
Grafana Explore: `No data`	Wrong label, wrong time range, or promtail not shipping	`curl localhost:3100/loki/api/v1/labels`; the promtail `/targets` page	Use labels that exist; widen the range; fix the promtail path and permissions
LogQL `parse error`	Unescaped quotes, missing backticks, or bad regex	The caret in the error text	Use backticks around patterns/regex; escape inner quotes
Promtail `permission denied` on a log file	The promtail user cannot read it	`ls -l`; `journalctl -u promtail`	`sudo usermod -aG adm promtail` and restart
Promtail `entry out of order` / `too far behind`	Old timestamps or clock skew	Loki logs	Fix the clock; check `reject_old_samples_max_age`

Symptom or message	Likely cause	Check	Fix
`weblog.py` parses 0 lines	The log is not in combined format	`head -2 log`	Adapt the regex, or convert with awk
Time zone offsets confuse the peak time	UI shows the browser's time zone, the log uses UTC	Compare one event	Set the same zone in the UI or convert both

Viva theory

Why centralized log analysis. It correlates events across servers, speeds up troubleshooting, supports security investigations and compliance, and keeps evidence when a server is lost.

ELK stack. Elasticsearch is a distributed search and analytics engine that stores JSON documents in indices and builds an inverted index of every field. Logstash is a data-processing pipeline (input, filter, output) that parses, enriches and routes logs. Kibana visualizes and queries. Beats (Filebeat) are light shippers. Together with Beats it is sometimes called the Elastic Stack.

Loki. A log aggregation system inspired by Prometheus. It indexes only metadata labels (such as `job`, `host`) and stores compressed log chunks, so it is cheaper and simpler than Elasticsearch but slower for full-text search. Query language: LogQL.

ELK vs Loki. ELK indexes all content (powerful full-text search, aggregations, but heavy on memory, disk and tuning); Loki indexes labels only (low cost and tight Grafana integration, but queries scan the matching chunks).

Log pipeline stages. Collect (Filebeat/Promtail), parse and enrich (Logstash grok, Beats modules, LogQL parsers), store and index, query and visualize, alert and retain.

Parsing. Structured logs (JSON) are easy; unstructured ones need regex or grok patterns. The combined access log format is: `ip ident user [time] "request" status bytes "referer" "user-agent"`.

HTTP status classes. 1xx informational, 2xx success, 3xx redirect, 4xx client error, 5xx server error. 500 internal error, 502 bad gateway (bad response from upstream), 503 service unavailable (overload or maintenance), 504 gateway timeout. 429 too many requests.

Error rate vs error count. Rate is relative to traffic, so it shows health irrespective of load; a count rises merely because traffic rose.

Cardinality. Each unique label combination makes a new stream in Loki; high-cardinality labels (IP, user, request ID) slow and bloat the system. Keep such values in the log text and filter with `|=` or parsers.

Typical viva questions: What does Filebeat do? (Tails files and ships lines to Elasticsearch or Logstash.) Default ports? (Elasticsearch 9200, Kibana 5601, Logstash 5044, Loki 3100, Grafana 3000, Promtail 9080.) What is an index? (A collection of documents with a mapping.) What is KQL? (Kibana Query Language.) Why is Loki cheaper? (Only labels are indexed.) How do you find 5xx errors in LogQL? (`{job="nginx"} |~ " 5[0-9]{2} "`.) What is the difference between 502 and 504? (Bad answer from upstream vs timeout waiting for it.) What would you do if Kibana shows no data? (Check time range, data view, index doc count, shipper status.) What is a data view? (Kibana's definition of which indices to query; formerly an index pattern.) How do you compute error rate in PromQL or LogQL? (Divide the rate of 5xx by the rate of all requests.)

Lab 8: Prometheus and Grafana infrastructure monitoring

Scenario: monitor an Ubuntu server using Prometheus and Node Exporter. Marks: Task 1 (2) verify Prometheus and Node Exporter availability and confirm the target is scraped; Task 2 (3) write PromQL for CPU utilization, available memory and network receive rate; Task 3 (3) analyze supplied metrics to find the likely bottleneck; Task 4 (2) create a Grafana panel and configure a threshold or alert rule.

Tested here: I installed `promtool` (Prometheus 2.45) and validated the `prometheus.yml` and alert rules below with `promtool check config` and `check rules`, and I ran a `promtool test rules` unit test that confirmed the CPU and memory expressions give the expected numbers on synthetic data (50% CPU, 12.5% memory available). The diagnosis script in Task 3 was run on generated scrapes. Grafana's web UI could not be run, so the Grafana menu paths follow Grafana 10/11 documentation.

Setup and how the pieces fit

Component	Role	Port
Node Exporter	Exposes host metrics (CPU, memory, disk, network) as text at `/metrics`	9100
Prometheus	Scrapes targets every N seconds, stores time series, evaluates rules, serves PromQL	9090
Alertmanager	Receives firing alerts from Prometheus, groups and routes notifications	9093
Grafana	Dashboards and alerting UI on top of Prometheus as a data source	3000

Pull model: Prometheus fetches `http://target:9100/metrics` on a schedule; the target does not push.

Install on Ubuntu 22.04 (if not already present):

```
sudo apt update && sudo apt install -y prometheus prometheus-node-exporter
# Ubuntu packages (Prometheus 2.31); binaries from GitHub also work
sudo systemctl enable --now prometheus prometheus-node-exporter
# Grafana comes from Grafana's apt repository (internet is allowed in the
exam):
sudo apt install -y apt-transport-https software-properties-common wget
sudo mkdir -p /etc/apt/keyrings && wget -q -O -
https://apt.grafana.com/gpg.key | sudo gpg --dearmor -o
/etc/apt/keyrings/grafana.gpg
echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com
stable main" | sudo tee /etc/apt/sources.list.d/grafana.list
sudo apt update && sudo apt install -y grafana && sudo systemctl enable --now
grafana-server
```

The Prometheus configuration (`/etc/prometheus/prometheus.yml`) that scrapes the node exporter. I validated this file with `promtool`:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
rule_files:
  - "alert.rules.yml"
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
  - job_name: "node"
    static_configs:
      - targets: ["localhost:9100"]
```

```
promtool check config /etc/prometheus/prometheus.yml            # SUCCESS
means valid
sudo systemctl restart prometheus                                 # or reload:
curl -X POST localhost:9090/-/reload (needs --web.enable-lifecycle)
```

Task 1: verify Prometheus and Node Exporter and confirm the target is scraped

```
sudo systemctl status prometheus prometheus-node-exporter --no-pager     #
active (running)
curl -s localhost:9100/metrics | head -n 5                               #
node exporter answers with text metrics
curl -s localhost:9100/metrics | grep -c '^node_'                        #
many metric lines
curl -s localhost:9090/-/healthy                                          #
"Prometheus Server is Healthy."
curl -s localhost:9090/-/ready                                            #
"Prometheus Server is Ready."
curl -s 'localhost:9090/api/v1/targets' | python3 -m json.tool | grep -E '"
(job|health|lastError)"'    # health: up
curl -s 'localhost:9090/api/v1/query?query=up'                            #
value "1" for each target
sudo ss -tlnp | grep -E '9090|9100'                                       #
listening ports
```

In the Prometheus web UI (`http://localhost:9090`): **Status > Targets** must show the `node` target with state **UP**, the last scrape time and the scrape duration. In **Graph**, run `up` ; `1` means the scrape succeeded and `0` means it failed. For Grafana, **Connections > Data sources > Add > Prometheus**, URL `http://localhost:9090` , then **Save & test** ("Successfully queried the Prometheus API").

Task 2: PromQL for CPU utilization, available memory and network receive rate

Run these in the Prometheus UI (**Graph** tab), in Grafana **Explore**, or from the shell with `curl -s --data-urlencode 'query=EXPR' localhost:9090/api/v1/query` .

Metric	PromQL	Explanation
CPU utilization % (per instance)	`100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`	`node_cpu_seconds_total` is a counter of seconds per CPU and mode. The idle fraction is the per-second rate of idle time averaged over the CPUs; subtract from 100
CPU per mode %	`avg by (mode) (rate(node_cpu_seconds_total[5m])) * 100`	Shows user, system, iowait, idle shares
Memory available (bytes)	`node_memory_MemAvailable_bytes`	Memory that can be allocated without swapping
Memory available %	`node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100`	Best single memory health number

Metric	PromQL	Explanation
Memory used %	`(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100`	
Network receive rate (bytes/s)	`rate(node_network_receive_bytes_total{device="eth0"}[5m])`	Per-second average over 5 minutes
Network receive (Mbit/s)	`rate(node_network_receive_bytes_total{device!~"lo"}[5m]) * 8 / 1e6`	Excludes loopback; `device` regex selects real interfaces
Network transmit	`rate(node_network_transmit_bytes_total{device="eth0"}[5m])`	
Disk busy %	`rate(node_disk_io_time_seconds_total[5m]) * 100`	Share of time the disk was busy
Filesystem used %	`100 - node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100`	
Load per core	`node_load1 / count without(cpu, mode) (node_cpu_seconds_total{mode="idle"})`	Above 1 means a queue
Is the target up	`up{job="node"}`	1 or 0

Important notes: `rate()` works on counters and needs a range selector such as `[5m]`; use a range of at least four times the scrape interval; gauges (`node_memory_*`) are used directly; the interface name (`eth0` , `ens33` , `enp0s3`) must match `ip -br link` ; `100 - idle` counts iowait as busy time, so a disk-bound host also shows a high CPU figure; the `node_` prefix means Node Exporter metrics.

I verified the CPU and memory expressions with `promtool test rules` using these input series (one CPU whose idle and user counters each grow 30 seconds per minute, memory available 1 GB of 8 GB). Expected: CPU 50, memory available 12.5.

```
mkdir -p ~/lab8 && cd ~/lab8
cat > test.yml <<'EOF'
rule_files: [alert.rules.yml]
evaluation_interval: 1m
tests:
  - interval: 1m
    input_series:
      - series: 'node_cpu_seconds_total{instance="a",cpu="0",mode="idle"}'
        values: '0+30x20'
      - series: 'node_cpu_seconds_total{instance="a",cpu="0",mode="user"}'
        values: '0+30x20'
      - series: 'node_memory_MemAvailable_bytes{instance="a"}'
```

```
        values: '1000000000x20'
      - series: 'node_memory_MemTotal_bytes{instance="a"}'
        values: '8000000000x20'
    promql_expr_test:
      - expr: 100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
        eval_time: 10m
        exp_samples:
          - labels: '{instance="a"}'
            value: 50
      - expr: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes *
100
        eval_time: 10m
        exp_samples:
          - labels: '{instance="a"}'
            value: 12.5
EOF
# needs alert.rules.yml from Task 4 in the same folder:  promtool test rules
test.yml   -> SUCCESS
```

Compare with the command line: `top` / `vmstat` (Lab 1) should agree with the CPU query within a few percent, and `free -b` "available" must equal `node_memory_MemAvailable_bytes`.

Task 3: analyze supplied metrics and identify the bottleneck

The supplied data may be a Prometheus graph, a table of query results or two `/metrics` captures. Reasoning table:

Evidence	Bottleneck
CPU above about 80% with iowait low, load above the core count	CPU
iowait above 20%, disk busy time near 100%, CPU user low	Disk I/O
`MemAvailable` under 10% and swap in use, with disk activity rising	Memory (and then swapping)
Receive or transmit near interface speed (`node_network_speed_bytes`)	Network
Everything normal but `up == 0`	Availability, not capacity

Capture two scrapes yourself and let the script do the arithmetic. Save `promdiag.py`: it reads two `/metrics` files and the seconds between them, calculates CPU, iowait, memory, swap, load, network rates and disk busy time, and names the bottleneck.

```
#!/usr/bin/env python3
"""Lab 8: find the bottleneck from two node_exporter scrapes.
Usage: python3 promdiag.py BEFORE.prom AFTER.prom SECONDS   (prompts if
omitted)
Capture with:  curl -s localhost:9100/metrics > a.prom ; sleep 30 ; curl -s
localhost:9100/metrics > b.prom"""
import re, sys
from collections import defaultdict
a = sys.argv[1:]
f1 = a[0] if len(a) > 0 else input("Scrape 1 file: ").strip()
f2 = a[1] if len(a) > 1 else input("Scrape 2 file: ").strip()
try:
    dt = float(a[2] if len(a) > 2 else input("Seconds between scrapes: "))
    assert dt > 0
except (ValueError, AssertionError):
    sys.exit("Seconds must be a positive number.")
LINE = re.compile(r'^([a-zA-Z_:][\w:]*)(?:\{([^}]*)\})?\s+(\S+)')
def load(p):
    d = defaultdict(dict)
    try:
        for ln in open(p):
            if ln.startswith("#"): continue
            m = LINE.match(ln)
            if m:
                lab = dict(re.findall(r'(\w+)="([^"]*)"', m[2] or ""))
                try: d[m[1]][tuple(sorted(lab.items()))] = float(m[3])
                except ValueError: pass
    except OSError as e:
        sys.exit(f"Cannot read {p}: {e}")
    return d
A, B = load(f1), load(f2)
def delta(metric, **flt):
    out = 0.0
    for k, v in B.get(metric, {}).items():
        l = dict(k)
        if all(l.get(x) == y for x, y in flt.items()):
            out += v - A.get(metric, {}).get(k, v)
    return out
def val(metric, **flt):
    return sum(v for k, v in B.get(metric, {}).items() if all(dict(k).get(x)
== y for x, y in flt.items()))
cpu_total = delta("node_cpu_seconds_total")
```

```
if cpu_total <= 0: sys.exit("No CPU counters found: is this node_exporter
output?")
idle, iow = delta("node_cpu_seconds_total", mode="idle"),
delta("node_cpu_seconds_total", mode="iowait")
cores = len({dict(k).get("cpu") for k in B["node_cpu_seconds_total"]})
cpu = 100 - idle / cpu_total * 100; iowp = iow / cpu_total * 100
memp = val("node_memory_MemAvailable_bytes") /
val("node_memory_MemTotal_bytes") * 100
swapp = 100 - val("node_memory_SwapFree_bytes") /
val("node_memory_SwapTotal_bytes") * 100 if
val("node_memory_SwapTotal_bytes") else 0
load = val("node_load1")
rx = delta("node_network_receive_bytes_total") * 8 / dt / 1e6; tx =
delta("node_network_transmit_bytes_total") * 8 / dt / 1e6
spd = val("node_network_speed_bytes", device="eth0") * 8 / 1e6 or 1000.0   #
Mbit/s; assume 1 Gbit/s if the gauge is absent
busy = max([delta("node_disk_io_time_seconds_total", device=dict(k)
["device"]) / dt * 100 for k in B.get("node_disk_io_time_seconds_total", {})]
or [0])
print(f"CPU {cpu:.1f}% (iowait {iowp:.1f}%) on {cores} cores | load1 {load:g}
| mem available {memp:.1f}% | swap used {swapp:.1f}% | net rx {rx:.1f} Mbit/s
tx {tx:.1f} Mbit/s | busiest disk {busy:.0f}% busy")
f = []
if iowp >= 20 or busy >= 80: f.append("DISK I/O (high iowait / disk busy
time)")
if cpu >= 80 and iowp < 20: f.append("CPU (busy, not waiting on disk)")
if memp <= 10 or swapp >= 20: f.append("MEMORY (low available memory /
swapping)")
if max(rx, tx) >= 0.8 * spd: f.append(f"NETWORK (busiest direction {max(rx,
tx):.0f} of {spd:.0f} Mbit/s)")
if load > 1.5 * cores and not f: f.append("CPU run-queue (load above 1.5 x
cores)")
print("Likely bottleneck:", "; ".join(f) if f else "none: all metrics within
normal range")
```

Test dataset. This generator writes `base.prom` plus five scenario scrapes (2 CPUs, 30 seconds apart) so you can pre-check the script:

```
cat > gen_prom.py <<'EOF'
def scrape(path, idle, user, iowait, mem, swapfree, load, rx, tx, disk):
    L = []
    for c in (0, 1):
        for mode, v in (("idle", idle), ("user", user), ("iowait", iowait)):
            L.append(f'node_cpu_seconds_total{{cpu="{c}",mode="{mode}"}} {v /
2}')
    L += [f"node_memory_MemAvailable_bytes {mem}",
```

```
"node_memory_MemTotal_bytes 8000000000", "node_memory_SwapTotal_bytes
2000000000",
          f"node_memory_SwapFree_bytes {swapfree}", f"node_load1 {load}",
'node_network_speed_bytes{device="eth0"} 12500000',
          f'node_network_receive_bytes_total{{device="eth0"}} {rx}',
f'node_network_transmit_bytes_total{{device="eth0"}} {tx}',
          f'node_disk_io_time_seconds_total{{device="sda"}} {disk}']
    open(path, "w").write("\n".join(L) + "\n")
B = 1000
scrape("base.prom", B, B, B, 4_000_000_000, 2_000_000_000, 0.5, 0, 0, 0)
for name, d in {"ok":   (50, 8, 2, 4_000_000_000, 2_000_000_000, 0.4,
3_750_000, 1_000_000, 1),
                "cpu":  (6, 54, 0, 4_000_000_000, 2_000_000_000, 3.9,
3_750_000, 1_000_000, 3),
                "disk": (10, 15, 35, 4_000_000_000, 2_000_000_000, 2.5,
3_750_000, 1_000_000, 28),
                "mem":  (40, 12, 8, 400_000_000, 400_000_000, 1.2, 3_750_000,
1_000_000, 6),
                "net":  (52, 6, 2, 4_000_000_000, 2_000_000_000, 0.4,
330_000_000, 2_000_000, 1)}.items():
    scrape(f"{name}.prom", B + d[0], B + d[1], B + d[2], *d[3:])
EOF
python3 gen_prom.py
for s in ok cpu disk mem net; do echo "== $s"; python3 promdiag.py base.prom
$s.prom 30; done
```

Scenario	Expected output
ok	CPU 16.7% (iowait 3.3%), mem 50%, swap 0%, net 1.0 / 0.3 Mbit/s, disk 3% busy; bottleneck none
cpu	CPU 90.0% (iowait 0.0%), load 3.9, disk 10%; bottleneck CPU
disk	CPU 83.3% (iowait 58.3%), disk 93% busy; bottleneck DISK I/O (the high CPU figure includes iowait, a viva trap)
mem	mem available 5.0%, swap used 80.0%; bottleneck MEMORY
net	net rx 88.0 Mbit/s of a 100 Mbit/s link; bottleneck NETWORK
missing file or seconds 0	`Cannot read ...` / `Seconds must be a positive number.`

On the real server, capture and analyze: `curl -s localhost:9100/metrics > a.prom; sleep 30; curl -s localhost:9100/metrics > b.prom; python3 promdiag.py a.prom b.prom 30`, and create a load with `stress-ng --cpu 2 --timeout 60s` during the 30 seconds to see the CPU case.

Task 4: Grafana panel with a threshold or alert rule

Panel: log in to Grafana (`http://localhost:3000`, default `admin` / `admin`, it asks for a new password at first login). Then:

1. **Dashboards > New > New dashboard > Add visualization**, choose the **Prometheus** data source.
2. In the **Queries** tab (Code mode), paste `100 - (avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`.
3. Visualization: **Time series** (or **Gauge**). Panel options: **Title** `CPU utilization %`, **Standard options > Unit** `Percent (0-100)`, **Min** 0, **Max** 100.
4. **Thresholds** (right pane): base `green`, add `80` as `orange` / `red`; **Show thresholds** `As lines (dashed)` so the limit is visible on the graph. For a gauge, thresholds colour the dial.
5. Click **Apply**, then **Save dashboard** (the disk icon) with a name such as `Node Lab 8`.

Faster option: **Dashboards > New > Import**, enter ID `1860` (Node Exporter Full), choose the Prometheus data source, **Import**; the ready-made dashboard shows CPU, memory, disk and network. Then edit one panel to add a threshold.

Alert rule in Grafana: panel menu **More > New alert rule** (or **Alerting > Alert rules > New alert rule**). Query A is the PromQL above; add a **Reduce** (Last) and a **Threshold** expression `IS ABOVE 80`; **Evaluation**: folder and group `lab8`, every `1m`, **pending period** `2m`; add a label `severity=warning` and a summary annotation. Choose a contact point (**Alerting > Contact points**: email, Slack or a webhook) in **Notifications**. The state goes Normal, Pending, Firing. Test it with `stress-ng --cpu 4 --timeout 300s`.

Alert rule in Prometheus (file validated with `promtool check rules`):

```
groups:
  - name: lab8
    rules:
      - alert: HighCpuUsage
        expr: 100 - (avg by (instance)
(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 2m
        labels:
          severity: warning
        annotations:
```

```
          summary: "CPU above 80% on {{ $labels.instance }}"
      - alert: LowMemory
        expr: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes *
100 < 10
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "Less than 10% memory available on {{ $labels.instance }}"
      - alert: InstanceDown
        expr: up == 0
        for: 1m
        labels:
          severity: critical
```

```
sudo cp alert.rules.yml /etc/prometheus/ && promtool check rules
/etc/prometheus/alert.rules.yml     # 3 rules found
sudo systemctl restart prometheus
# UI: Alerts tab shows inactive, pending, firing. API: curl -s
localhost:9090/api/v1/rules | python3 -m json.tool | head -40
# demo: stress-ng --cpu 4 --timeout 300s   -> HighCpuUsage pending for 2
minutes, then firing
```

Safe test: use a low threshold such as `> 5` for a short test, then restore it, or generate bounded load with `stress-ng ... --timeout`.

Likely examiner modifications

- Different query: disk used %, load average, swap used, per-core CPU (`by (cpu)`), top interface by traffic (`topk(1, rate(node_network_receive_bytes_total[5m]))`), uptime (`time() - node_boot_time_seconds`).
- Network transmit instead of receive, or in Mbit/s or packets per second (`node_network_receive_packets_total`), or errors (`node_network_receive_errs_total`).
- Change the scrape interval or add a second target (`localhost:9101` or another host) in `static_configs`, then show it under **Targets**.
- A different threshold, `for` duration or severity; or a Grafana variable (for example `$instance`) in the query.
- Use a different panel type (stat, bar gauge) or change the time range / refresh rate.
- Explain what happens if `rate()` is used on a gauge or without a range.
- Show the alert firing, and then resolving after the load stops.

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
Target shows `DOWN`, `connection refused`	Node Exporter not running or wrong port	`systemctl status prometheus-node-exporter`; `curl localhost:9100/metrics`	Start it; correct the `targets:` entry
`context deadline exceeded`	Target slow, firewall, or `scrape_timeout` too small	Status > Targets, last error; `curl -m 5 target:9100`	Open 9100; raise `scrape_timeout`
`promtool check config` error `yaml: line N`	Indentation or tab characters in YAML	The line number	Use spaces only, consistent two-space indentation
Changes in `prometheus.yml` are not applied	Not reloaded	`systemctl status prometheus`; Status > Configuration	Restart or reload the service
Query returns `No data` / empty	Wrong metric or label, or the time range before the first scrape	Type `node_` and use the autocomplete; check `up`	Fix the label values; wait two scrape intervals
`rate()` returns nothing	Range shorter than two scrape intervals, or used on a gauge	Range `[5m]` vs interval	Use at least `[1m]` for a 15 s interval; use rate on counters only
Network panel empty	Wrong interface name	`ip -br link`; query `node_network_receive_bytes_total`	Use the right `device` label, or `device!~"lo"`
CPU shows over 100% or odd values	`sum` over CPUs without dividing	Use `avg by (instance)`	Use the formula in Task 2
Grafana `Bad Gateway` / `connection refused` when testing the data source	Wrong URL (`localhost` inside a container) or Prometheus down	`curl localhost:9090/-/healthy` from the Grafana host	Use the correct URL; in Docker use the container or host IP
Grafana login `admin/admin` rejected	The password was changed	Ask for the exam credentials; `grafana-cli admin reset-admin-password NEW`	Reset the password
Grafana alert never fires	Condition not met, pending period, or no contact point	Alert rule state and **State history**	Use a lower threshold; shorten `for`; create a contact point
Alert stuck on `Pending`	The `for` duration has not elapsed	Alerts tab	Wait or shorten `for`
`Error: bind: address already in use` on 9090 or 9100	Another instance running	`sudo ss -tlnp` and look for 9090 or 9100	Stop the duplicate process
Disk full, Prometheus down	TSDB retention too large	`df -h`; `du -sh /var/lib/prometheus`	Reduce `--storage.tsdb.retention.time`; free space

Viva theory

Prometheus. An open-source monitoring and alerting toolkit with a multi-dimensional data model: every time series is identified by a metric name and key-value labels, for example `node_cpu_seconds_total{cpu="0",mode="idle"}`. It scrapes (pulls) metrics

over HTTP, stores them in its local time-series database (TSDB) and answers PromQL queries.

Pull vs push. Pull makes it easy to know whether a target is alive (`up`), keeps control at the server and avoids targets needing server credentials. Short-lived jobs use the **Pushgateway**. Zabbix is also able to pull (passive) or accept pushes (active).

Metric types. Counter only increases (resets at restart): use `rate()` . **Gauge** goes up and down (memory, temperature). **Histogram** counts observations in buckets (latency; use `histogram_quantile`). **Summary** gives quantiles on the client side.

PromQL basics. Instant vector (one value per series now), range vector (`[5m]` samples over time), scalar. Functions: `rate` , `irate` (last two points, spiky), `increase` , `avg_over_time` , `sum by` , `avg by` , `topk` , `histogram_quantile` . Operators: arithmetic, comparison filter (`> 80`), `and` , `or` , `unless` .

Node Exporter. Exposes hardware and OS metrics of a Linux host from `/proc` and `/sys` ; one per host; port 9100.

Alerting flow. Prometheus evaluates rules every `evaluation_interval` ; an alert is Pending until the condition has lasted `for` , then Firing and sent to Alertmanager, which deduplicates, groups, silences and routes to receivers (email, Slack, PagerDuty).

Grafana. A visualization and dashboard platform that queries many data sources (Prometheus, Loki, Elasticsearch, SQL). Panels, variables, annotations, and unified alerting. It does not store the metrics.

Prometheus vs Zabbix. Prometheus: pull model, dimensional labels, PromQL, cloud-native and Kubernetes friendly, bring your own dashboards/alerts, local TSDB. Zabbix: agent-based with a built-in UI, templates, SQL database, strong for traditional infrastructure and SNMP.

Why `for:` in rules. It avoids flapping alerts from brief spikes.

Typical viva questions: Default ports? (Prometheus 9090, Node Exporter 9100, Alertmanager 9093, Grafana 3000.) What does `up` mean? (1 if the last scrape succeeded.) Why `rate()` for CPU? (The counter always grows; rate gives seconds per second.) What does `100 - avg(rate(idle[5m])) * 100` compute? (CPU busy %.) Where are rules defined? (A rules file referenced by `rule_files` .) Difference between `rate` and `irate` ? (Average over the window vs the last two samples.) How do you check config? (`promtool check config` .) What is a label? (A key-value dimension of a series.) What is cardinality? (The number of unique series; high cardinality hurts performance.) What is retention? (How long data is kept; default 15 days.)

Lab 9: SRE golden signals (LETS) monitoring

Scenario: evaluate the reliability of a university examination portal. Marks: Task 1 (2) use curl to measure DNS, TCP, TLS, TTFB and total HTTP response times; Task 2 (3) use supplied request logs or Prometheus metrics to calculate request rate and HTTP error rate; Task 3 (3) analyze latency, errors, traffic and saturation to identify the most likely service degradation; Task 4 (2) recommend an SLI, SLO and alert condition with justification.

Tested here: the curl timing script (against a local web server and a refused port), the log analyzer and the SLO calculator, all with the expected outputs shown. The sandbox blocks outside websites, so the TLS phase of curl was not measured live; its arithmetic is shown below and is straightforward. Prometheus queries use the common metric names `http_requests_total` and `http_request_duration_seconds_bucket`; the supplied data may name them differently, so check with the autocomplete first.

Setup and the four golden signals

The four golden signals (Google SRE book) are remembered as **LETS**:

Signal	Question it answers	Typical measurement
Latency	How long do requests take? Separate successful from failed requests	p50, p95, p99 response time (not the average)
Errors	What fraction of requests fail?	5xx rate, plus wrong content or too-slow responses counted as errors
Traffic	How much demand is on the system?	Requests per second
Saturation	How full is the most constrained resource?	CPU, memory, disk I/O, connection pool or queue depth

```
mkdir -p ~/lab9 && cd ~/lab9
sudo apt update && sudo apt install -y curl jq bc
curl --version | head -1
```

Task 1: measure DNS, TCP, TLS, TTFB and total time with curl

One request with every timer curl exposes (all values are cumulative seconds from the start):

```
curl -s -o /dev/null -w 'dns=%{time_namelookup} tcp_done=%{time_connect}
tls_done=%{time_appconnect} ttfb=%{time_starttransfer} total=%{time_total}
http=%{http_code} ip=%{remote_ip}\n' https://example.com/
```

Because the timers are cumulative, the phase durations are differences:

Phase	Formula	Meaning
DNS	`time_namelookup`	Resolving the name
TCP connect	`time_connect - time_namelookup`	Three-way handshake (about one round trip)
TLS handshake	`time_appconnect - time_connect`	Certificate and key exchange (0 for plain HTTP)
Server wait (TTFB phase)	`time_starttransfer - time_appconnect` (or `- time_connect` for HTTP)	Request sent until the first byte arrives: the application and backend time
Transfer	`time_total - time_starttransfer`	Downloading the body
TTFB as a cumulative figure	`time_starttransfer`	Often quoted as "TTFB"; state which definition you use
Total	`time_total`	Everything

Worked example: dns 0.020, connect 0.045, appconnect 0.130, starttransfer 0.480, total 0.520 gives DNS 20 ms, TCP 25 ms, TLS 85 ms, server wait 350 ms, transfer 40 ms, total 520 ms. The largest slice is server wait, so the slowness is in the application or backend, not the network.

Save `curltime.sh`. It asks for the URL if you give none, repeats N times, prints each phase in milliseconds and the average, and reports failed requests (HTTP 000) clearly.

```
#!/bin/bash
# usage: ./curltime.sh [URL] [COUNT]   -> DNS, TCP, TLS, TTFB, transfer and
total in milliseconds, per request and averaged
URL=${1:-}; N=${2:-5}
[ -z "$URL" ] && read -rp "URL (e.g. https://example.com/): " URL
[[ "$N" =~ ^[0-9]+$ ]] && [ "$N" -ge 1 ] || { echo "COUNT must be a positive
integer"; exit 2; }
command -v curl >/dev/null || { echo "curl not installed: sudo apt install -y
curl"; exit 2; }
printf '%-4s %7s %7s %7s %7s %9s %8s %5s\n' "#" DNS TCP TLS TTFB Transfer
Total HTTP
for i in $(seq 1 "$N"); do
  curl -s -o /dev/null --max-time 15 -w '%{time_namelookup} %{time_connect} %
{time_appconnect} %{time_starttransfer} %{time_total} %{http_code}\n' "$URL"
done | awk '{
  if ($6 == "000") { printf "%-4d request failed (HTTP 000: DNS, connect or
timeout error)\n", NR; next }
  dns=$1*1000; tcp=($2-$1)*1000; tls=($3>0)?($3-$2)*1000:0; pre=($3>0)?$3:$2;
ttfb=($4-pre)*1000; tr=($5-$4)*1000; tot=$5*1000
  printf "%-4d %7.1f %7.1f %7.1f %7.1f %9.1f %8.1f %5s\n", NR, dns, tcp, tls,
ttfb, tr, tot, $6
  s1+=dns; s2+=tcp; s3+=tls; s4+=ttfb; s5+=tr; s6+=tot; n++ }
  END { if (n) printf "%-4s %7.1f %7.1f %7.1f %7.1f %9.1f %8.1f\n", "avg",
s1/n, s2/n, s3/n, s4/n, s5/n, s6/n }'
```

```
chmod +x curltime.sh
./curltime.sh https://portal.example.edu/ 5        # the portal URL from the
task sheet
```

Test it without any real site by running a local server in a second terminal:

```
python3 -m http.server 8099 &                      # then:
./curltime.sh http://127.0.0.1:8099/ 3             # I got rows such as: DNS
0.0, TCP 0.1, TLS 0.0, TTFB 0.6, total 0.8, HTTP 200, plus an avg row
./curltime.sh http://127.0.0.1:1/ 2                # refused port: prints
"request failed (HTTP 000 ...)" for each
./curltime.sh http://127.0.0.1:8099/ x             # prints: COUNT must be a
positive integer
kill %1
```

Useful extras: `curl -v https://host/ 2>&1 | head -40` shows TLS version and certificate; `curl -w '%{http_code}\n' -o /dev/null -s URL` gives the status; `curl -I URL` shows headers; `--resolve host:443:IP` bypasses DNS to test one backend; `-k` ignores certificate errors (diagnosis only); `curl --http1.1` / `--http2` selects the

protocol; `dig +stats host` and `ping` / `mtr` separate DNS and network from server time. Always repeat the measurement several times and report the median or average, because the first request pays DNS and connection costs.

Interpretation: DNS above about 100 ms means a slow resolver; TCP higher than the round-trip time means network loss or distance; TLS above 200 ms points to certificate chain or CPU problems on the server; high server wait is the application or database; a large transfer time means a big payload or low bandwidth.

Task 2: request rate and HTTP error rate from logs or Prometheus

From request logs. Request rate = requests / seconds in the window. Error rate = 5xx / total requests. Shell one-liners (combined log format, status is field 9):

```
LOG=req_sample.log
wc -l < $LOG                                                    # total
requests
awk '$9>=500' $LOG | wc -l                                      # 5xx
awk '{print substr($4,2,17)}' $LOG | sort | uniq -c             #
requests per minute (divide by 60 for req/s)
awk '$9>=500 {print substr($4,2,17)}' $LOG | sort | uniq -c     # 5xx
per minute
awk '{n++; if ($9>=500) e++} END {printf "error rate %.2f%%\n", e/n*100}'
$LOG
```

Save `golden.py`. It expects each log line to end with the request time in seconds (nginx: add `$request_time` to `log_format`), then reports per minute: requests per second, 5xx %, p50/p95/p99 latency and the percentage of "good" requests (status below 500 and latency within the target). It then compares the overall SLI with the SLO and reports error-budget use.

```
#!/usr/bin/env python3
"""Lab 9: request rate, error rate and latency percentiles per minute from an
access log whose LAST field is the
request time in seconds (nginx: log_format ... '$request_time').  Usage:
python3 golden.py [LOG] [--slo-ms 500] [--slo 99.5]"""
import argparse, math, re, sys
from collections import defaultdict
from datetime import datetime
ap = argparse.ArgumentParser(); ap.add_argument("log", nargs="?");
ap.add_argument("--slo-ms", type=float, default=500); ap.add_argument("--
slo", type=float, default=99.5)
a = ap.parse_args()
path = a.log or input("Access log (last field = request time in s):
").strip()
```

```
RX = re.compile(r'\[([^\]]+)\] "[^"]*" (\d{3}) \S+.* ([\d.]+)\s*$')
try:
    lines = open(path, errors="replace").read().splitlines()
except OSError as e:
    sys.exit(f"Cannot read {path}: {e}")
per = defaultdict(list); bad = 0
for ln in lines:
    m = RX.search(ln)
    if not m: bad += 1; continue
    t = datetime.strptime(m[1].split()[0],
"%d/%b/%Y:%H:%M:%S").replace(second=0)
    per[t].append((int(m[2]), float(m[3]) * 1000))
if not per: sys.exit("No parsable lines (need status and a trailing request-
time field).")
def pct(v, p): v = sorted(v); return v[min(len(v) - 1, math.ceil(p / 100 *
len(v)) - 1)]
print(f"{'Minute':<7}{'Req/s':>7}{'5xx%':>7}{'p50ms':>7}{'p95ms':>7}
{'p99ms':>7}{'Good%':>7}")
good_t = tot_t = 0
for t in sorted(per):
    r = per[t]; n = len(r); e = sum(1 for s, _ in r if s >= 500); lat = [l
for _, l in r]
    good = sum(1 for s, l in r if s < 500 and l <= a.slo_ms); good_t += good;
tot_t += n
    print(f"{t:%H:%M}  {n/60:7.2f}{e/n*100:7.1f}{pct(lat,50):7.0f}
{pct(lat,95):7.0f}{pct(lat,99):7.0f}{good/n*100:7.1f}")
sli = good_t / tot_t * 100
print(f"\nOverall SLI (status<500 and latency<={a.slo_ms:g} ms): {sli:.2f}%
vs SLO {a.slo}%  ->  {'MET' if sli >= a.slo else 'VIOLATED'}")
budget = (100 - a.slo) / 100 * tot_t
print(f"Error budget in this sample: {budget:.1f} bad requests allowed; used
{tot_t - good_t} ({(tot_t - good_t) / budget * 100:.0f}% of budget)")
```

Test dataset (6 minutes x 60 requests, with an incident around 11:02 to 11:04):

```
python3 - <<'EOF'
plan = [(0, 0), (0, 2), (3, 15), (9, 30), (2, 12), (0, 1)]      # per minute:
(number of 503s, number of slow requests)
rows = []
for m, (e5, slow) in enumerate(plan):
    for i in range(60):
        st = 503 if i < e5 else 200
        rt = 0.8 + (i % 5) * 0.1 if (e5 <= i < e5 + slow) else 0.05 + (i % 5)
* 0.02
        rows.append(f'10.1.1.{i+1} - - [03/Nov/2026:11:{m:02d}:{i:02d} +0000]
"GET /exam HTTP/1.1" {st} 900 "-" "ua" {rt:.3f}')
open("req_sample.log", "w").write("\n".join(rows) + "\n")
EOF
python3 golden.py req_sample.log
```

Minute	Req/s	5xx %	p50 ms	p95 ms	p99 ms	Good %
11:00	1.00	0.0	90	130	130	100.0
11:01	1.00	0.0	90	130	900	96.7
11:02	1.00	5.0	110	1100	1200	70.0
11:03	1.00	15.0	130	1200	1200	35.0
11:04	1.00	3.3	110	1100	1200	76.7
11:05	1.00	0.0	90	130	800	98.3

Expected summary: overall SLI 79.44% against a 99.5% SLO, VIOLATED; budget in the sample 1.8 bad requests allowed, 74 used. With `--slo-ms 1000 --slo 99` it reports 89.72%, VIOLATED. A wrong file name gives `Cannot read ...`. Overall request rate here is 1.0 req/s and overall 5xx rate is 14 of 360 = 3.89%.

From Prometheus (if the portal exports metrics):

```
Traffic:         sum(rate(http_requests_total[5m]))
Traffic by code: sum by (status) (rate(http_requests_total[5m]))
Error rate:      sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m]))
Error rate %:    100 * sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m]))
Latency p95:     histogram_quantile(0.95, sum by (le)
(rate(http_request_duration_seconds_bucket[5m])))
Latency avg:     rate(http_request_duration_seconds_sum[5m]) /
rate(http_request_duration_seconds_count[5m])
Saturation CPU:  100 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) *
100
Saturation mem:  100 - node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes * 100
```

If the metrics are named differently, type the first letters in the Prometheus UI to see the real names (for example `nginx_http_requests_total`, `http_server_requests_seconds_count`). For a supplied table of Prometheus values, apply the same formulas by hand: error rate = 5xx rate / total rate.

Task 3: analyze the four signals and identify the degradation

Read the signals together; a single signal can mislead.

Pattern	Most likely degradation
Latency up, errors flat, traffic flat, CPU/DB saturation high	Backend slowing down from resource exhaustion (slow queries, saturated database, thread pool full)
Latency up, traffic sharply up, saturation near 100%	Overload from demand (exam start rush): capacity problem, needs scaling or rate limiting
Errors (5xx) up, latency of successes normal, traffic normal	Failing dependency or bad deploy: some instances broken
Errors up and latency up together with saturation high	Overload turning into failure (timeouts become 503/504): the classic saturation cascade
Traffic drops while errors rise	Users cannot reach the service (outage upstream, DNS or network problem); low traffic hides the failure

Pattern	Most likely degradation
Latency up only for the first request or only DNS or TLS phase	Network or DNS or certificate problem, not the application
All normal except the p99	A small slice of requests are slow (one bad node, one slow endpoint, garbage collection)
Slow only at the same time each day	Scheduled job or backup competing for resources

For the test dataset above: traffic is flat at 1 req/s, so the cause is not a demand surge; at 11:02 to 11:04 the 5xx rate peaks at 15% and p95 jumps from 130 ms to 1200 ms together, the signature of a backend degradation (503 means the service was unavailable or overloaded), which recovers by 11:05. Saturation was not given, so the conclusion should say a resource check (CPU, database connections, thread pool) is the next step. Averages hide the problem: p50 only moved from 90 to 130 ms, while p95 grew nine-fold. This is why percentiles matter. Describe also the user impact: during those three minutes only 35% to 76% of requests were good.

Tools to confirm: Lab 1 commands on the server (`vmstat` , `iostat`), Lab 8 PromQL for CPU and memory, `ss -s` for connection counts, and database `SHOW PROCESSLIST` /slow query log.

Task 4: recommend an SLI, SLO and alert condition

Definitions: an **SLI** (indicator) is a measured ratio of good events to valid events. An **SLO** (objective) is the target for that ratio over a window. An **SLA** (agreement) is the contractual promise with consequences. The **error budget** is `100% - SLO` , the unreliability you may spend.

Recommendation for an examination portal (give your own numbers and justify them):

Item	Recommendation	Justification
Availability SLI	Share of requests with status below 500 (excluding health checks)	Directly what students experience: can they load and submit
Latency SLI	Share of requests completed within 500 ms (use 1 s for the submit endpoint)	Slow pages during an exam cause panic and resubmits; a percentage threshold is more robust than the mean

Item	Recommendation	Justification
Combined "good" request	status below 500 and latency within the target	One number that captures both
SLO	99.5% good requests over a rolling 30 days (99.9% during exam windows)	The portal is business-critical only at defined times; 99.5% allows 216 minutes of full outage per 30 days, 99.9% only 43 minutes
Alert (page)	Burn rate above 14.4 over 1 hour **and** over 5 minutes	Spends 2% of the monthly budget in an hour: urgent, and the short window confirms it is still happening
Alert (ticket)	Burn rate above 1 over 3 days	Slow leak that will exhaust the budget before the window ends
Simple alternative	5xx rate above 5% for 5 minutes, or p95 above 1 s for 10 minutes	Easy to explain and test in an exam

PromQL for the simple alerts (this expression form is correct; the metric names may differ):

```
- alert: PortalHighErrorRate
  expr: sum(rate(http_requests_total{status=~"5.."}[5m])) /
sum(rate(http_requests_total[5m])) > 0.05
  for: 5m
  labels: {severity: page}
- alert: PortalSlow
  expr: histogram_quantile(0.95, sum by (le)
(rate(http_request_duration_seconds_bucket[5m]))) > 1
  for: 10m
  labels: {severity: ticket}
```

Save `slo.py` to compute the error budget and burn rate from numbers you are given. Arguments are optional; it asks for anything missing.

```
#!/usr/bin/env python3
"""Lab 9: error-budget and burn-rate calculator.  Usage: python3 slo.py
[SLO_PERCENT] [WINDOW_DAYS] [GOOD] [TOTAL] [ELAPSED_DAYS]"""
import sys
a = sys.argv[1:]
def ask(i, q, d):
    return float(a[i]) if len(a) > i else float(input(f"{q} [{d}]: ") or d)
try:
```

```
    slo, days = ask(0, "SLO %", 99.5), ask(1, "Window days", 30)
    good, total, el = ask(2, "Good events so far", 99000), ask(3, "Total
events so far", 100000), ask(4, "Days elapsed", 10)
    assert 0 < slo < 100 and days > 0 and total > 0 and 0 < el <= days and 0
<= good <= total
except (ValueError, AssertionError):
    sys.exit("Invalid input: need 0<SLO<100, 0<elapsed<=window,
0<=good<=total.")
allowed = 1 - slo / 100
print(f"Error budget: {allowed*100:.3g}% = {allowed*days*24*60:.0f} minutes
of full outage per {days:g} days")
bad = (total - good) / total
burn = bad / allowed
used = burn * el / days * 100
print(f"Current SLI {good/total*100:.3f}% | burn rate {burn:.2f}x | budget
used {used:.0f}% after {el:g} of {days:g} days")
print(f"At this burn rate the whole budget is gone on day {days/burn:.1f} of
the window" if bad else "No errors so far.")
print("Page if burn rate > 14.4 over 1 h AND 5 min (2% of a 30-day budget
gone in 1 h); ticket if > 1 over 3 days.")
```

Command	Expected output
`python3 slo.py 99.9 30 99900 100000 1`	Budget 0.1% = 43 minutes; SLI 99.900%, burn 1.00x, 3% used after 1 day; gone on day 30.0
`python3 slo.py 99.5 30 99000 100000 10`	Budget 0.5% = 216 minutes; burn 2.00x, 67% used after 10 days; gone on day 15.0
`python3 slo.py 99.9 30 98560 100000 0.0416`	Burn 14.40x: the paging threshold (about one hour)
`python3 slo.py 101 30 1 1 1`	`Invalid input` message

How to write the justification: choose SLIs that reflect what users feel, pick an SLO that is achievable and cheaper than perfection (100% is neither possible nor affordable), alert on symptoms (user-visible burn) instead of causes (a CPU spike that nobody notices), use two windows to be both fast and precise, and review the SLO after exam periods. The error budget also guides decisions: if it is spent, freeze risky releases and fix reliability.

Likely examiner modifications

- A different URL or protocol (`http://` , `--http2`), or compare two URLs/backends, or measure from a specific IP with `--resolve` .

- Report the median of 10 runs, the slowest request, or use `-w` with different variables (`size_download`, `speed_download`, `num_redirects`, `time_redirect`).
- Calculate the error rate for one endpoint, one hour or only 4xx; compute requests per second for a specific minute.
- A different SLO (99%, 99.9%, 99.99%) and the allowed downtime: 99% gives 432 minutes per 30 days, 99.9% gives 43.2, 99.99% gives 4.3.
- Convert the alert to a PromQL expression with a different threshold and duration.
- Explain why the average latency is misleading, and compute p95 by hand: sort the values and take the value at rank `ceil(0.95 x n)`.
- Identify which signal to alert on if only one is allowed (errors seen by users, or the burn rate of the combined SLI).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`curl: (6) Could not resolve host`	DNS failure or typo	`dig host`; `cat /etc/resolv.conf`	Correct the name; fix the resolver; use `--resolve` to test
`curl: (7) Failed to connect`	Port closed, firewall, service down	`nc -vz host 443`; `ss -tlnp` on the server	Start the service; open the port
`curl: (28) Operation timed out`	Packet loss, firewall drop, or a very slow backend	`--max-time`, `ping`, `mtr`	Check the network path; raise the timeout to measure
`curl: (35)` or `(60) SSL certificate problem`	TLS version mismatch, self-signed or expired certificate	`curl -v`; `openssl s_client -connect host:443 -servername host`	Fix the certificate; use `-k` only to measure
All timers `0.000000` and HTTP `000`	The request never completed	Remove `-s` to see the error text	Fix the cause above
`time_appconnect` is 0	The URL is HTTP, not HTTPS	The scheme	Expected; TLS phase is zero

Symptom or message	Likely cause	Check	Fix
DNS time near zero on repeats	Cached resolver result	First vs later runs	Report the first run separately, or use a fresh name
Phase times negative in a hand calculation	Subtracted in the wrong order	Timers are cumulative	`connect - namelookup`, not the reverse
`golden.py` says `No parsable lines`	Log lacks the trailing request time or the format differs	`tail -2 log`	Add `$request_time` to the log format; adapt the regex
Percentiles look wrong with few samples	p99 of 20 requests is just the maximum	Number of requests per bucket	Use bigger buckets or say the sample is small
PromQL returns `NaN`	Division by zero when no traffic	Query the denominator	Use `or vector(0)`, or add `> 0` guards; note low traffic
`histogram_quantile` returns nothing	Metric is not a histogram, missing `le` label, or `by (le)` omitted	`..._bucket` series exist?	Use `sum by (le) (rate(..._bucket[5m]))`
The error rate looks low but users complain	Failures counted as 200, or timeouts never logged	Look at 499/504 and client-side errors	Count timeouts and slow responses as bad events
`bc: command not found`	Not installed	`which bc`	`sudo apt install -y bc`

Viva theory

Site Reliability Engineering (SRE). Applying software engineering to operations: automate, measure reliability, set explicit targets (SLOs) and use error budgets to balance feature speed against stability. Origin: Google.

Golden signals (LETS). Latency, Errors, Traffic, Saturation: the minimum set that describes the health of a user-facing service. The related **RED** method (Rate, Errors, Duration) suits request-driven services; **USE** (Utilization, Saturation, Errors) suits resources such as CPU and disks.

Percentiles. p50 is the median, p95 means 95% of requests were faster, p99 reveals the slow tail. Averages hide outliers; always graph percentiles. Latency of failed requests should be tracked separately because fast errors can make averages look good.

Saturation. How close the constrained resource is to its limit; often predicts future latency and errors (queues build before failures). Examples: CPU run queue, memory pressure, connection pool use, disk queue.

SLI, SLO, SLA, error budget. SLI is the measurement, SLO the internal target, SLA the external contract with penalties, error budget the allowed unreliability. Burn rate is how fast the budget is consumed relative to the allowed rate (1x spends exactly the budget over the window).

Availability numbers. 99% = 7.2 h downtime per 30 days; 99.9% = 43.2 min; 99.99% = 4.3 min. Each extra nine costs far more.

HTTP timing phases. DNS lookup, TCP three-way handshake, TLS handshake (adds one to two round trips), request, server processing (TTFB), content transfer. TTFB measures backend responsiveness plus network latency.

Alerting philosophy. Alert on symptoms that need a human now; avoid alerts that are noisy or have no action. Use thresholds with duration, multi-window burn-rate alerts, and runbooks.

Typical viva questions: What are the four golden signals? (Latency, errors, traffic, saturation.) Why use p95 rather than the average? (The tail shows what the unluckiest users feel; averages hide it.) What does `time_starttransfer` measure? (Time until the first byte, cumulative.) How do you compute the TLS time? (`time_appconnect - time_connect`.) What is an error budget? (100% minus the SLO.) What is the allowed downtime at 99.9% in 30 days? (About 43 minutes.) Why do we not set a 100% SLO? (Impossible and costly, and it blocks change.) Which HTTP codes count as errors? (5xx always; 4xx usually not, unless they show a server problem such as 429 or wrong redirects.) What is burn rate? (Speed of budget consumption versus the allowed rate.) How would you detect saturation? (CPU run queue, memory pressure, queue length, connection pool use.)

Lab 10: Container and Kubernetes monitoring

Scenario: monitor a containerized application running in a prepared Minikube environment. Marks: Task 1 (2) use kubectl to identify the status of nodes, pods and deployments; Task 2 (3) retrieve pod logs and resource utilization using available Kubernetes monitoring components; Task 3 (3) analyze a supplied pod failure, restart event or resource-pressure condition; Task 4 (2) recommend and verify a corrective action using an approved configuration or recovery procedure.

Honesty note: no Kubernetes cluster can run in my sandbox, so the kubectl commands and the practice manifests below follow the Kubernetes documentation and have not been executed here. The pod-diagnosis script in Task 3 reads `kubectl get pods -o json` output and was run and tested on a sample file that has the same structure.

Setup (Minikube)

```
minikube status                       # host: Running, kubelet: Running,
apiserver: Running, kubeconfig: Configured
minikube start --driver=docker        # only if stopped (needs Docker and the
user in the docker group: sudo usermod -aG docker $USER, then log in again)
kubectl version --client              # kubectl installed (else: sudo snap
install kubectl --classic)
kubectl config current-context        # should print minikube
minikube addons list | grep -E 'metrics-server|dashboard'
minikube addons enable metrics-server # needed for kubectl top (wait about a
minute)
alias k=kubectl                       # shorter typing
```

Kubernetes in one picture: a **node** runs **pods**; a pod holds one or more **containers**; a **Deployment** keeps a desired number of identical pods through a **ReplicaSet**; a **Service** gives them a stable address; the **kubelet** on each node runs the pods and reports status; the **API server** is what kubectl talks to; **metrics-server** supplies live CPU and memory for `kubectl top`.

Task 1: status of nodes, pods and deployments

```
kubectl get nodes -o wide                          # STATUS must be Ready;
shows version, internal IP, OS, runtime
kubectl get pods -A                                # all namespaces; READY x/y,
STATUS, RESTARTS, AGE
kubectl get pods -n NAMESPACE -o wide              # also shows the node and
pod IP
kubectl get deployments -A                         # READY n/m, UP-TO-DATE,
AVAILABLE
kubectl get all -n NAMESPACE                       # pods, services,
deployments, replicasets together
kubectl get ns                                     # namespaces (default, kube-
system, ...)
kubectl get events -A --sort-by=.lastTimestamp | tail -20    # recent
warnings and failures
kubectl describe node minikube | sed -n '/Conditions/,/Addresses/p;/Allocated
resources/,/Events/p'
kubectl cluster-info                               # API server and CoreDNS
addresses
```

Reading the columns: **READY 1/1** means all containers in the pod passed their readiness checks (`0/1` is not serving traffic). **STATUS**: `Running` , `Pending` (not scheduled or still pulling), `Succeeded` , `Failed` , or the reasons `CrashLoopBackOff` , `ImagePullBackOff` , `ErrImagePull` , `ContainerCreating` , `Error` , `OOMKilled` , `Terminating` . **RESTARTS** counts container restarts. For deployments, `READY 2/3` means two of three desired replicas are available.

Useful filters: `kubectl get pods --field-selector=status.phase!=Running -A` (only unhealthy), `kubectl get pods -l app=NAME` (by label), `kubectl get pod POD -o yaml` (full detail), `kubectl get pods -A -o json` (for scripts), `kubectl api-resources` (what object kinds exist).

What to report: node name and Ready status; every pod with status and restarts; each deployment with desired and available replicas; and any abnormal object.

Task 2: pod logs and resource utilization

```
kubectl logs POD -n NAMESPACE                    # current container logs
kubectl logs POD -n NAMESPACE --tail=50          # last 50 lines
kubectl logs POD -n NAMESPACE -f                 # follow live (Ctrl+C to
stop)
kubectl logs POD -n NAMESPACE --previous         # the PREVIOUS crashed
container (essential for CrashLoopBackOff)
kubectl logs POD -c CONTAINER -n NAMESPACE       # a specific container in a
multi-container pod
kubectl logs deployment/NAME -n NAMESPACE        # logs of one pod of a
deployment
kubectl logs -l app=NAME --all-containers --tail=20   # by label
kubectl logs POD --since=10m                     # only the last 10 minutes

kubectl top nodes                                # CPU(cores), CPU%,
MEMORY(bytes), MEMORY%  (needs metrics-server)
kubectl top pods -A --sort-by=memory             # heaviest pods first (also
--sort-by=cpu)
kubectl top pod POD --containers                 # per container
kubectl describe pod POD -n NAMESPACE            # requests, limits, state,
last state, conditions, Events
kubectl get pod POD -o jsonpath='{.spec.containers[*].resources}'   #
configured requests and limits
```

Interpretation: CPU is shown in millicores (`250m` = 0.25 core) and memory in `Mi` . Compare **usage** (`kubectl top`) with **requests** and **limits** (from `describe`). Usage near the memory limit means an OOM kill is imminent; CPU usage at the limit means throttling (the pod is slowed, not killed). `kubectl top` shows `error: Metrics API not available` until metrics-server is enabled and ready.

Other monitoring components that may be mentioned: **kubernetes dashboard** (`minikube dashboard --url`), **kube-state-metrics** (object states as Prometheus metrics), **cAdvisor** (container metrics inside kubelet), **Prometheus and Grafana** (Lab 8, installed with Helm: `helm install kps prometheus-community/kube-prometheus-stack`), `kubectl get --raw /metrics` (API server metrics) and `minikube ssh` followed by `docker ps` / `top` for node-level inspection.

Task 3: analyze a pod failure, restart event or resource pressure

The investigation recipe never changes:

```
kubectl get pod POD -n NS                                    # 1 status and
restarts
kubectl describe pod POD -n NS | sed -n '/State:/,/Ready:/p;/Events:/,$p'   #
2 reason, exit code, last state, scheduling events
kubectl logs POD -n NS --previous                            # 3 why it died
kubectl get events -n NS --sort-by=.lastTimestamp            # 4 cluster's view
kubectl top pod POD -n NS                                    # 5 resource use
```

Symptom (STATUS)	Where the cause shows	Typical causes	Fix
`CrashLoopBackOff`	`logs --previous`, `Last State: Terminated, Exit Code`	Application exits at start: bad command, missing config or env, failed dependency, wrong port	Correct the command or configuration, redeploy
`ImagePullBackOff` / `ErrImagePull`	`describe` Events: `Failed to pull image ... not found` or `unauthorized`	Misspelled image or tag, private registry without credentials, no network	`kubectl set image` with the correct tag; add an `imagePullSecret`
`OOMKilled`, exit code 137	`Last State: Terminated, Reason: OOMKilled`	Memory use exceeded `limits.memory`	Raise the limit, reduce memory use, fix leaks
`Pending` with `FailedScheduling`	`describe` Events: `Insufficient cpu`, `Insufficient memory`, `taints`	Requests larger than free node capacity, node selector or taint mismatch	Lower requests, add node resources (`minikube start --cpus --memory`), fix the selector
`CreateContainerConfigError`	Events mention a missing ConfigMap or Secret	Referenced object absent or key wrong	Create it or fix the reference
`Running` but `0/1` ready	`describe`: `Readiness probe failed`	Wrong probe path or port, slow start	Fix the probe or increase `initialDelaySeconds`
Restarts increase but pod is Running	`Last State` and exit code	Liveness probe failing or OOM	Check the probe, memory and logs
Evicted	`Reason: Evicted` (node pressure)	Node ran out of memory or disk	Free resources, add limits, scale the node
CPU throttling	`top` equals the CPU limit, slow responses	CPU limit too low	Raise `limits.cpu`

Exit codes: 0 normal, 1 application error, 137 killed by SIGKILL (OOM or `kubectl delete` timeout), 139 segmentation fault, 143 SIGTERM.

Save `podcheck.py`. It reads the JSON from `kubectl get pods -A -o json` (from a pipe or a file), lists each pod's phase and restarts, and states the finding and the next step.

```
#!/usr/bin/env python3
"""Lab 10: diagnose pods from `kubectl get pods -A -o json`.
Usage: kubectl get pods -A -o json | python3 podcheck.py      or      python3
podcheck.py pods.json"""
import json, sys
try:
    raw = open(sys.argv[1]).read() if len(sys.argv) > 1 else sys.stdin.read()
    items = json.loads(raw)["items"]
except (OSError, ValueError, KeyError) as e:
    sys.exit(f"Could not read pod JSON ({e}). Use: kubectl get pods -A -o
json | python3 podcheck.py")
FIX = {
    "CrashLoopBackOff": "App exits repeatedly: kubectl logs POD --previous ;
kubectl describe pod POD ; fix command/config/env",
    "ImagePullBackOff": "Cannot pull image: check name/tag/registry
credentials ; kubectl set image deployment/NAME CONTAINER=GOOD_IMAGE",
    "ErrImagePull": "Cannot pull image: check name/tag/registry credentials ;
kubectl set image deployment/NAME CONTAINER=GOOD_IMAGE",
    "OOMKilled": "Container exceeded its memory limit: raise limits.memory
(kubectl set resources) or fix the leak",
    "CreateContainerConfigError": "Bad ConfigMap/Secret reference: kubectl
describe pod POD ; create the missing object",
    "Unschedulable": "No node fits: requests too large or taints: kubectl
describe pod POD (Events) ; lower requests / add capacity",
    "Error": "Container terminated with an error: kubectl logs POD",
}
bad = 0
print(f"{'NAMESPACE':<12}{'POD':<26}{'PHASE':<10}{'RESTARTS':>8}  FINDING")
for p in items:
    ns, name = p["metadata"]["namespace"], p["metadata"]["name"]; st =
p.get("status", {}); phase = st.get("phase", "?")
    finds, restarts = set(), 0
    for c in st.get("containerStatuses", []):
        restarts += c.get("restartCount", 0)
        w = c.get("state", {}).get("waiting"); lt = c.get("lastState",
{}).get("terminated")
        if w and w.get("reason") not in (None, "ContainerCreating",
"PodInitializing"): finds.add(w["reason"])
        if lt and lt.get("reason") in ("OOMKilled", "Error"):
finds.add(lt["reason"])
    for cd in st.get("conditions", []):
        if cd.get("type") == "PodScheduled" and cd.get("status") == "False":
finds.add("Unschedulable")
    if phase == "Failed" and not finds: finds.add("Error")
    if restarts >= 3 and not finds: finds.add("CrashLoopBackOff")
    if phase == "Running" and not finds and not all(c.get("ready") for c in
```

```
st.get("containerStatuses", [])): finds.add("NotReady")
    if finds: bad += 1
    print(f"{ns:<12}{name[:25]:<26}{phase:<10}{restarts:>8}  {',
'.join(sorted(finds)) or 'healthy'}")
    for f in sorted(finds):
        print(f"{'':<12}-> {FIX.get(f, 'Readiness probe failing: kubectl
describe pod POD ; check probe path/port')}")
print(f"\n{len(items)} pods, {bad} unhealthy.")
```

Test dataset (builds a sample with the same JSON structure kubectl produces, then runs the script):

```
python3 - <<'EOF'
import json
def pod(n, phase, cs=None, cond=None, ns="default"):
    return {"metadata":{"namespace":ns,"name":n},"status":
{"phase":phase,"containerStatuses":cs or [],"conditions":cond or []}}
ok=[{"name":"c","ready":True,"restartCount":0,"state":{"running":{}}}]
crash=[{"name":"c","ready":False,"restartCount":7,"state":{"waiting":
{"reason":"CrashLoopBackOff"}},"lastState":{"terminated":
{"reason":"Error","exitCode":1}}}]
img=[{"name":"c","ready":False,"restartCount":0,"state":{"waiting":
{"reason":"ImagePullBackOff"}}}]
oom=[{"name":"c","ready":True,"restartCount":4,"state":{"running":
{}},"lastState":{"terminated":{"reason":"OOMKilled","exitCode":137}}}]
items=[pod("web-6d5f-abc","Running",ok),pod("crash-7c8-
xyz","Running",crash),pod("badimg-5b-q1","Pending",img),pod("memhog-9f-
m2","Running",oom),
 pod("big-0","Pending",[],
[{"type":"PodScheduled","status":"False","reason":"Unschedulable"}]),pod("cor
edns-1","Running",ok,ns="kube-system")]
json.dump({"items":items},open("pods_sample.json","w"))
EOF
python3 podcheck.py pods_sample.json
```

Pod	Expected finding
web-6d5f-abc	healthy
crash-7c8-xyz (7 restarts)	`CrashLoopBackOff, Error` with the logs `--previous` advice
badimg-5b-q1	`ImagePullBackOff` with the `kubectl set image` advice
memhog-9f-m2 (4 restarts)	`OOMKilled` with the raise-memory-limit advice

Pod	Expected finding
big-0	`Unschedulable`
coredns-1 (kube-system)	healthy
Summary line	`6 pods, 4 unhealthy.`
Empty or wrong input	`Could not read pod JSON (...)`

If no cluster is available to produce the failure, create practice failures in a throwaway namespace (the three manifests below are standard patterns; I did not run them):

```
kubectl create ns lab10
cat <<'EOF' | kubectl apply -n lab10 -f -
apiVersion: apps/v1
kind: Deployment
metadata: {name: crashy}
spec:
  replicas: 1
  selector: {matchLabels: {app: crashy}}
  template:
    metadata: {labels: {app: crashy}}
    spec:
      containers:
      - name: c
        image: busybox:1.36
        command: ["sh", "-c", "echo starting; echo fatal config error >&2;
exit 1"]
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: badimage}
spec:
  replicas: 1
  selector: {matchLabels: {app: badimage}}
  template:
    metadata: {labels: {app: badimage}}
    spec:
      containers:
      - name: c
        image: nginx:no-such-tag
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: memhog}
spec:
```

```
  replicas: 1
  selector: {matchLabels: {app: memhog}}
  template:
    metadata: {labels: {app: memhog}}
    spec:
      containers:
      - name: c
        image: polinux/stress
        command: ["stress"]
        args: ["--vm", "1", "--vm-bytes", "250M", "--vm-hang", "1"]
        resources:
          limits: {memory: "100Mi"}
EOF
kubectl get pods -n lab10 -w        # Ctrl+C after a minute
```

Example wording for the analysis: "Pod `memhog` restarts repeatedly with Last State Terminated, Reason OOMKilled, exit code 137. Its memory limit is 100Mi but the process tries to allocate 250M, so the kernel's cgroup kills it. The cause is a memory limit below real demand (or a leak), not a node shortage, since `kubectl top nodes` shows free memory."

Task 4: corrective action and verification

Use an approved action only (the question says "approved configuration or recovery procedure"): change the faulty setting, apply it declaratively or with `kubectl set`, then verify. Never delete the whole namespace or the cluster as a fix.

Failure	Corrective action (commands)
Bad image tag	`kubectl set image deployment/badimage c=nginx:1.25 -n lab10`
Crash from wrong command or config	`kubectl edit deployment/crashy -n lab10` (fix `command`/env), or `kubectl set env deployment/crashy KEY=value`, or `kubectl apply -f fixed.yaml`
OOMKilled	`kubectl set resources deployment/memhog -n lab10 --limits=memory=512Mi --requests=memory=256Mi`
Bad rollout (a new version broke it)	`kubectl rollout undo deployment/NAME -n NS` (rolls back to the previous revision); `kubectl rollout history deployment/NAME`

Failure	Corrective action (commands)
Stuck or hung pod	`kubectl rollout restart deployment/NAME -n NS`, or `kubectl delete pod POD -n NS` (the ReplicaSet recreates it)
Not enough capacity	`kubectl scale deployment/NAME --replicas=3`, or `minikube stop; minikube start --cpus=4 --memory=4096`
Missing ConfigMap or Secret	`kubectl create configmap NAME --from-literal=KEY=value -n NS` then restart the deployment

Verification (always show these):

```
kubectl rollout status deployment/NAME -n NS           # "deployment ...
successfully rolled out"
kubectl get pods -n NS                                 # new pod Running,
READY 1/1, restart count not increasing
kubectl describe pod NEWPOD -n NS | tail -15           # no new Warning
events
kubectl logs NEWPOD -n NS --tail=20                    # healthy application
output
kubectl top pod NEWPOD -n NS                           # usage below the new
limit
kubectl get deployment NAME -n NS                      # READY n/n,
AVAILABLE n
```

Clean up practice objects: `kubectl delete ns lab10`. Explain the choice: raising a limit without knowing demand only moves the problem, so check `kubectl top` over time and pick a limit with headroom above the observed peak; roll back first when a recent release caused the incident, because it restores service fastest.

Likely examiner modifications

- Scale a deployment up or down and watch the pods (`kubectl scale ... --replicas=N`, `kubectl get pods -w`).
- Show logs from a specific container, a specific time window (`--since=5m`) or the previous container.
- Show node capacity versus allocated resources (`kubectl describe node minikube`).
- Expose the application and test it: `kubectl expose deployment NAME --port=80 --type=NodePort`, `minikube service NAME -n NS --url`, then `curl` the URL.

- Use labels and selectors, filter pods by namespace, or output custom columns: `kubectl get pods -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,RESTARTS:.status.containerStatuses[0].restartCount`.
- Delete a pod and show that the Deployment recreates it (self-healing).
- Describe what a liveness and a readiness probe do and add one.
- Set resource requests and limits in the YAML and explain QoS classes (Guaranteed, Burstable, BestEffort).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`The connection to the server localhost:8080 was refused`	kubectl has no kubeconfig context, or Minikube is stopped	`kubectl config current-context`; `minikube status`	`minikube start`; `minikube update-context`
`Unable to connect to the server: dial tcp ...: i/o timeout`	Minikube VM or container stopped, or the IP changed	`minikube status`; `docker ps`	`minikube start`; `minikube update-context`
`permission denied ... /var/run/docker.sock`	The user is not in the docker group	`groups`	`sudo usermod -aG docker $USER`, log out and in (or `newgrp docker`)
`minikube start` fails with `RSRC_INSUFFICIENT_...`	Too little CPU or RAM for the VM	The error text	Close programs; `minikube start --cpus=2 --memory=2048`
`error: Metrics API not available`	metrics-server missing or not ready	`minikube addons list`; `kubectl get pods -n kube-system`	`minikube addons enable metrics-server`; wait one to two minutes
`Error from server (NotFound): pods "x" not found`	Wrong name or namespace	`kubectl get pods -A`	Add `-n NAMESPACE`; copy the exact name

Symptom or message	Likely cause	Check	Fix
`Error from server (BadRequest): container ... is waiting to start`	Logs requested while the container has not started	`kubectl describe pod`	Wait or read `--previous`
Logs of a crashed pod are empty	Looking at the new container		Use `--previous`
`ImagePullBackOff` for a local image	Image exists only in the host Docker, not Minikube's	`minikube image ls`	`minikube image load IMAGE` or `eval $(minikube docker-env)` then build; set `imagePullPolicy: IfNotPresent`
YAML `error validating data` / `mapping values are not allowed`	Indentation or tab characters	The line in the message	Use spaces only; validate with `kubectl apply --dry-run=client -f file.yaml`
`deployment ... exceeded its progress deadline`	The rollout cannot become ready	`kubectl rollout status`; `describe`	Fix the cause; `kubectl rollout undo`
Pod stays `Terminating`	Finalizer or node problem	`kubectl describe pod`	`kubectl delete pod POD --grace-period=0 --force` as a last resort
Nodes `NotReady`	kubelet or container runtime down	`kubectl describe node`; `minikube logs`	`minikube stop` then `minikube start`; recreate with `minikube delete` only if allowed
NodePort service unreachable	Wrong URL or IP	`minikube service NAME --url`; `minikube ip`	Use the URL minikube prints; with the docker driver use `minikube service` or `minikube tunnel`

Viva theory

Container vs VM. A container packages an application with its dependencies and shares the host kernel (isolation by namespaces and cgroups); it starts in seconds and is light. A VM virtualizes hardware and runs a full guest OS.

Kubernetes. An orchestrator that schedules containers across nodes, keeps the desired state (self-healing, scaling, rolling updates), and provides service discovery and load balancing. Control plane: **kube-apiserver**, **etcd** (state store), **kube-scheduler**, **kube-controller-manager**. Node: **kubelet**, **kube-proxy**, container runtime (containerd, Docker).

Objects. Pod (smallest unit), ReplicaSet, Deployment (declarative updates, rollback), StatefulSet, DaemonSet (one pod per node), Job and CronJob, Service (ClusterIP, NodePort, LoadBalancer), Ingress, ConfigMap, Secret, Namespace, PersistentVolume and Claim.

Minikube. A tool that runs a single-node local Kubernetes cluster in a VM or Docker container for learning and development.

Requests vs limits. The request is the amount the scheduler reserves for the pod; the limit is the cap enforced at run time. Exceeding the CPU limit causes throttling; exceeding the memory limit causes an OOM kill. QoS classes: Guaranteed (requests equal limits for all), Burstable, BestEffort (none set; first to be evicted).

Probes. Liveness: should the container be restarted? Readiness: should it receive traffic? Startup: wait for slow starters.

CrashLoopBackOff. The container keeps failing, and the kubelet restarts it with an exponentially growing delay (10 s, 20 s, 40 s up to 5 minutes). It is a symptom, not a cause: logs reveal the cause.

Monitoring layers in Kubernetes. Node and pod resource metrics (metrics-server, cAdvisor), object state (kube-state-metrics), application metrics (Prometheus endpoints), logs (Loki/ELK, `kubectl logs`), events, and traces.

Typical viva questions: Smallest deployable unit? (A pod.) Command to see why a pod is failing? (`kubectl describe pod`, then `kubectl logs --previous`.) What does exit code 137 mean? (The process was killed with SIGKILL, commonly OOM.) What is the difference between a Deployment and a Pod? (A Deployment manages replicas and updates of pods.) How do you roll back? (`kubectl rollout undo`.) What does `kubectl top` need? (metrics-server.) What is a namespace? (A logical partition of cluster resources.) How are pods recreated when deleted? (The ReplicaSet controller restores the desired replica count.) What is etcd? (The key-value store holding cluster state.) Difference between liveness and readiness? (Restart vs receive traffic.)

Lab 11: Industrial and physical infrastructure monitoring

Scenario: monitor a simulated industrial gateway or data-center environmental monitoring system. Marks: Task 1 (2) verify connectivity to a prepared Modbus TCP simulator, OPC UA server or SNMP-enabled environmental sensor; Task 2 (3) retrieve temperature, humidity, power or simulated process measurements; Task 3 (3) analyze supplied measurements to identify abnormal operating conditions; Task 4 (2) configure or demonstrate a threshold alert and recommend an operational response.

The exam gives you **one of three** protocols; read the part for yours and skim the others for the viva. I built and tested a simulator and a reader for each: a pure-Python Modbus TCP simulator and client, an OPC UA server and client (`asyncua` 2.1), and a real `snmpd` configured as a fake sensor read with `snmpget`. Everything below was run, and the outputs shown are the real ones. In the exam, the simulator is supplied, so my simulators are only for practice. The exam's register map will differ from mine (addresses, scaling, units), so always read the supplied documentation first.

Setup and quick comparison

```
mkdir -p ~/lab11 && cd ~/lab11
sudo apt update && sudo apt install -y python3-pip netcat-openbsd snmp
pip3 install --user asyncua            # OPC UA (only if your task is OPC UA)
pip3 install --user pymodbus           # optional Modbus library; the raw-
socket script below needs nothing
```

	Modbus TCP	OPC UA	SNMP
Typical use	PLCs, meters, sensors, simple and old	Modern industrial interoperability, gateways, SCADA	Network and facility devices (UPS, PDU, CRAC, environmental probes)
Port	502/tcp (5020 or others in labs)	4840/tcp (`opc.tcp://host:4840`)	161/udp
Data model	Flat tables of 16-bit registers and bits, no names or types	Browsable address space of typed, named nodes	MIB/OID tree
Security	None built in	Certificates, signing, encryption, users	v3 authentication and encryption
Access style	Poll registers by address	Read, subscribe (events), browse, call methods	Get, walk, traps

Modbus data tables: **coils** (read/write bits, function 1, 5, 15), **discrete inputs** (read-only bits, function 2), **input registers** (read-only 16-bit, function 4), **holding registers** (read/write 16-bit, function 3, 6, 16). Values are scaled integers (temperature 225 means 22.5 C), and 32-bit or floating numbers span two registers. Many documents use 1-based "4xxxx" addresses (40001 equals holding register 0); the protocol itself is 0-based.

Task 1: verify connectivity

Modbus TCP.

```
nc -vz 127.0.0.1 5020                       # TCP port open? (use the host and
port from the task sheet; default 502)
ping -c 2 HOST                              # basic reachability
sudo ss -tlnp | grep -E '502|5020'          # on the simulator host, is it
listening?
python3 modread.py HOST PORT UNIT           # an actual Modbus read proves the
protocol works (script below)
```

If the port is open but the read fails or times out, the **unit id** (slave id) or the register range is wrong. A device that answers with an exception code is alive but you asked the wrong thing.

OPC UA.

```
nc -vz HOST 4840
python3 opcread.py opc.tcp://HOST:4840/PATH        # connects, browses, reads
(script below)
# GUI alternatives: UaExpert (Windows/Linux) or  pip install opcua-client
then  opcua-client
```

SNMP sensor.

```
snmpget -v2c -c public HOST 1.3.6.1.2.1.1.1.0      # sysDescr proves SNMP
works
snmpget -v2c -c public HOST 1.3.6.1.2.1.1.3.0      # sysUpTime
snmpwalk -v2c -c public HOST 1.3.6.1.2.1.99        # ENTITY-SENSOR-MIB, if
the device implements it
snmpwalk -v2c -c public HOST 1.3.6.1.4.1            # vendor (enterprise)
subtree: where most sensors live
```

Task 2: retrieve the measurements

Modbus TCP

Save the simulator (for practice) and the reader. The reader uses a raw socket, so there is no dependency or library-version problem, and it shows the real protocol frame: a 7-byte MBAP header (transaction id, protocol id 0, length, unit id) followed by the PDU (function code 3, start address, quantity).

```
#!/usr/bin/env python3
"""Lab 11: minimal Modbus TCP simulator (functions 3, 4, 6). Usage: python3
modsim.py [PORT] [normal|hot|power]
Registers (holding and input): 0 temp x10 C | 1 humidity x10 %RH | 2 power W
| 3 fan RPM | 4 UPS load %"""
import socketserver, struct, sys
port = int(sys.argv[1]) if len(sys.argv) > 1 else 5020
sc = sys.argv[2] if len(sys.argv) > 2 else "normal"
REG = {"normal": [225, 450, 3200, 4200, 55], "hot": [341, 280, 4100, 800,
62], "power": [248, 470, 7600, 4300, 94]}.get(sc)
if REG is None: sys.exit("scenario must be normal, hot or power")
class H(socketserver.BaseRequestHandler):
    def handle(self):
        while True:
            hdr = self.request.recv(7)
            if len(hdr) < 7: return
            tid, pid, ln, uid = struct.unpack(">HHHB", hdr)
            pdu = self.request.recv(ln - 1); fc = pdu[0]
            if fc in (3, 4):
                a, n = struct.unpack(">HH", pdu[1:5])
                if a + n > len(REG): out = bytes([fc | 0x80, 2])           #
illegal data address
                else: out = bytes([fc, n * 2]) + b"".join(struct.pack(">H",
REG[a + i]) for i in range(n))
            elif fc == 6:
                a, v = struct.unpack(">HH", pdu[1:5])
                if a < len(REG): REG[a] = v; out = pdu
                else: out = bytes([fc | 0x80, 2])
            else: out = bytes([fc | 0x80, 1])
# illegal function
            self.request.sendall(struct.pack(">HHHB", tid, 0, len(out) + 1,
uid) + out)
socketserver.ThreadingTCPServer.allow_reuse_address = True
print(f"Modbus TCP simulator ({sc}) on 0.0.0.0:{port}, unit any. Ctrl+C to
stop", flush=True)
socketserver.ThreadingTCPServer(("0.0.0.0", port), H).serve_forever()
```

```
#!/usr/bin/env python3
"""Lab 11: read a Modbus TCP device and apply thresholds.  Usage: python3
modread.py [HOST] [PORT] [UNIT] [--set REG VALUE]
No external libraries: raw Modbus TCP (MBAP header + function 3)."""
import socket, struct, sys
a = [x for x in sys.argv[1:]]
setv = None
if "--set" in a:
    i = a.index("--set"); setv = (int(a[i + 1]), int(a[i + 2])); del a[i:i +
3]
host = a[0] if len(a) > 0 else input("Host [127.0.0.1]: ") or "127.0.0.1"
port = int(a[1]) if len(a) > 1 else int(input("Port [502]: ") or 502)
unit = int(a[2]) if len(a) > 2 else int(input("Unit id [1]: ") or 1)
def txn(pdu, tid=1):
    s = socket.create_connection((host, port), timeout=3)
    try:
        s.sendall(struct.pack(">HHHB", tid, 0, len(pdu) + 1, unit) + pdu)
        h = s.recv(7); rest = s.recv(struct.unpack(">H", h[4:6])[0] - 1)
        return rest
    finally: s.close()
try:
    if setv:
        r = txn(struct.pack(">BHH", 6, *setv)); print("write reply:", "OK" if
r[0] == 6 else f"exception {r[1]}"); sys.exit(0)
    r = txn(struct.pack(">BHH", 3, 0, 5))
except (OSError, IndexError, struct.error) as e:
    sys.exit(f"Cannot talk to {host}:{port} unit {unit}: {e}  (is the device
up? firewall? port 502 vs 5020?)")
if r[0] & 0x80: sys.exit(f"Modbus exception code {r[1]} (1 illegal function,
2 illegal address, 3 illegal value, 4 device failure)")
v = struct.unpack(f">{r[1] // 2}H", r[2:]); t, h, p, fan, ups = v[0] / 10,
v[1] / 10, v[2], v[3], v[4]
rows = [("Temperature", f"{t:.1f} C", "HIGH: above 27 C" if t > 27 else "LOW:
below 18 C" if t < 18 else "ok"),
        ("Humidity", f"{h:.1f} %RH", "HIGH: above 60 %" if h > 60 else "LOW:
below 20 %" if h < 20 else "ok"),
        ("Power", f"{p} W", "OVERLOAD: above 7000 W" if p > 7000 else "ok"),
        ("Fan speed", f"{fan} rpm", "FAN FAULT: below 1500 rpm" if fan < 1500
else "ok"),
        ("UPS load", f"{ups} %", "CRITICAL: above 90 %" if ups > 90 else
"WARNING: above 80 %" if ups > 80 else "ok")]
print(f"Modbus {host}:{port} unit {unit}, registers 0-4 (function 3)")
for n, val, st in rows: print(f"  {n:<12}{val:<12}{st}")
print("Overall:", "ALARM" if any(s != "ok" for *_, s in rows) else "all
normal")
```

To adapt it to the exam device, change three things: the register start and count in `struct.pack(">BHH", 3, 0, 5)`, the scaling (`/ 10`), and the labels and limits. Use function 4 instead of 3 if the values are input registers.

```
python3 modsim.py 5020 normal &            # practice simulator in the
background
python3 modread.py 127.0.0.1 5020 1
```

Real output I got (normal scenario):

```
Modbus 127.0.0.1:5020 unit 1, registers 0-4 (function 3)
  Temperature 22.5 C      ok
  Humidity    45.0 %RH    ok
  Power       3200 W      ok
  Fan speed   4200 rpm    ok
  UPS load    55 %        ok
Overall: all normal
```

The other scenarios: `hot` (34.1 C, fan 800 rpm: `HIGH: above 27 C` and `FAN FAULT`, ALARM) and `power` (7600 W `OVERLOAD`, UPS 94% `CRITICAL`, ALARM). `python3 modread.py 127.0.0.1 5020 1 --set 0 341` writes 341 to register 0 and the next read then shows 34.1 C `HIGH`. Wrong port gives `Cannot talk to ... Connection refused`.

The same read with the `pymodbus` library (note the keyword changed between versions: newer 3.x, including 3.16 which I tested, uses `device_id=`; older 3.x uses `slave=`; check with `pip show pymodbus`):

```
from pymodbus.client import ModbusTcpClient
c = ModbusTcpClient("127.0.0.1", port=5020)
c.connect()
r = c.read_holding_registers(0, count=5, device_id=1)      # older versions:
slave=1
print(r.registers)                                          # [225, 450,
3200, 4200, 55]
print("temperature:", r.registers[0] / 10, "C")
c.close()
```

Command-line alternative: `mbpoll -m tcp -a 1 -r 1 -c 5 -t 4 HOST -p 502` (the `mbpoll` tool uses 1-based register numbers; install with `sudo apt install mbpoll` if available).

OPC UA

```
#!/usr/bin/env python3
"""Lab 11: tiny OPC UA server.  pip install asyncua   Usage: python3
opcsim.py [PORT]"""
import asyncio, sys
from asyncua import Server
async def main():
    port = int(sys.argv[1]) if len(sys.argv) > 1 else 4840
    s = Server(); await s.init(); s.set_endpoint(f"opc.tcp://0.0.0.0:
{port}/lab/")
    idx = await s.register_namespace("urn:lab11")
    obj = await s.nodes.objects.add_object(idx, "Plant")
    for name, val in (("Temperature", 22.5), ("Pressure", 101.3),
("MotorSpeed", 1450.0)):
        v = await obj.add_variable(idx, name, val); await v.set_writable()
    print(f"OPC UA server on opc.tcp://localhost:{port}/lab/", flush=True)
    async with s:
        while True: await asyncio.sleep(1)
asyncio.run(main())
```

```
#!/usr/bin/env python3
"""Lab 11: read OPC UA variables and apply limits.  pip install asyncua
Usage: python3 opcread.py [URL] [--set NAME VALUE]    e.g.
opc.tcp://localhost:4840/lab/"""
import asyncio, sys
from asyncua import Client
a = sys.argv[1:]; setv = None
if "--set" in a:
    i = a.index("--set"); setv = (a[i + 1], float(a[i + 2])); del a[i:i + 3]
url = a[0] if a else input("Endpoint URL [opc.tcp://localhost:4840/lab/]: ")
or "opc.tcp://localhost:4840/lab/"
LIM = {"Temperature": (10, 30, "C"), "Pressure": (95, 110, "kPa"),
"MotorSpeed": (1000, 1600, "rpm")}
async def main():
    try:
        async with Client(url, timeout=4) as c:
            idx = await c.get_namespace_index("urn:lab11")
            obj = await c.nodes.objects.get_child([f"{idx}:Plant"])
            if setv:
                n = await obj.get_child([f"{idx}:{setv[0]}"]); await
n.write_value(setv[1]); print("written"); return
            print("Browse Objects/Plant:", [(await n.read_browse_name()).Name
for n in await obj.get_children()])
            for n in await obj.get_children():
                name = (await n.read_browse_name()).Name; val = await
```

```
n.read_value(); lo, hi, u = LIM.get(name, (None, None, ""))
                st = "ok" if lo is None or lo <= val <= hi else f"OUT OF
RANGE ({lo}-{hi})"
                print(f"  {name:<12}{val:>8.1f} {u:<4}{st}")
    except Exception as e:
        sys.exit(f"OPC UA error: {type(e).__name__}: {e}  (check URL, port
4840, server running)")
asyncio.run(main())
```

```
python3 opcsim.py 4841 &
python3 opcread.py opc.tcp://localhost:4841/lab/
```

Real output:

```
Browse Objects/Plant: ['Temperature', 'Pressure', 'MotorSpeed']
  Temperature     22.5 C   ok
  Pressure       101.3 kPa ok
  MotorSpeed    1450.0 rpm ok
```

After `python3 opcread.py opc.tcp://localhost:4841/lab/ --set Temperature 38.2` the next read shows `Temperature 38.2 C OUT OF RANGE (10-30)`. A wrong port prints `OPC UA error: ConnectionRefusedError ...`. (Harmless warnings about encryption policy and session timeout appear in the output of this library.) In the exam the namespace URI and node names will differ: browse first (UaExpert, or print `await c.nodes.objects.get_children()`) and read nodes by `NodeId`, for example `c.get_node("ns=2;i=2")`.

SNMP environmental sensor

Real sensors expose temperature and humidity under vendor OIDs or the standard ENTITY-SENSOR-MIB (`entPhySensorValue` 1.3.6.1.2.1.99.1.1.1.4, with scale and precision objects) and UPS-MIB (`1.3.6.1.2.1.33`). Find the right OIDs by walking the enterprise subtree and matching names and plausible values. Values are usually integers (tenths of a degree), so check the MIB description for scaling.

I tested with a real `snmpd` that serves three fake sensor OIDs through a `pass` script (so the commands below are real; only the sensor is fake):

```
snmpget -v2c -c public HOST:161 1.3.6.1.4.1.8072.9999.1.1.0
1.3.6.1.4.1.8072.9999.1.2.0 1.3.6.1.4.1.8072.9999.1.3.0
# iso.3.6.1.4.1.8072.9999.1.1.0 = INTEGER: 225      temperature, tenths of C
(22.5 C)
# iso.3.6.1.4.1.8072.9999.1.2.0 = INTEGER: 450      humidity, tenths of %RH
(45.0 %)
# iso.3.6.1.4.1.8072.9999.1.3.0 = INTEGER: 3200     power in W
snmpwalk -v2c -c public HOST 1.3.6.1.4.1.8072.9999.1
snmpget -v2c -c public HOST 1.3.6.1.4.1.8072.9999.1.9.0     # No Such
Instance currently exists at this OID  (wrong OID)
snmpget -v2c -c wrong -t 1 -r 0 HOST OID                    # Timeout: No
Response  (wrong community)
```

The threshold reader `envsnmp.sh` (arguments: host[:port], community, then the three OIDs; defaults are my test OIDs):

```
#!/bin/bash
# usage: ./envsnmp.sh [HOST[:PORT]] [COMMUNITY] [TEMP_OID] [HUM_OID]
[POWER_OID]  -> reads an SNMP environmental sensor and applies thresholds
H=${1:-}; C=${2:-public}
[ -z "$H" ] && read -rp "Sensor host[:port]: " H
T=${3:-1.3.6.1.4.1.8072.9999.1.1.0}; U=${4:-1.3.6.1.4.1.8072.9999.1.2.0};
P=${5:-1.3.6.1.4.1.8072.9999.1.3.0}
command -v snmpget >/dev/null || { echo "snmpget missing: sudo apt install -y
snmp"; exit 2; }
get() { snmpget -v2c -c "$C" -t 2 -r 1 -Ovq "$H" "$1" 2>/dev/null; }
t=$(get $T); u=$(get $U); p=$(get $P)
for x in "$t" "$u" "$p"; do [[ "$x" =~ ^[0-9]+$ ]] || { echo "No numeric
answer (wrong host, community, OID, or sensor down): '$x'"; exit 1; }; done
awk -v t="$t" -v u="$u" -v p="$p" 'BEGIN {
  t/=10; u/=10; bad=0
  printf "Temperature %.1f C  %s\n", t, (t>27)?"HIGH (>27)":(t<18)?"LOW
(<18)":"ok"
  printf "Humidity    %.1f %%RH %s\n", u, (u>60)?"HIGH (>60)":(u<20)?"LOW
(<20)":"ok"
  printf "Power       %d W    %s\n", p, (p>7000)?"OVERLOAD (>7000)":"ok"
  if (t>27||t<18||u>60||u<20||p>7000) { print "ALARM"; exit 3 } else print
"all normal" }'
```

Results I got: normal sensor gives `Temperature 22.5 C ok / Humidity 45.0 %RH ok / Power 3200 W ok / all normal` (exit 0); a sensor at 34.1 C and 7600 W gives `HIGH (>27)`, `OVERLOAD (>7000)`, `ALARM` (exit 3); a wrong community gives the `No numeric answer ...` message (exit 1). If the sensor is a real `snmpd`-style agent, the `-Ovq` option prints only the value, which is why the script can parse numbers. To rebuild my fake sensor for practice you need a `pass` handler script that prints the OID, type and value for

`-g` (get) and `-n` (next) requests and these lines in `/etc/snmp/snmpd.conf`: `rocommunity public 127.0.0.1` and `pass .1.3.6.1.4.1.8072.9999.1 /path/to/handler.sh`, then `sudo systemctl restart snmpd`. This is optional for the exam; the exam supplies the sensor.

Task 3: analyze measurements and identify abnormal conditions

Reference ranges (state your source; ASHRAE TC 9.9 recommends 18 to 27 C inlet temperature for data centers, and relative humidity roughly 40 to 60%, with 20% to 80% as the allowable envelope):

Measurement	Normal	Abnormal and likely meaning
Temperature	18 to 27 C	Above 27 C: cooling failure, blocked airflow, overload. Fast rise (0.5 C/min or more) with steady power: cooling unit/fan/CRAC failure. Below 18 C: overcooling, wasted energy
Humidity	40 to 60 %RH (alarm outside 20 to 80)	Too high: condensation and corrosion, leak or door open. Too low (under 20%): static discharge risk
Power	Below about 80% of circuit or UPS capacity	Near or over capacity: overload; sudden drop to zero: power loss or breaker trip
UPS load/battery	Load under 80%; battery full on mains	Load above 90%: critical. Battery discharging: mains failure
Fan speed	Within the normal band (e.g. 3000 to 6000 rpm)	Near zero: fan failure, heat soon follows
Process values (pressure, motor speed)	Inside the engineering limits	Out of range for a sustained period: process fault; a flat line (identical values for a long time) can mean a frozen sensor
Comms	Fresh timestamps	Stale data or timeouts: device or network failure; no data is also an alarm

Save `envcheck.py`. It reads a CSV (`time,temp_c,humidity,power_w`), marks every row that breaks a limit or shows a fast rise, reports min/average/max and the first time each problem appeared, and prints a pattern hint. Limits can be changed with `--tmax --hmax --pmax`.

```
#!/usr/bin/env python3
"""Lab 11: analyse environmental/process measurements.  Usage: python3
envcheck.py [FILE.csv] [--tmax 27] [--hmax 60] [--pmax 7000]
CSV header: time,temp_c,humidity,power_w   (time as HH:MM)"""
import argparse, csv, sys
ap = argparse.ArgumentParser(); ap.add_argument("file", nargs="?");
ap.add_argument("--tmax", type=float, default=27); ap.add_argument("--hmax",
type=float, default=60); ap.add_argument("--pmax", type=float, default=7000)
a = ap.parse_args(); path = a.file or input("CSV file: ").strip()
try:
    rows = [(r["time"], float(r["temp_c"]), float(r["humidity"]),
float(r["power_w"])) for r in csv.DictReader(open(path))]
except (OSError, KeyError, ValueError) as e:
    sys.exit(f"Cannot use {path}: {e}")
if len(rows) < 2: sys.exit("Need at least 2 rows.")
mins = lambda s: int(s[:2]) * 60 + int(s[3:5])
print(f"{'Time':<7}{'Temp':>6}{'RH%':>6}{'Power':>7}  Findings")
first = {}
for i, (t, te, h, p) in enumerate(rows):
    f = []
    if te > a.tmax: f.append(f"temp>{a.tmax:g}")
    if h > a.hmax: f.append(f"humidity>{a.hmax:g}")
    if h < 20: f.append("humidity<20 (static risk)")
    if p > a.pmax: f.append(f"power>{a.pmax:g}")
    if i:
        dt = mins(t) - mins(rows[i - 1][0]) or 1; rate = (te - rows[i - 1]
[1]) / dt
        if rate >= 0.5: f.append(f"rapid rise {rate:.2f} C/min")
    for k in f: first.setdefault(k.split()[0], t)
    print(f"{t:<7}{te:6.1f}{h:6.1f}{p:7.0f}  {'; '.join(f) or 'ok'}")
T = [r[1] for r in rows]
print(f"\nTemp min/avg/max: {min(T):.1f}/{sum(T)/len(T):.1f}/{max(T):.1f} C |
peak power {max(r[3] for r in rows):.0f} W")
print("First occurrence:", first or "none")
if any(k.startswith("rapid") for k in first) and any(k.startswith("temp") for
k in first):
    print("Pattern: fast temperature rise with normal power points to cooling
failure (CRAC/fan/airflow), not an electrical overload.")
```

```
if any(k.startswith("power") for k in first) and any(k.startswith("temp") for
k in first):
    print("Pattern: power and temperature rise together: load growth is
heating the room; check cooling capacity.")
```

Test dataset:

```
cat > env_sample.csv <<'EOF'
time,temp_c,humidity,power_w
10:00,22.4,45,3200
10:02,22.6,45,3210
10:04,23.1,44,3190
10:06,25.0,42,3200
10:08,27.8,40,3205
10:10,31.2,37,3200
10:12,34.0,34,3195
10:14,34.5,33,3200
EOF
python3 envcheck.py env_sample.csv
```

Expected: 10:00 to 10:04 `ok`; 10:06 `rapid rise 0.95 C/min`; 10:08 `temp>27; rapid rise 1.40 C/min`; 10:10 `temp>27; rapid rise 1.70 C/min`; 10:12 `temp>27; rapid rise 1.40 C/min`; 10:14 `temp>27`. Summary line `Temp min/avg/max: 22.4/27.6/34.5 C | peak power 3210 W`; first occurrence `rapid` at 10:06 and `temp>27` at 10:08; the pattern line says a fast rise with normal power points to cooling failure. With `--tmax 30` the first temperature breach moves to 10:10. A missing file prints `Cannot use ...`.

Example wording: "From 10:06 the temperature climbed about 1 to 1.7 C per minute and passed the 27 C limit at 10:08, reaching 34.5 C, while power stayed flat at about 3200 W and humidity slowly fell. Because the load did not change, this is a cooling failure (CRAC unit, chiller, fan or blocked airflow), not an electrical overload. The first warning was the rate of rise at 10:06, two minutes before the absolute limit, which shows why rate-of-change alerts give earlier notice."

Task 4: threshold alert and operational response

Demonstrate the alert with the tools you already have (all of this is tested above):

- **Script alert:** `modread.py`, `opcread.py` or `envsnmp.sh` print the status per metric and `ALARM`. `envsnmp.sh` exits with code 3 on alarm, so it can drive other tools. Trigger the alarm safely by changing the simulated value (for example `python3 modread.py HOST PORT 1 --set 0 341`, or `opcread.py URL --set Temperature 38.2`, or by starting the simulator in its `hot` scenario), run the reader, show `ALARM`, then restore the value and show recovery.

- **Scheduled check with logging:** run it every minute and log alarms to syslog (Lab 3 shows where they go):

```
# crontab -e   (every minute)
* * * * * /home/USER/lab11/envsnmp.sh 127.0.0.1:1161 >/tmp/env.out || logger
-p user.crit -t envmon "ALARM $(tr '\n' ' ' </tmp/env.out)"
```

- **Monitoring-system alert:** the same threshold can be expressed in Zabbix (`last(/host/env.temp)>27`, Lab 4), a Prometheus rule (`environment_temperature_celsius > 27 for 2m`, Lab 8; the Modbus/OPC UA values would need an exporter) or a LibreNMS sensor alert rule (Lab 5, LibreNMS discovers ENTITY-SENSOR sensors and has a built-in sensor alert).

Good alert design: two levels (warning at 27 C, critical at 32 C), a short delay or count to ignore single bad readings, a rate-of-rise alert, an alert on missing data, and recovery messages.

Recommended operational response (state it in order, adapted to the fault):

Condition	Immediate action	Follow-up
High temperature (cooling failure)	Confirm with a second sensor; check CRAC/chiller status and alarms; verify airflow and that doors/panels are closed; start backup or portable cooling	If it continues to climb: shed non-critical load, then graceful shutdown of equipment before thermal protection trips (for example at 35 C); notify facilities; root-cause analysis
Humidity high/low	Check for leaks, door seals, humidifier/dehumidifier status	Adjust setpoints; inspect for condensation or static control
Power overload (above 80 to 90% of capacity)	Identify the heaviest loads, move or shed non-critical loads, balance phases	Add capacity or redundancy (N+1), review power budget
Power loss, UPS on battery	Confirm battery runtime remaining, start generator, graceful shutdown timers	Investigate mains/transfer-switch failure, test UPS
Fan or equipment fault	Replace or bypass, move load	Preventive maintenance and spare parts

Condition	Immediate action	Follow-up
Sensor stale or comms lost	Treat as an alarm; check network and device; verify locally	Redundant sensors and watchdog alarms

Safety principle: in industrial systems the monitoring side must not control the process without authorization. Writing to registers or nodes can change a real machine, so only write in a simulator or with approval, and prefer read-only access and a separate, authorized control path.

Likely examiner modifications

- Read a different register range, or convert values with a different scaling or sign (two's complement for negative temperatures: if value is above 32767 subtract 65536).
- Read the same data using function 4 (input registers) instead of 3, or read a coil/bit (door open, alarm).
- Combine two registers into a 32-bit value or float (`struct.unpack('>f', struct.pack('>HH', hi, lo))`, and mention word order differences).
- Change the unit id (slave id), poll every N seconds in a loop, or show the min, max and average across 10 readings.
- For OPC UA: browse the address space, read a node by NodeId, read data type and status code, or subscribe to changes.
- For SNMP: get the sysUpTime and sysName, or compute a rate; use `snmpwalk` to find the sensor table; read the thresholds that the device itself exposes.
- Modify a threshold and show the alarm firing and clearing; add a second level (warning and critical).
- Explain what happens if the sensor value freezes or the reading fails (stale data handling).

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`Connection refused` (Modbus)	Simulator not running, wrong port (502 needs root on a server; labs often use 5020)	`ss -tlnp`, `nc -vz HOST PORT`	Start it; use the right port

Symptom or message	Likely cause	Check	Fix
Timeout, no response	Firewall, wrong IP, wrong unit id on a serial gateway	`ping`, `nc`; try unit ids 0, 1, 255	Open the port; correct IP and unit
Modbus exception 2 (illegal data address)	Reading registers that do not exist, or 0-based vs 1-based confusion	The register map; try address minus 1	Correct start address and count
Modbus exception 1 (illegal function)	Wrong function for that table (3 vs 4)	Datasheet	Use function 4 for input registers
Modbus exception 3 or 4	Bad quantity/value, device busy	Count limit is 125 registers	Reduce the count; retry
Values look 10 or 100 times too large	Scaling factor not applied	Datasheet, `/ 10`	Divide as documented
Negative temperature shows 65xxx	Unsigned reading of a signed value	Value above 32767	Subtract 65536
Float values nonsense	Word or byte order of 32-bit values	Try swapped register order	Swap the two registers
`TypeError ... unexpected keyword argument 'slave'` (pymodbus)	Library version changed the keyword	`pip show pymodbus`	Use `device_id=` in new versions, `slave=` in old
`ModuleNotFoundError: asyncua`	Not installed	`pip3 list | grep asyncua`	`pip3 install --user asyncua`; add `~/.local/bin` to PATH
OPC UA `BadNodeIdUnknown` / `get_child` fails	Wrong namespace index or browse name	Browse the tree first	Use the correct NodeId (`ns=2;i=...`)

Symptom or message	Likely cause	Check	Fix
OPC UA `BadSecurityChecksFailed` / certificate errors	Server requires a secure policy or trusted certificate	The endpoints list	Use the server's policy, accept/trust the certificate, or ask for the anonymous endpoint
`Timeout: No Response` (SNMP)	Wrong community, ACL, wrong port	`snmpget -v2c -c COMM HOST sysDescr.0`	Fix the community or add your IP to the agent's allow list
`No Such Object` / `No Such Instance`	OID wrong or missing the `.0` instance suffix	`snmpwalk` the subtree	Use the full OID with `.0`
Sensor value is a string like `22.5 C`	The sensor returns text	`snmpget` output type	Strip the unit before arithmetic (`sed`)
Readings never change	Frozen sensor or cached poll	Compare timestamps over time	Check the device, restart the poller

Viva theory

Why monitor physical infrastructure. IT availability depends on power, cooling and environment. Heat, humidity, water, power faults and door/physical intrusion cause outages and damage. Monitoring gives early warning and supports capacity planning and compliance.

Industrial control basics. A **PLC** (programmable logic controller) controls machines; **RTUs** are remote terminal units; **SCADA** supervises and collects data from many sites; **HMI** is the operator screen; **sensors** measure (temperature, pressure) and **actuators** act (valves, motors). Industrial networks follow the Purdue model, with the control network separated from the business network by firewalls (IT/OT segmentation).

Modbus. A simple master-slave (client-server) protocol from 1979. Modbus TCP wraps the frame in TCP with the MBAP header; no authentication or encryption, so it must live on isolated networks. Address model: coils, discrete inputs, input registers, holding registers. Function codes: 1 read coils, 2 read discrete inputs, 3 read holding registers, 4 read input

registers, 5 write single coil, 6 write single register, 15 write multiple coils, 16 write multiple registers. Exceptions set the high bit of the function code (0x83 means function 3 failed) plus an exception code.

OPC UA. Platform-independent, service-oriented industrial standard (IEC 62541). Provides a browsable address space of nodes (objects, variables, methods), data types, subscriptions with monitored items, historical access, and built-in security (certificates, signing, encryption, user authentication). It replaces classic OPC (which needed Windows COM/DCOM).

SNMP environmental monitoring. Rack and room sensors, PDUs and UPS devices expose temperature, humidity and power through SNMP MIBs (ENTITY-SENSOR-MIB, UPS-MIB, vendor MIBs); NMS tools poll or receive traps.

Thresholds. Static limits (fixed value), rate-of-change limits, and baselines. Use hysteresis and delays to avoid flapping. Multi-level (warning, critical). Alert on stale or missing data too.

PUE. Power usage effectiveness is total facility power divided by IT equipment power; 1.0 is perfect, around 1.5 is typical, below 1.2 is excellent.

Safety and security. Read-only monitoring by default; segment OT from IT; no direct internet exposure for Modbus; log every write; change control for set-points.

Typical viva questions: Default Modbus TCP port? (502.) Which function reads holding registers? (3.) What is the difference between coils and registers? (Bits versus 16-bit words.) Why is Modbus insecure? (No authentication or encryption.) What is the MBAP header? (Transaction id, protocol id, length, unit id.) How does OPC UA differ from Modbus? (Typed, named, browsable, secure, and event-driven versus flat registers.) What port does OPC UA use? (4840.) What temperature range does ASHRAE recommend? (18 to 27 C.) What causes a rapid temperature rise with constant power? (Cooling failure.) What would you do on a critical temperature alarm? (Verify, restore cooling, shed load, graceful shutdown, notify.) What is a PLC? (A rugged industrial computer that controls machines.) What does SCADA do? (Supervisory control and data acquisition across sites.)

Lab 12: Event correlation and monitoring automation

Scenario: investigate alerts generated by an application server and its network dependencies. Marks: Task 1 (2) import or inspect the supplied infrastructure event dataset; Task 2 (3) apply filtering, deduplication and time-based correlation to group related events; Task 3 (3) analyze correlated events to identify a probable root cause and

distinguish primary events from secondary symptoms; Task 4 (2) demonstrate a prepared Ansible remediation playbook in check mode or a controlled sandbox, and specify how recovery would be verified.

Tested here: the correlation script on a 12-event dataset, and the Ansible playbook with `ansible-core` 2.22 (syntax check, check mode with diff, a real apply into a temporary folder, an idempotent second run, and drift detection). The exam's playbook is "prepared", so you will mostly run and explain it; my playbook is practice material that exercises the same commands. Ubuntu 22.04's `apt install ansible` gives an older release (2.10 or 5.x); the commands and the modules used here work on both.

Setup

```
mkdir -p ~/lab12 && cd ~/lab12
sudo apt update && sudo apt install -y python3 ansible          # or:  pip3
install --user ansible-core
ansible --version | head -2
python3 --version
```

Task 1: import or inspect the event dataset

The dataset is usually a CSV or JSON/log file exported from a monitoring system. Always inspect before analyzing:

```
F=events.csv                                    # the supplied file
file $F; wc -l $F; head -5 $F                   # type, size, header and first
rows
head -1 $F | tr ',' '\n' | cat -n               # list the columns (timestamp,
host, source, severity, message)
cut -d, -f2 $F | tail -n +2 | sort | uniq -c | sort -rn      # events per
host
cut -d, -f4 $F | tail -n +2 | sort | uniq -c | sort -rn      # events per
severity
cut -d, -f5 $F | tail -n +2 | sort | uniq -c | sort -rn      # most repeated
messages (duplicates)
sort -t, -k1 $F | head -3; sort -t, -k1 $F | tail -3         # time span
covered (first and last)
# JSON input:  jq -r '.[] | [.timestamp,.host,.severity,.message] | @csv'
events.json
# Excel/other: python3 -c "import pandas as pd;
print(pd.read_csv('events.csv').describe(include='all'))"
```

Note the timestamp format and time zone, whether hosts are named consistently, and any missing values. Create the practice dataset now (12 events: a switch port failure that cascades through the database, the application and the web tier, followed by unrelated noise):

```
cat > events.csv <<'EOF'
timestamp,host,source,severity,message
2026-11-03 10:00:05,sw01,snmp,critical,Interface Gi0/3 operational status
down
2026-11-03 10:00:05,sw01,snmp,critical,Interface Gi0/3 operational status
down
2026-11-03 10:00:12,db01,zabbix,high,Host unreachable (agent.ping failed)
2026-11-03 10:00:20,app01,app,error,Database connection timeout
2026-11-03 10:00:25,app01,app,error,Database connection timeout
2026-11-03 10:00:31,app01,app,error,Database connection timeout
2026-11-03 10:00:40,web01,nginx,error,502 Bad Gateway from upstream app01
2026-11-03 10:00:55,web01,nginx,error,502 Bad Gateway from upstream app01
2026-11-03 10:01:10,app01,prometheus,warning,HighLatency p95 above 1s
2026-11-03 10:25:00,web02,syslog,info,Disk usage 71%
2026-11-03 10:40:00,app01,systemd,warning,Service exporter restarted
2026-11-03 10:40:02,app01,systemd,warning,Service exporter restarted
EOF
```

Task 2: filtering, deduplication and time-based correlation

The three techniques, in order:

1. **Filter.** Remove events you do not need: below a minimum severity, other hosts, other sources, or outside a time range.
2. **Deduplicate.** Collapse repeats of the same event (same host and message) that occur within a short interval into one event with a count. This reduces alert noise without losing the fact that it repeated.
3. **Correlate by time (and dependency).** Group events that occur close together in time (here: a new group starts when the gap to the previous event exceeds a window, 120 s by default). Events in one group are probably one incident. Then use topology (which host depends on which) to find the cause.

Quick shell versions:

```
awk -F, 'NR>1 && $4=="error"||$4=="critical"||$4=="high"' events.csv
# filter by severity
awk -F, 'NR>1 && $2=="app01"' events.csv
# filter by host
awk -F, 'NR>1 {print $2","$5}' events.csv | sort | uniq -c | sort -rn
# count duplicates by host+message
sort -t, -k2,2 -k5,5 -u events.csv
# crude de-duplication (keeps the first of each host+message, loses the time
order)
awk -F, 'NR>1 {print substr($1,12,5)}' events.csv | uniq -c
# events per minute (time buckets)
```

Save `correlate.py`. It reads the CSV, filters (`--minsev`, `--host`), deduplicates (`--dedup` seconds), groups into incidents (`--window` seconds), orders events, labels the earliest event on the lowest dependency layer as PRIMARY and the others as secondary, and proposes the root cause. The dependency layers are guessed from the host name prefix: `sw`, `rt`, `fw` (network) below `db` (database) below `app` (application) below `web` (front end). If your hosts are named differently, edit the `LAYER` line to match the topology you are given.

```
#!/usr/bin/env python3
"""Lab 12: filter, deduplicate and time-correlate infrastructure events;
propose a root cause.
Usage: python3 correlate.py [events.csv] [--window 120] [--dedup 60] [--
minsev info] [--host H]
CSV header: timestamp,host,source,severity,message   (timestamp YYYY-MM-DD
HH:MM:SS)"""
import argparse, csv, sys
from datetime import datetime
ap = argparse.ArgumentParser(); ap.add_argument("file", nargs="?");
ap.add_argument("--window", type=int, default=120)
ap.add_argument("--dedup", type=int, default=60); ap.add_argument("--minsev",
default="info"); ap.add_argument("--host")
a = ap.parse_args(); path = a.file or input("Events CSV: ").strip()
SEV = {"info": 0, "warning": 1, "error": 2, "high": 3, "critical": 4}
LAYER = {"sw": 0, "rt": 0, "fw": 0, "db": 1, "app": 2, "web": 3}      # lower
= deeper dependency (network < database < application < web)
layer = lambda h: next((v for k, v in LAYER.items() if h.startswith(k)), 2)
try:
    ev = [dict(t=datetime.strptime(r["timestamp"], "%Y-%m-%d %H:%M:%S"),
host=r["host"], src=r["source"], sev=r["severity"].lower(), msg=r["message"])
          for r in csv.DictReader(open(path))]
    assert a.minsev in SEV and all(e["sev"] in SEV for e in ev)
except (OSError, KeyError, ValueError, AssertionError) as e:
    sys.exit(f"Cannot use {path}: {e!r} (check header, timestamp format,
```

```
severity names)")
ev.sort(key=lambda e: e["t"]); n0 = len(ev)
ev = [e for e in ev if SEV[e["sev"]] >= SEV[a.minsev] and (not a.host or
e["host"] == a.host)]
n1 = len(ev); last, ded = {}, []
for e in ev:                                                    # deduplicate:
same host+message within --dedup seconds of the previous occurrence
    k = (e["host"], e["msg"])
    if k in last and (e["t"] - last[k]["t"]).total_seconds() <= a.dedup:
last[k]["count"] += 1; last[k]["t_last"] = e["t"]; continue
    e["count"] = 1; e["t_last"] = e["t"]; last[k] = e; ded.append(e)
groups = []                                                     # correlate: a new
group starts when the gap since the previous event exceeds --window
for e in ded:
    if groups and (e["t"] - groups[-1][-1]["t"]).total_seconds() <= a.window:
groups[-1].append(e)
    else: groups.append([e])
print(f"Events read {n0} | after filter {n1} | after dedup {len(ded)} |
correlated incidents {len(groups)}")
for i, g in enumerate(groups, 1):
    root = min(g, key=lambda e: (layer(e["host"]), e["t"]))
    print(f"\nIncident {i}: {g[0]['t']:%H:%M:%S} to {g[-1]
['t_last']:%H:%M:%S}, {len(g)} distinct events, hosts {sorted({e['host'] for
e in g})}")
    for e in g:
        tag = "PRIMARY  " if e is root and len(g) > 1 else "secondary" if
len(g) > 1 else "isolated "
        print(f"  {e['t']:%H:%M:%S} {tag} {e['host']:<6}{e['sev']:<9}
{e['msg']}" + (f"  (x{e['count']})" if e["count"] > 1 else ""))
    if len(g) > 1: print(f"  Probable root cause: {root['host']} -
{root['msg']} (earliest event on the lowest dependency layer; the rest are
downstream symptoms)")
```

```
python3 correlate.py events.csv
```

Expected output (this is the real output):

```
Events read 12 | after filter 12 | after dedup 7 | correlated incidents 3

Incident 1: 10:00:05 to 10:01:10, 5 distinct events, hosts ['app01', 'db01',
'sw01', 'web01']
  10:00:05 PRIMARY   sw01  critical Interface Gi0/3 operational status down
(x2)
  10:00:12 secondary db01  high     Host unreachable (agent.ping failed)
  10:00:20 secondary app01 error    Database connection timeout  (x3)
  10:00:40 secondary web01 error    502 Bad Gateway from upstream app01  (x2)
  10:01:10 secondary app01 warning  HighLatency p95 above 1s
  Probable root cause: sw01 - Interface Gi0/3 operational status down
(earliest event on the lowest dependency layer; the rest are downstream
symptoms)

Incident 2: 10:25:00 to 10:25:00, 1 distinct events, hosts ['web02']
  10:25:00 isolated  web02 info     Disk usage 71%

Incident 3: 10:40:00 to 10:40:02, 1 distinct events, hosts ['app01']
  10:40:00 isolated  app01 warning  Service exporter restarted  (x2)
```

Other runs and what they prove:

Command	Result
`python3 correlate.py events.csv --minsev error --window 30`	12 read, 8 after filter, 4 after dedup, 1 incident; the same primary (sw01) and the same cascade without the noise and without the warning events
`python3 correlate.py events.csv --host web01`	2 events after filter, 1 after dedup, one isolated incident (`502 Bad Gateway` , x2): filtering by host alone loses the cause
`python3 correlate.py nofile.csv`	`Cannot use nofile.csv: ...`
`--dedup 2`	The duplicate `Database connection timeout` events 5 s apart are no longer merged, so more distinct events appear: the dedup window matters

The numbers: 12 raw events reduced to 7 by deduplication (5 duplicates removed: one switch repeat, two database timeouts, one 502, one exporter restart) and grouped into 3 incidents, of which only one needs action. Raw count to incidents is a 75% noise reduction. This is the point of event correlation: fewer, more meaningful alerts.

Task 3: identify the root cause, primary and secondary events

Rules of thumb for root-cause reasoning:

- **Time order:** the cause precedes its effects. The earliest event in the cluster is the first candidate, but clocks can differ, so check NTP.
- **Dependency direction:** failures flow up the dependency chain: network, then database, then application, then web. An event on a lower layer that is followed by events on higher layers is the cause; the reverse is rarely true.
- **Event content:** messages naming a dependency ("Database connection timeout", "502 from upstream app01") are symptoms that point at that dependency.
- **Scope:** one cause with many symptoms across hosts is more probable than many independent causes happening in the same minute.
- **Confirmation:** verify the hypothesis with a direct check, never from the alerts alone.

For the practice data: sw01 reports `Interface Gi0/3 down` at 10:00:05 (primary). Seven seconds later db01 becomes unreachable (the database is cabled to that port), then app01 cannot reach the database (timeouts at 10:00:20 to 10:00:31), then web01 gets bad gateways from app01 at 10:00:40, and finally latency rises at 10:01:10. The sequence follows the dependency chain exactly, so the probable root cause is the network switch port failure, and the db01, app01 and web01 events are secondary symptoms. The events at 10:25 (disk 71%) and 10:40 (exporter restart) are isolated and unrelated, which shows why the time window matters.

What to do to confirm: check `ping db01` and the switch interface status (Lab 2 and Lab 5: `snmpget ... ifOperStatus.3` or LibreNMS port page), look for link flaps in the switch log, check the cable or SFP, then see whether the database and application recover when the link is restored.

Wording for the answer: "The probable root cause is the operational failure of interface Gi0/3 on sw01 at 10:00:05, the earliest event and on the lowest dependency layer. The database host behind it became unreachable seven seconds later, causing connection timeouts on app01 and 502 errors on web01. These are secondary symptoms, so alerting on them separately would create four alerts for a single fault." Mention the confidence level (probable, not certain) and the verification step.

Task 4: demonstrate the Ansible remediation playbook in check mode and verify recovery

Ansible in one minute: it runs **tasks** from a **playbook** (YAML) on hosts listed in an **inventory**, over SSH and without an agent. Modules are **idempotent**: they change the system only if it differs from the desired state, so you can run a playbook repeatedly.

Check mode (`--check` , also called dry run) simulates the changes without making them; `--diff` shows what would change in files.

The inventory and playbook I tested (the exam provides its own; run theirs the same way):

```
[app]
localhost ansible_connection=local
```

```
---
- name: Lab 12 remediation - restore app configuration and service
  hosts: app
  become: false            # set true (and use sudo) on the real server
  vars:
    conf_dir: /tmp/lab12   # real server: /etc/myapp
    db_host: 10.0.0.5
    service_name: cron     # real server: the exam application service
  tasks:
    - name: Ensure the config directory exists
      ansible.builtin.file:
        path: "{{ conf_dir }}"
        state: directory
        mode: "0755"

    - name: Ensure the application points to the right database host
      ansible.builtin.lineinfile:
        path: "{{ conf_dir }}/app.conf"
        regexp: '^db_host='
        line: "db_host={{ db_host }}"
        create: true
        mode: "0644"
      notify: Restart application

    - name: Ensure the application service is running and enabled
      ansible.builtin.systemd:
        name: "{{ service_name }}"
        state: started
        enabled: true
      when: ansible_facts['service_mgr'] == 'systemd'
      tags: service

    - name: Verify the config value is present
      ansible.builtin.command: grep -q '^db_host={{ db_host }}$' {{ conf_dir
}}/app.conf
      changed_when: false
      when: not ansible_check_mode
```

```
  handlers:
    - name: Restart application
      ansible.builtin.systemd:
        name: "{{ service_name }}"
        state: restarted
      when: ansible_facts['service_mgr'] == 'systemd'
```

The sequence to demonstrate (all run and checked):

```
ansible-playbook -i inventory.ini remediate.yml --syntax-check          #
prints: playbook: remediate.yml
ansible-playbook -i inventory.ini remediate.yml --list-tasks            #
what it would do
ansible-playbook -i inventory.ini remediate.yml --check --diff          # DRY
RUN: nothing is changed
ls /tmp/lab12                                                           #
still does not exist: proof that check mode changed nothing
ansible-playbook -i inventory.ini remediate.yml                         #
real run in the sandbox
ansible-playbook -i inventory.ini remediate.yml                         #
second run: changed=0 proves idempotence
```

What I saw: the check run reported `ok=3 changed=2 ... skipped=3` and showed the diff `state: absent -> directory` and `+db_host=10.0.0.5`, with no folder created; the real run reported `changed=2`; the repeat reported `changed=0`. When the file had drifted to `db_host=1.1.1.1`, check mode showed `-db_host=1.1.1.1` and `+db_host=10.0.0.5` and left the file unchanged. (The service tasks are skipped in my sandbox because it has no systemd; on a normal Ubuntu server they run.)

Useful options: `-i inventory.ini` (inventory), `-l HOST` (limit to a host), `-u USER -k` (SSH user and password prompt), `-K` or `--ask-become-pass` (sudo password), `-b` (become), `--tags service` / `--skip-tags service`, `-e db_host=10.0.0.9` (override a variable), `-v` / `-vvv` (verbosity), `--step` (confirm every task), `ansible all -i inventory.ini -m ping` (connectivity test), `ansible app -i inventory.ini -m setup -a 'filter=ansible_service_mgr'` (facts).

Two gotchas I hit while building it: (1) a verification task that reads a file created earlier fails in check mode because the earlier task did not really create it; protect such tasks with `when: not ansible_check_mode` (or use `check_mode: false` only when the target already exists); (2) in newer Ansible, use `ansible_facts['service_mgr']` instead of the old `ansible_service_mgr` variable to avoid a deprecation warning.

How recovery would be verified (state this explicitly; the task asks for it):

Layer	Check	Success criterion
Network	`snmpget ... ifOperStatus` for Gi0/3, or the LibreNMS port page; `ping db01`	Interface `up(1)`; ping replies, no loss
Database	`nc -vz db01 5432` (or 3306); a simple query	Port open, query answers
Application	`systemctl status APP`; the application's health URL; `journalctl -u APP`	`active (running)`; HTTP 200; no more connection timeouts in the log
Web tier	`curl -s -o /dev/null -w '%{http_code}\n' http://web01/` and `curl` timing (Lab 9)	200 and normal latency; error rate back below the SLO threshold (Lab 7/9)
Monitoring	Alerts resolve in Zabbix/Prometheus/LibreNMS (Lab 4, 5, 8); the problem list is empty	Recovery events received; no new alerts for a soak period such as 15 minutes
Automation	Re-run the playbook	`changed=0` shows the system already matches the desired state

Operational practice to mention: always run `--check --diff` first, limit to the affected host with `-l`, take a backup or snapshot, have a rollback (git-tracked playbooks, or run a reversing playbook), run in a sandbox or staging before production, and record the change.

Likely examiner modifications

- Change the correlation window or dedup window and explain how the result changes (a very small window splits one incident into many; a very large one merges unrelated incidents).
- Filter to one host, a severity, a time range or a source (SNMP only, application only).
- Count events per host, severity or per minute and identify the busiest host.
- Identify the primary event when timestamps are not in order (sort first), or when two clusters overlap.
- Run the playbook against a different host group with `-l`, with an extra variable (`-e`), with only one tag, or print the diff for a file change.
- Explain what each Ansible task would do in check mode and why one is reported `changed` or `ok` or `skipped`.

- Write a small new task, for example ensure a package is installed (`ansible.builtin.apt: name=nginx state=present update_cache=true`), a service restarted, or a file copied (`ansible.builtin.copy`).
- Show that the playbook is idempotent by running it twice.

Common errors and how to resolve them

Symptom or message	Likely cause	Check	Fix
`ansible-playbook: command not found`	Ansible not installed or `~/.local/bin` not in PATH	`which ansible-playbook`	`sudo apt install ansible` or `export PATH=$PATH:~/.local/bin`
`[WARNING]: No inventory was parsed, only implicit localhost is available`	Inventory path wrong	`ls inventory.ini`	Pass `-i inventory.ini` with the right path
`Host key verification failed` / `UNREACHABLE! ... Permission denied (publickey,password)`	SSH not set up for the target	`ssh user@host` manually	Copy keys (`ssh-copy-id`), set `-u USER -k`, or `ansible_connection=local` for the local machine
`Missing sudo password` / `Permission denied` on a task	`become` needs a password	The task output	Add `-K` (`--ask-become-pass`) or configure passwordless sudo
`ERROR! We were unable to read ... as json or yaml` / `mapping values are not allowed`	YAML indentation or tab characters	The line and column in the message	Fix spaces (two-space indent), no tabs; run `--syntax-check`
`The task includes an option with an undefined variable`	Variable not defined	The variable name in the message	Define it in `vars:` or pass `-e`
Check mode fails on a task that depends on an earlier change	The earlier change was not really made in check mode	The failing task	`when: not ansible_check_mode`, or skip with a tag
`couldn't resolve module/action`	Module name wrong, or a collection missing	`ansible-doc -l | grep NAME`	Use the fully qualified `ansible.builtin.NAME`; install the collection
Service task: `Could not find the requested service`	Wrong service name	`systemctl list-units --type=service | grep NAME`	Use the exact unit name
Everything shows `changed` on every run	Non-idempotent task (raw `command` / `shell`)	Use of `command`	Use a proper module, or add `creates:` or `changed_when:`

Symptom or message	Likely cause	Check	Fix
`Failed to connect to the host via ssh ... Connection timed out`	Network or firewall problem (it may be the incident itself)	`ping`, `nc -vz host 22`	Fix connectivity first; this is also a recovery check
`correlate.py` says `Cannot use ... KeyError`	Header names differ	`head -1 events.csv`	Rename the columns or edit the field names in the script
`ValueError: time data ... does not match format`	Different timestamp format	`head -3`	Change the format string (for example `%d/%b/%Y %H:%M:%S`)
Everything ends up in one big incident	Window too large, or a flood of events	Try `--window 30`	Tune the window to the system's propagation delay

Viva theory

Event correlation. Combining many raw events from monitoring sources into fewer, meaningful incidents. Benefits: less alert fatigue, faster root-cause analysis, and one ticket per problem. Techniques: filtering, deduplication, suppression (during maintenance), aggregation or counting, time-window correlation, topology-based (dependency) correlation, rule or pattern based correlation, and statistical or machine-learning approaches (AIOps).

Primary vs secondary events. A primary (root-cause) event starts the chain; secondary events are symptoms caused by it. Alert on the cause and attach the symptoms to the same incident; do not page separately for every symptom.

Deduplication vs correlation. Deduplication merges identical events (same source and message). Correlation relates different events that belong to the same incident.

Topology awareness. Knowing dependencies (a CMDB, service map, or network diagram) lets a tool suppress downstream alerts when an upstream device is down. For example, if a switch is down, do not alert on every server behind it.

Alert fatigue. Too many low-value alerts cause people to ignore real ones. Reduce with thresholds with duration, deduplication, correlation, and clear ownership and severity.

Root cause analysis (RCA). Methods: timeline analysis, 5 Whys, fishbone diagrams, change correlation (what changed before the failure?). The goal is the underlying cause, not the symptom.

Automation and IaC. Automating routine remediation reduces human error and time to recover (MTTR). **Ansible** is agentless (SSH), uses human-readable YAML, and is idempotent. Alternatives: Puppet and Chef (agent-based), Terraform (provisioning).

Check mode. A dry run that predicts changes without applying them; modules that do not support it are skipped or report unchanged. With `--diff` it shows file differences. Not all tasks are accurately predicted (commands cannot be simulated).

Safe automation. Test in a sandbox first, use check mode, limit the blast radius, have rollback, require approval for production, and log all runs. Self-healing automation should have guardrails (rate limits and escalation).

Metrics. MTTD (mean time to detect), MTTA (acknowledge), MTTR (repair or recover). Good correlation lowers all three.

Typical viva questions: What is event correlation? (Grouping related events into one incident.) What is the difference between deduplication and correlation? (Identical events merged vs different but related events grouped.) How did you decide the root cause? (Earliest event on the lowest dependency layer, confirmed by direct checks.) What does Ansible check mode do? (Simulates changes without making them.) What does idempotent mean? (Running it again produces no change once the desired state exists.) Which flag shows file differences? (`--diff`.) How do you limit a run to one host? (`-l HOST`.) How do you verify the playbook is correct before running? (`--syntax-check`, `--list-tasks`, `--check --diff`.) How do you know recovery worked? (Layer-by-layer checks, alerts clear, and a playbook re-run shows `changed=0`.) What is a handler? (A task that runs only when notified by a change, typically a service restart.)

Rapid-fire viva question bank

Short questions with one-line answers for last-minute revision. The lab number tells you which section has the full explanation. Read the question, answer aloud, then check.

Cross-lab concepts

Lab	Question	Short answer
All	What are the three pillars of observability?	Metrics (numbers over time), logs (events), traces (request paths)
All	Monitoring versus observability?	Monitoring checks known failure modes with predefined metrics and alerts; observability lets you ask new questions of the system from its telemetry

Lab	Question	Short answer
All	Agent-based versus agentless monitoring?	Agent: software on the host (Zabbix agent, Wazuh agent, Node Exporter), rich data; agentless: remote protocols such as SNMP, SSH, ICMP, HTTP, nothing installed
All	Push versus pull collection?	Pull: the server scrapes targets (Prometheus, Zabbix passive); push: the source sends (syslog, Zabbix active, Pushgateway)
All	What is MTTD, MTTR?	Mean time to detect, mean time to repair or recover
All	Why alert on symptoms, not causes?	Symptoms are what users feel and are fewer; cause-based alerts are noisy and may never affect anyone
All	What is alert fatigue and how is it reduced?	Too many low-value alerts get ignored; use thresholds with duration, dedup, correlation, severity and ownership
All	What is a baseline?	The normal behavior over time, used to judge whether a value is abnormal
All	Static threshold versus dynamic or anomaly detection?	Fixed limit versus a learned pattern that adapts to time of day and trend
All	What is hysteresis in alerting?	Using a lower recovery threshold than the trigger threshold to stop flapping
All	USE method?	Utilization, Saturation, Errors for each resource
All	RED method?	Rate, Errors, Duration for each service
All	Four golden signals?	Latency, errors, traffic, saturation (LETS)

Labs 1 to 4

Lab	Question	Short answer
1	What does load average of 4 on a 4-core machine mean?	The CPUs are fully used on average, no queue; above the core count means waiting work

Lab	Question	Short answer
1	Which `top` field shows time waiting for disk?	`wa` (iowait)
1	How do you tell a disk bottleneck from a CPU bottleneck?	High `wa`, high `await` and `%util` in `iostat`, many blocked processes (`b` in vmstat) versus high `us+sy` and a long run queue
1	Why use "available" rather than "free" memory?	Linux uses spare memory for cache, which is reclaimable; available estimates what can be allocated without swapping
1	What do `si` and `so` in `vmstat` mean?	Memory swapped in from, and out to, disk; nonzero `so` shows memory pressure
1	Which command shows per-device disk statistics?	`iostat -xz 1`
1	What is `sar` and what does it add over `top`?	Collects and stores history (sysstat), so you can look at the past
2	What is SNMP and which port?	Simple Network Management Protocol for managing devices; UDP 161 (traps on 162)
2	What is an OID and a MIB?	OID: numeric address of a variable; MIB: a file describing and naming the OIDs
2	Why use `ifHCInOctets`?	64-bit counter; a 32-bit counter wraps in about 34 seconds at 1 Gbps
2	How do you compute interface utilization?	(delta octets x 8) / (seconds x ifSpeed) x 100, using sysUpTime for the interval
2	How do you handle counter wrap?	Add 2^32 (or 2^64) to the delta when the new value is smaller
2	Difference between v1, v2c and v3?	v1/v2c use a clear-text community; v3 adds authentication and encryption; v2c adds GETBULK and 64-bit counters
2	`snmpget` versus `snmpwalk`?	One OID versus a whole subtree (GETNEXT/GETBULK repeatedly)
3	Facility versus severity in syslog?	Source category (auth, kern, local0) versus importance 0 to 7
3	How is the PRI value calculated?	Facility x 8 + severity
3	`@` versus `@@` in rsyslog?	UDP versus TCP forwarding

Lab	Question	Short answer
3	Why centralize logs?	Search and correlate in one place, tamper resistance, retention, alerting
3	How do you test forwarding?	`logger` with a unique marker and grep it on the server
3	Which file holds SSH login events on Ubuntu?	`/var/log/auth.log`
3	What severity is a failed SSH login logged at?	info, so filter by content, not severity
4	Zabbix server and agent ports?	10051 and 10050
4	Item, trigger, template, action?	Metric, condition on metric data, reusable bundle, response to an event
4	Passive versus active checks?	Server polls the agent versus the agent pulls its checks and pushes data
4	What does `Hostname=` in the agent config do?	Must match the frontend host name for active checks
4	Zabbix 6 trigger expression for CPU above 80% averaged over 5 minutes?	`avg(/host/system.cpu.util,5m)>80`
4	How do you test a trigger safely?	Lower the threshold temporarily, send a value with `zabbix_sender`, or run bounded `stress-ng`

Labs 5 to 8

Lab	Question	Short answer
5	How does LibreNMS find devices?	SNMP polling with auto-discovery (also ping, CDP/LLDP, ARP)
5	Poll interval default?	5 minutes
5	Errors versus discards?	Errors: damaged frames (cabling, duplex); discards: valid frames dropped, usually congestion or policy
5	`ifOperStatus` versus `ifAdminStatus`?	Actual link state versus configured state

Lab	Question	Short answer
5	What does high errors at low utilization indicate?	Physical-layer fault
5	Why use delay in an alert rule?	To ignore short spikes and avoid flapping
6	What are Wazuh's main components?	Agent, manager, indexer, dashboard
6	What is a decoder versus a rule?	Decoder extracts fields from logs; rule decides whether an alert is raised
6	Custom rule IDs range?	100000 and above
6	Which tool tests a log line against the rules?	`wazuh-logtest`
6	What is active response?	Automatic command (for example a firewall block) triggered by a rule
6	What does `frequency` and `timeframe` do?	Fire after N matching events within T seconds
6	What do you do after a successful login that follows many failures?	Treat as compromise: contain, lock account, investigate, reset credentials
7	ELK components?	Elasticsearch, Logstash, Kibana (plus Beats)
7	Loki indexes what?	Only labels, not the full log text
7	Error rate formula?	5xx responses divided by all responses, times 100
7	502 versus 503 versus 504?	Bad response from upstream, service unavailable or overloaded, upstream timeout
7	Why not use IP as a Loki label?	High cardinality creates too many streams
7	How do you filter 5xx in LogQL?	`{job="nginx"} |~ " 5[0-9]{2} "`
8	Prometheus ports and model?	9090, Node Exporter 9100; pull model with time-series and labels
8	Why `rate()` on a counter?	Counters only grow; rate converts them to a per-second speed
8	CPU utilization PromQL?	`100 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100`

Lab	Question	Short answer
8	`rate` versus `irate`?	Average over the window versus last two samples (spiky)
8	What does `up` mean?	1 if the last scrape succeeded, 0 if not
8	Role of Alertmanager?	Dedup, group, silence and route Prometheus alerts
8	What does `for: 2m` do in an alert rule?	Requires the condition to hold for 2 minutes before firing

Labs 9 to 12

Lab	Question	Short answer
9	`time_starttransfer` measures?	Time until the first response byte (cumulative from the start)
9	TLS handshake time from curl?	`time_appconnect - time_connect`
9	Why percentiles instead of averages?	Averages hide the slow tail that users feel
9	SLI, SLO, SLA, error budget?	Measurement, internal target, contractual promise, 100% minus SLO
9	Allowed downtime at 99.9% per 30 days?	About 43 minutes
9	What is burn rate?	Speed at which the error budget is used compared with the allowed rate
9	Which signal is saturation?	How full the constrained resource is, for example CPU run queue or connection pool
10	Smallest deployable unit in Kubernetes?	A pod
10	Why `kubectl logs --previous`?	Shows the logs of the container that crashed before the restart
10	Exit code 137?	SIGKILL, commonly OOMKilled
10	CrashLoopBackOff?	Container keeps failing and the kubelet restarts it with growing delay
10	Requests versus limits?	Scheduler reservation versus run-time cap (CPU throttles, memory kills)

Lab	Question	Short answer
10	What does `kubectl top` need?	metrics-server
10	How to roll back a bad deployment?	`kubectl rollout undo deployment/NAME`
11	Modbus TCP port and addressing?	502; coils, discrete inputs, input registers, holding registers
11	Which function reads holding registers?	Function 3 (4 for input registers)
11	Why is Modbus insecure?	No authentication or encryption
11	OPC UA versus Modbus?	OPC UA is typed, browsable, secure and event-capable; Modbus is flat registers
11	ASHRAE recommended inlet temperature?	18 to 27 degrees C
11	Fast temperature rise with constant power means?	Cooling failure, not electrical overload
11	A value of 65500 for a temperature register?	Signed 16-bit value read as unsigned: subtract 65536
12	Deduplication versus correlation?	Merge identical events versus group related different events
12	How do you pick the root cause in a cluster?	Earliest event on the lowest dependency layer, confirmed by direct checks
12	What does `ansible-playbook --check` do?	Simulates changes without applying them
12	What does idempotent mean?	Re-running makes no further change once the state matches
12	What does `--diff` show?	The changes to files that would be made
12	How do you verify recovery?	Layer checks, alerts clear, and a playbook re-run reports changed=0

Common "trap" questions

Lab	Question	Short answer
1	A host has load average 7 on 4 cores but low CPU use. Is it CPU-bound?	Not necessarily: load counts processes blocked on I/O too; check `wa` and `b` first
3	Why can alerting on severity err miss brute force?	SSH failures are logged at info
4	Zabbix agent is running but the host shows red. Why?	Server IP not in `Server=`, firewall on 10050, or wrong interface IP
5	Interface at 3% utilization is raising alerts for errors. Is it congested?	No, low load with errors is a physical problem
7	Average error rate is low but users complain. Why?	A short burst or one endpoint is hidden by a long window or by aggregation
8	CPU shows 83% but the bottleneck is disk. How?	`100 - idle` counts iowait as busy; read `iowait` and disk busy time
9	p50 is flat but users are unhappy. What next?	Check p95 and p99 and the error rate
10	Pod is Running but not serving traffic. Why?	Readiness probe failing (READY 0/1)
11	Sensor shows the same value for an hour. Is it stable?	It may be frozen or stale; check timestamps and a second sensor
12	Four alerts at once: do you have four problems?	Probably one root cause and three secondary symptoms